

队伍编号	
题号	D

基于分层预测与贪心-MG 算法的短途物流动态调度模型及应用

摘 要

在物流末端短途运输场景中，货量预测面临预知数据存在计划线路异常、未涵盖未下单货量、已下单货量可能取消等多重挑战。如何通过构建数据驱动的货量预测模型与多目标资源调度优化模型，同时在满足时效要求的前提下，实现自有车辆资源的高效周转、运输成本的最小化及整体运营效率的提升，成为当前短途物流运输优化的核心问题。

针对问题一，我们首先进行了数据的可视化分析，发现日总包裹量趋势总体稳定但略有波动，对于 10min 颗粒度包裹量呈现出明显的**季节性和非线性**的变化特征，接着我们进行了数据整合与清洗，特征提取，接着建立了 **Gradient Boosted Regression Tree-Gradient Boosted Regression 分层预测模型**首先通过历史总包裹量和预知包裹量预测出了总包裹量，进而对于 10min 颗粒度包裹量预测出了占当日总量的比例，按比例日总量拆分为 10min 颗粒度，预测 R 方高达 0.99，得到“场地 3-站点 83-0600”和“场地 3-站点 83-1400”的日总包裹量分别为 **7956 和 2553**，对颗粒度包裹绘制趋势图发现总体符合结果，同时我们通过与随机森林，LSTM，SARIMA 等多种模型进行比较，并对模型进行**敏感度分析**，结果显示我们的模型表现优越，性能稳定。

针对问题二，我们首先根据问题确定了相关的决策变量，并构建相应的串点集（二联串点 174，三联串点 152），同时以最大化自有车周转率，最大化车辆均包裹率，最小化总成本（二联串点，三联串点，独立线路）为目标函数，以发运时间，车辆装载量，自有车辆周转和串点数量为约束条件建立优化模型，利用**贪心策略结合 Misra & Gries 算法（贪心 MG）**由高峰时段开始分配求解出**最终方案总成本为 61880 元，自有车辆利用率为 68.85%，平均周转率为 408%**。同时与模拟退火，整数规划等算法从时间复杂度和结果准确度进行对比，本算法具有**效果好，速度快**的特点。

针对问题三，我们对于决策变量进行扩展，适配串点策略，调整车辆装载量，自有车辆周转约束条件，从时间效益，包裹利用率，路线时间，包裹数量，串点任务考量和高峰期决策六个角度确定标准容器使用策略，利用贪心 MG 进行求解，得到**最终方案总成本为 54158 元，自有车辆利用率为 72.83%，平均周转率为 435%，容器使用占比为 21.48%**，与问题二相比均有所改进。

针对问题四，在问题三的基础上我们对于问题 1 的预测结果引入随机误差，系统性误差，周期性误差，时间偏移误差，突发性误差，蒙特卡洛模拟**六种扰动模型**，以总包裹量，车辆需求，运输成本，自有车周转率，自有车使用率，容器使用率，串点率七个指标来评价我们的模型的鲁棒性，发现所有指标的**平均敏感性均不超过 10%**，总包裹量对于扰动最为敏感（8.15%），串点率对于其敏感度最低（4.07%），本模型的鲁棒性较强。

关键词：Gradient Boosted Regression Tree 分层预测模型，贪心 MG，敏感度分析，多模型对比

目录

一、 问题重述	1
1.1 问题背景	1
1.2 问题要求	1
二、 问题分析	1
2.1 问题一的分析	1
2.2 问题二的分析	2
2.3 问题三的分析	2
2.4 问题四的分析	2
三、 模型假设	3
四、 符号说明	3
五、 问题一模型建立与求解	3
5.1 数据可视化	3
5.2 数据预处理	5
5.3 时序特征工程（多维度时间与滞后特征）	5
5.4 模型建立	5
5.4.1 第一阶段：每日总货量预测	5
5.4.2 第二阶段：时间分布预测（10 分钟颗粒度占比）	6
5.4.3 第三阶段：10 分钟颗粒度货量拆分	6
5.5 模型求解	6
5.6 模型评价与敏感性分析	9
六、 问题二模型建立与求解	13
6.1 模型建立	13
6.2 模型求解	17
七、 问题三模型建立与求解	19
7.1 模型建立	19
7.2 模型求解	21
八、 问题四模型建立与求解	22
8.1 预测误差扰动模型构建	22
8.2 模型求解	24
九、 模型评价	26
十、 参考文献	26
十一、 附录	27

一、问题重述

1.1 问题背景

短途运输作为物流末端配送的核心环节，具有运输半径短、资源复用性强、时效敏感性高等特征，其运营效率直接影响履约时效性、客户满意度及企业成本结构。而现行运输体系以分拣中心为起点、营业部为终点的两级节点网络为基础，执行每日 06:00 和 14:00 双发运节点的定时运输机制，面临多维运营挑战：

1.需求预测不确定性：现有货量预测模型存在一定偏差，具体表现为线路规划与实操场景偏离、未下单潜在货量未被纳入预测体系、已下单订单存在取消风险，导致运力资源配置失准等问题。

2.车辆调度复杂性：运输需求满足度受限于自有车队动态周转效率，需综合考量车辆装载率（包裹/车）、任务衔接时效性等关键指标。当运力缺口超过自有车队服务能力时，需启动高成本外协运力，形成成本管控与服务质量的双重约束。

3.成本效益均衡难题：在行业竞争加剧背景下，需通过优化发运时序规划、提升车辆周转效率、完善需求预测模型等策略，实现运输成本集约化与运营效能的提升。

因此为了更好地满足运输市场需求，达到物流企业运营成本与服务质量的协同优化目标，有必要构建实际数据驱动的短途物流优化模型，通过精准需求预测和智能调度算法，进而解决运力资源配置的动态适配问题。

1.2 问题要求

附件 1、2、3、4、5 所给出的是各车队负责线路的在途时长、变动成本、各线路历史 15 天实际包裹量和每天和未来 1 天 21 点时的预知货量、可串点具体站点、各车队自有车数量等信息。此外，所有车队车型统一，日固定成本 100，单次装载 1000 个包裹，装/卸车时长均为 45 分钟。为优化物流运输调度分配作出合理决策，现需结合实际情况与上述数据建立数学模型分析以下问题：

问题一：建立合适的货量预测模型，对未来一天（12 月 15 日 14:00 至 12 月 16 日 14:00）不同线路包裹总运输量进行预测，并进一步将每条线路预测货量细分到十分钟颗粒度。

问题二：根据问题一得到的每条线路货量结果合理安排车辆运输方案，包括发运车辆数、发运时间和串点方案等，其中每个运输需求的发运时间不能晚于线路的发运节点。同时做到自有车的周转率、所有车辆均包裹尽可能高，总成本尽可能低。

问题三：在问题一和问题二的基础上引入一种可以提升自有车辆利用率的标准容器，可使装/卸车时长缩短为 10 分钟，但同时也会使车辆装载包裹量下降至 800 个。要求在假定该容器数量无限的情况下根据问题一的结果重新确定问题二的运输需求，并给出是否使用该容器。

问题四：若问题一的货量预测结果出现一定偏差时，基于问题三评估对给出的调度模型优化结果的影响。

二、问题分析

2.1 问题一的分析

针对问题一，首先每日总包裹量变化趋势图和十分钟颗粒度包裹量折线图来观察数

据分布和变化规律，接着进行相应的数据预处理，进行数据整合和清洗，特征提取。观察数据日总包裹量和 10min 颗粒度包裹量呈现出的数据特征作为我们后续预测模型选择的依据，同时将两者相互联系建立一个合适的分层预测方案，同时我们通过与随机森林，LSTM，SARIMA 等多种模型进行比较，并对模型进行敏感度分析以进一步分析模型的性能。

2.2 问题二的分析

针对问题二，首先根据问题确定了相关的决策变量，并构建相应的串点集（二联串点 174，三联串点 152），同时以最大化自有车周转率，最大化车辆均包裹率，最小化总成本（二联串点，三联串点，独立线路）为目标函数，以发运时间，车辆装载量，自有车辆周转和串点数量为约束条件建立优化模型，同时通过问题一的结果可以看出数据实际上呈现出明显的高峰时段和平缓时段，因此我们引入 Misra & Gries 算法识别高峰区域并结合贪心策略进行调度分配，同时与与模拟退火，整数规划等算法对于求解结果进行多指标的对比，判断算法的实际表现和性能。

2.3 问题三的分析

针对问题三，我们对于决策变量进行扩展，适配串点策略，调整车辆装载量，自有车辆周转约束条件，从时间效益，包裹利用率，路线时间，包裹数量，串点任务考量和高峰期决策六个角度确定标准容器使用策略，利用贪心 MG 进行求解得到最终结果。

2.4 问题四的分析

针对问题四，在问题三的基础上我们对于问题 1 的预测结果引入随机误差，系统性误差，周期性误差，突发性误差，蒙特卡洛模拟六种扰动模型，以总包裹量，车辆需求，运输成本，自有车周转率，自有车使用率，容器使用率，串点率七个指标来评价我们的模型的鲁棒性，并对问题结果进行相应的可视化分析，以展示不同扰动情况下模型的直观表现。

文章总体思路图如下：

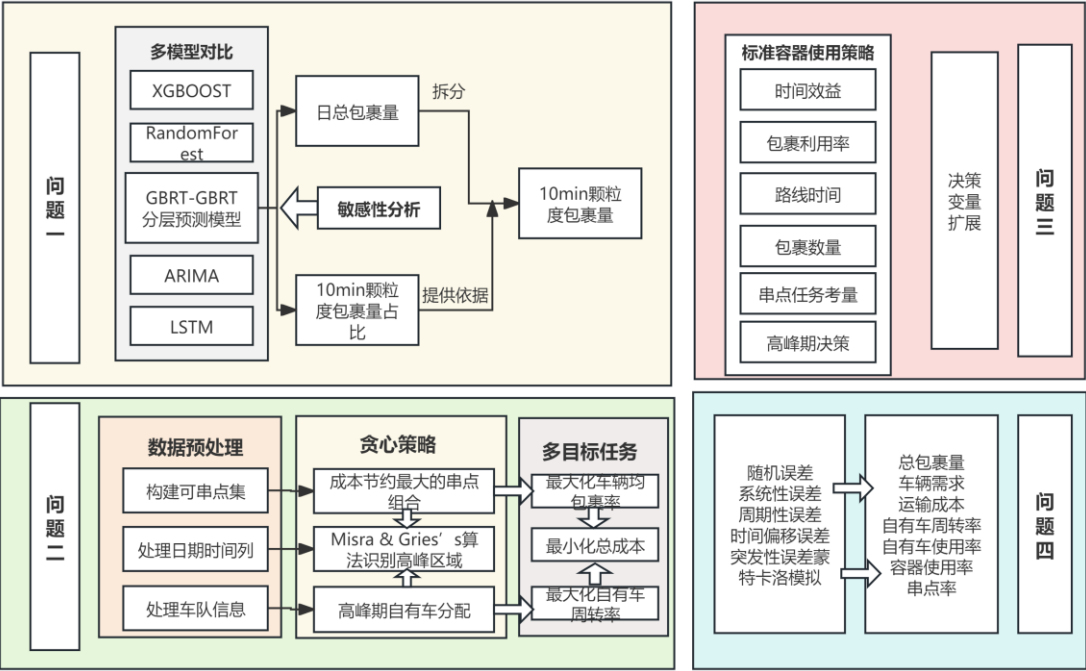


图 1 文章总体思路流程图

三、模型假设

- 1.假设历史 15 天的包裹量数据（附件 2）能够充分反映各线路的货量波动规律，包括季节性、周期性及异常波动特征，未来 1 天的货量模式与历史数据具有一致性，不存在未被历史数据覆盖的极端异常情况。
- 2.假设所有线路的运输需求必须在指定发运节点（06:00 和 14:00）前完成发运，且发运时间以 10 分钟颗粒度划分，忽略分钟级的微小时间误差。
- 3.假设自有车辆在完成前一个运输任务后，返回分拣中心的时间仅受装卸时间、在途时间影响，不考虑车辆故障、交通拥堵等外部干扰因素，确保周转约束公式（前任务发运时间+装卸时间+往返在途时间<后任务发运时间）的严格成立。
- 4.假设车辆日固定成本、单次变动成本（附件 1）及外部承运成本已知且稳定，不随运输时段、货量波动或串点方案变化而改变。

四、符号说明

符号	说明
$Q_{history}$	各线路近 15 天的每天预知货量及未来 1 天在 21 点时的预知货量
$Q_{adjusted}$	各线路校准后的预知货量
TR_{SV}	自有车周转率
AV_P	车辆均包裹率
T_{fixed}	每辆自有车每日的固定成本
$t_{loading}$	表示自有车一次运输需求装运时间
t_{route}	表示自有车单程在途时间
$t_{unloading}$	表示自有车一次运输需求卸车时间

五、问题一模型建立与求解

问题分析：问题一是一个预测类问题，首先我们应该对于数据进行相应的可视化处理，分析数据的波动情况和分布状态，同时题目中要求我们对于每日的总货量和颗粒度都进行相应的预测，这启示我们应该寻找两者之间的关系并根据可视化结果选取相关的变量建立一个合适的预测模型。

5.1 数据可视化

为更好地分析和理解数据，我们以题目中要求的“场地 3-站点 83-0600”和“场地 3-站点 83-1400”两条线路为例利用附件 2 和附件 3 中的线路和货量信息进行数据可视化分析来观察预知总货量，历史总货量（10min 颗粒度求和）和十分钟颗粒度包裹量的数据分布和变化规律。我们利用 Python 分别绘制了预知总货量和历史总货量的日总包裹量的变化情况如图 2 所示，十分钟颗粒度包裹量的变化情况如图 3 所示。

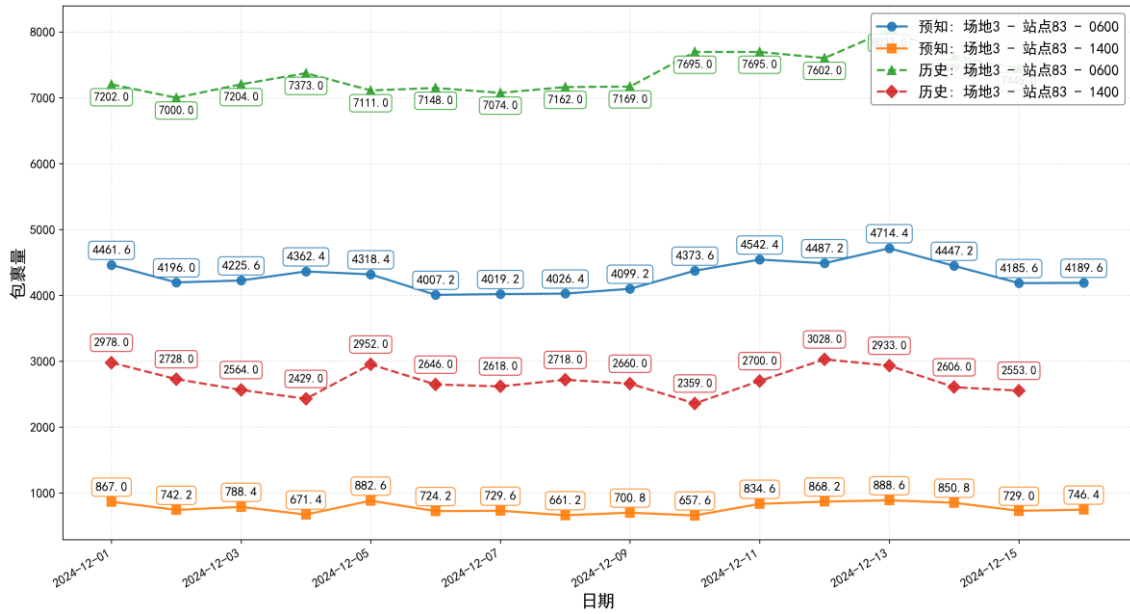


图2 场地3 站点 83 - 0600 与 83 - 1400 每日总包裹量变化趋势图

由图2可知，“场地3-站点83-0600”日总包裹量整体处于较高水平，每日在4000左右波动；“场地3-站点83-1400”包裹量较低，基本在数百区间波动。可以看出，日总包裹量总数的影响实际上是由于两条线路的产生包裹的持续时间不同（前者为前一天的21:00至当天的5:50；后者为当天的11:00至13:50），但总体趋势趋于平稳。对于预知包裹数和历史包裹数的对比，实际上题目中所提出的1）计划线路可能与实际不符，即发运异常；2）未涵盖部分未下单的货量；3）已下单的货量可能会取消。三种问题还是对包裹数量的变化产生了一定的影响。

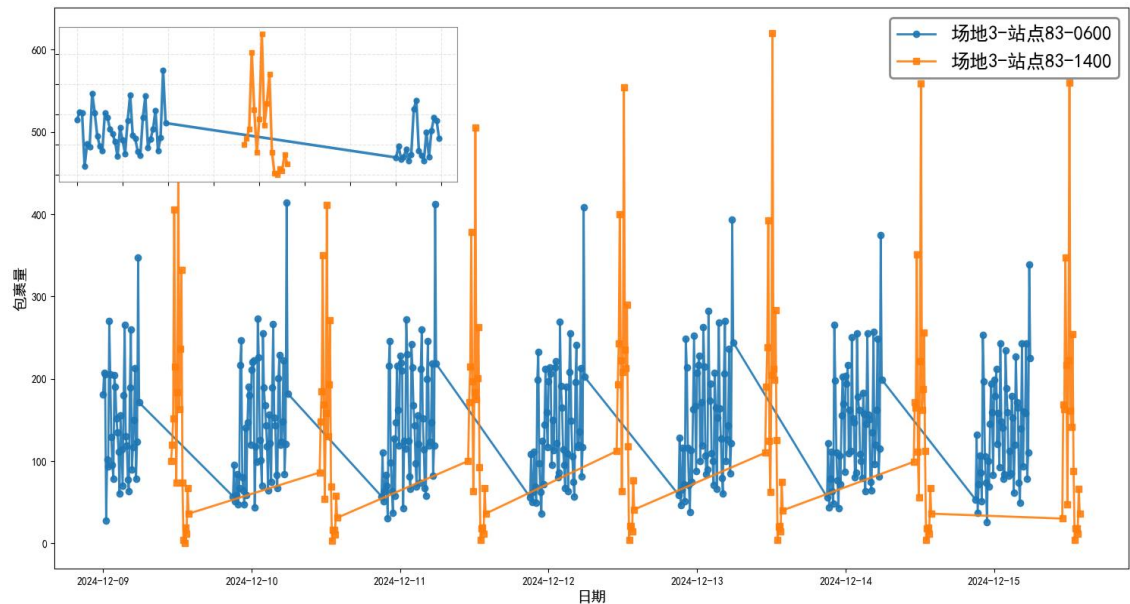


图3 场地3 站点 83 - 0500 与 83 - 1400 十分钟颗粒度包裹量折线图

由图3可知，无论哪条线路的包裹量均呈现出季节性（周期性）和非线性波动模式，在每天6点和21点固定时段出现包裹量高峰，与物流分拣班次节点高度吻合，为我们后续的模式选择提供了一定的依据。

5.2 数据预处理

- 数据整合与清洗：为解决预测货量时线路和时间格式不同的问题，我们对附件 2 和附件 3 的线路编码空格进行清洗，同时对时间序列统一合并转化为 `datetime` 格式，确保时间粒度的一致性，消除原始数据实验误差。
- 时间窗口划分：根据线路编码后缀我们将时间窗口类型划分为两类：1.跨午夜线路（0600）：前一天 21:00-23:50+当天 00:00-05:50（共 9 小时）2.有效时段为当天 11:00-13:50（共 3 小时），确保模型仅学习真实业务时段的货量规律，避免无关时段干扰。
- 时间粒度占比转换：对每个 10 分钟时段，计算出其货量占当日总货量的比例作为时间分布预测的标签。

5.3 时序特征工程（多维度时间与滞后特征）

为捕捉货量波动规律，我们接着构建多维度时间序列周期性特征工程，具体步骤如下：

- 基础时间特征：根据自然日期时间分组，提取出小时、分钟、工作日、周末等基础时间特征；添加时间段（上午/下午/晚上/夜间）和工作时段标志，捕捉时段规律性。
- 滞后特征（时间序列相关性）：添加前 1 天和前 7 天同时间段的货量数据作为滞后特征，得到两者同时段占比 r_{t-1} 和 r_{t-7} ，以捕捉货量在 10 分钟颗粒度上的短期和长期时序依赖，为后续模型提供细粒度的历史同期特征输入。
- 预知数据融合：为避免直接使用原始预知量引入偏差，方便后续模型加权调整，我们将附件 3 的预知货量按线路和日期匹配到历史货量数据生成预知总量列，计算预知货量（ Q_{known} ）实际比例（预知货量 Q_{known} / 实际货量 Q_{actual} ）。

5.4 模型建立

为更加精准适配物流场景的多时间尺度需求，在预测精度、业务对接、模型效率三方面实现平衡，我们建立 Gradient Boosted Regression Tree-Gradient Boosted Regression Tree（GBRT-GBRT）分层预测模型，具体步骤如下：

5.4.1 第一阶段：每日总货量预测

为精准预测目标时间窗口（12 月 15 日 14:00 至 12 月 16 日 14:00）内各线路总货量，我们提出基于梯度提升回归树（Gradient Boosted Regression Tree, GBRT）的融合预测模型，通过集成历史时序特征与预知数据校准机制提高预测精度，具体架构如下：

首先采用 GBRT 算法建模，其目标函数可表述为：

$$\hat{Q}_{total} = \arg \min_Q \sum_{i=1}^N L(Q_{actual,i}, f_{GBRT}(\mathbf{X}_i)) + \lambda \Omega(f_{GBRT}), \quad (1)$$

其中 $\mathbf{X}_i$ 为特征向量（包括时间维度、滞后特征融合、历史预测货量比例等时序特征）， L 为均方误差损失函数， Ω 为复杂度惩罚项， λ 为正则化系数。

最终我们将日总货量的输入值选取为历史模型输出 $Q_{history}$ 与校准预知货量 $Q_{adjusted}$ 的加权平均。校准和最终总货量预测公式如下：

$$\bar{\rho} = \frac{1}{n} \sum_{i=1}^n \frac{Q_{known,i}}{Q_{actual,i}}, \quad (2)$$

$$Q_{adjusted} = \frac{Q_{known}}{\bar{\rho}}, \quad (3)$$

$$Q_{final} = \alpha Q_{history} + (1 - \alpha) Q_{adjusted}, \quad (4)$$

其中 $\bar{\rho}$ 为基于历史同期数据预知货量与实际货量的平均比例, $Q_{adjusted}$ 为校准后的预知货量, 权重系数 $\alpha \in [0, 1]$, 通过网格搜索优化确定, 坚持对预知数据的高置信度原则。

5.4.2 第二阶段: 时间分布预测 (10 分钟颗粒度占比)

为预测各 10 分钟时段的货量占当日总量的比例, 我们提出分时段动态建模方法, 具体流程如下:

- 时间分箱编码: 将全天时间轴划分为 144 个 10 分钟时段, 即 $T_{bin} \in [0, 143]$, 定义对应映射关系如下:

$$T_{bin} = 6h + \left\lfloor \frac{m}{10} \right\rfloor, \quad (5)$$

其中 h 为小时数 ($h \in [0, 23]$), m 为分钟数 ($m \in [0, 10, 20, 30, 40, 50]$)。

- 接着计算每个分箱货量占当日总货量的比例, 再对每个有效时段 T_{bin} 独立训练 GBRT 算法子模型, 其输入特征为 $X_{T_{bin}} = [week, weekend, r_{t-1}, r_{t-7}]$, 输出目标为该时段货量占比 $\hat{r}_{T_{bin}}$, 同时对占比归一化处理使其总和为 1, 避免累计误差。归一化公式如下:

$$\hat{r}_{T_{bin}}^{norm} = \frac{\hat{r}_{T_{bin}}}{\sum_{k=0}^{143} \hat{r}_k}, \quad k \in \{0, 1, \dots, 143\} \quad (6)$$

- 针对历史数据量 nT_{bin} 十分钟时段不足的情况 (若有缺失值) 我们采用邻近时段插补的方法取相邻 ± 1 小时内同星期属性的时段占比均值:

$$r_{T_{bin}} = \frac{1}{|S|} \sum_{s \in S} \bar{r}_s, \quad S = \{s \mid |s - T_{bin}| \leq 6\} \quad (7)$$

5.4.3 第三阶段: 10 分钟颗粒度货量拆分

结合上述两个阶段和线路运营约束生成最终 10 分钟颗粒度货量预测, 具体流程如下:

- 有效时间点生成: 根据不同线路时间窗口, 生成预测时段内的所有 10 分钟时间点 (如 0600 线路生成 12 月 15 日 21:00-23:50 和 12 月 16 日 0:00-5:50 的时间点)。
- 货量分配与校准: 将时间点映射到分箱编码, 结合每日总预测货量和分箱占比, 计算各时段预测货量:

$$\hat{q}_{T_{bin}} = Q_{final} \hat{r}_{T_{bin}}^{norm}, \quad (8)$$

其中 $\hat{q}_{T_{bin}}$ 为时间分箱的时段预测货量, Q_{final} 为日级总货量预测值, $\hat{r}_{T_{bin}}^{norm}$ 为时间分箱的归一化占比。最终通过四舍五入和逐行调整确保总货量与日级预测一致

5.5 模型求解

在第一阶段每日总货量预测模型中我们设置梯度提升回归树数量为 100 棵并设置了随机种子, 最终总货量的权重参数 α 取 0.3, 在第二阶段时间分布预测中树数量设置为 50 棵, 同样设置随机种子, 保证实验的可重复性, 其余参数如学习率 (learning_rate=0.1)、最大深度 (max_depth=3) 等均使用模型默认参数。接着我们利用 Python 预测得到 12 月 15 日 14:00 至 12 月 16 日 14:00 各条线路的总货量和拆解到 10 分钟颗粒度的货量结

果表如下（以场地 3-站点 83 - 0600 和场地 3-站点 83 - 1400 为例，其余预测结果见附件结果表 1 和表 2）：

表 1 场地 3-站点 83-0600 和场地 3-站点 83-1400 的日总货量结果表

线路编码	日期	货量
场地 3-站点 83-0600	2024/12/16	7956
场地 3-站点 83-1400	2024/12/16	2553

表 2 场地 3-站点 83-0600 和场地 3-站点 83-1400 的 10 分钟颗粒度货量结果表

线路编码	日期	分钟起始	货量
场地 3-站点 83-0600	2024/12/15	21:00:00	60
场地 3-站点 83-0600	2024/12/15	21:10:00	131
场地 3-站点 83-0600	2024/12/15	21:20:00	40
场地 3-站点 83-0600	2024/12/15	21:30:00	77
场地 3-站点 83-0600	2024/12/15	21:40:00	116
场地 3-站点 83-0600	2024/12/15	21:50:00	65
场地 3-站点 83-0600	2024/12/15	22:00:00	70
场地 3-站点 83-0600	2024/12/15	22:10:00	265
场地 3-站点 83-0600	2024/12/15	22:20:00	210
场地 3-站点 83-0600	2024/12/15	22:30:00	113
场地 3-站点 83-0600	2024/12/15	22:40:00	79
场地 3-站点 83-0600	2024/12/15	22:50:00	64
场地 3-站点 83-0600	2024/12/15	23:00:00	108
场地 3-站点 83-0600	2024/12/15	23:10:00	75
场地 3-站点 83-0600	2024/12/15	23:20:00	156
场地 3-站点 83-0600	2024/12/15	23:30:00	214
场地 3-站点 83-0600	2024/12/15	23:40:00	169
场地 3-站点 83-0600	2024/12/15	23:50:00	91
场地 3-站点 83-0600	2024/12/16	00:00:00	214
场地 3-站点 83-0600	2024/12/16	00:10:00	192
场地 3-站点 83-0600	2024/12/16	00:20:00	202
场地 3-站点 83-0600	2024/12/16	00:30:00	130
场地 3-站点 83-0600	2024/12/16	00:40:00	170
场地 3-站点 83-0600	2024/12/16	00:50:00	102
场地 3-站点 83-0600	2024/12/16	01:00:00	277
场地 3-站点 83-0600	2024/12/16	01:10:00	159
场地 3-站点 83-0600	2024/12/16	01:20:00	150
场地 3-站点 83-0600	2024/12/16	01:30:00	84
场地 3-站点 83-0600	2024/12/16	01:40:00	90
场地 3-站点 83-0600	2024/12/16	01:50:00	252
场地 3-站点 83-0600	2024/12/16	02:00:00	195
场地 3-站点 83-0600	2024/12/16	02:10:00	170
场地 3-站点 83-0600	2024/12/16	02:20:00	91
场地 3-站点 83-0600	2024/12/16	02:30:00	118

场地3-站点83-0600	2024/12/16	02:40:00	88
场地3-站点83-0600	2024/12/16	02:50:00	160
场地3-站点83-0600	2024/12/16	03:00:00	164
场地3-站点83-0600	2024/12/16	03:10:00	66
场地3-站点83-0600	2024/12/16	03:20:00	143
场地3-站点83-0600	2024/12/16	03:30:00	249
场地3-站点83-0600	2024/12/16	03:40:00	174
场地3-站点83-0600	2024/12/16	03:50:00	177
场地3-站点83-0600	2024/12/16	04:00:00	79
场地3-站点83-0600	2024/12/16	04:10:00	53
场地3-站点83-0600	2024/12/16	04:20:00	150
场地3-站点83-0600	2024/12/16	04:30:00	257
场地3-站点83-0600	2024/12/16	04:40:00	89
场地3-站点83-0600	2024/12/16	04:50:00	149
场地3-站点83-0600	2024/12/16	05:00:00	157
场地3-站点83-0600	2024/12/16	05:10:00	261
场地3-站点83-0600	2024/12/16	05:20:00	84
场地3-站点83-0600	2024/12/16	05:30:00	118
场地3-站点83-0600	2024/12/16	05:40:00	429
场地3-站点83-0600	2024/12/16	05:50:00	210
场地3-站点83-1400	2024/12/16	11:00:00	30
场地3-站点83-1400	2024/12/16	11:10:00	169
场地3-站点83-1400	2024/12/16	11:20:00	163
场地3-站点83-1400	2024/12/16	11:30:00	349
场地3-站点83-1400	2024/12/16	11:40:00	182
场地3-站点83-1400	2024/12/16	11:50:00	48
场地3-站点83-1400	2024/12/16	12:00:00	222
场地3-站点83-1400	2024/12/16	12:10:00	561
场地3-站点83-1400	2024/12/16	12:20:00	161
场地3-站点83-1400	2024/12/16	12:30:00	168
场地3-站点83-1400	2024/12/16	12:40:00	255
场地3-站点83-1400	2024/12/16	12:50:00	91
场地3-站点83-1400	2024/12/16	13:00:00	4
场地3-站点83-1400	2024/12/16	13:10:00	18
场地3-站点83-1400	2024/12/16	13:20:00	18
场地3-站点83-1400	2024/12/16	13:30:00	12
场地3-站点83-1400	2024/12/16	13:40:00	66
场地3-站点83-1400	2024/12/16	13:50:00	36

观察表 1 和表 2 结果我们可以看出，场地 3-站点 83 不同时段线路的货量同样呈现出存在季节性变化与固定时段高峰值特征，我们在图 4 中具体绘制了预测结果的变化，发现与历史实际趋势较为相似，结果具有较高可靠性。同时我们绘制了具体线路包裹分布图和全部线路包裹分布热力图如下所示：

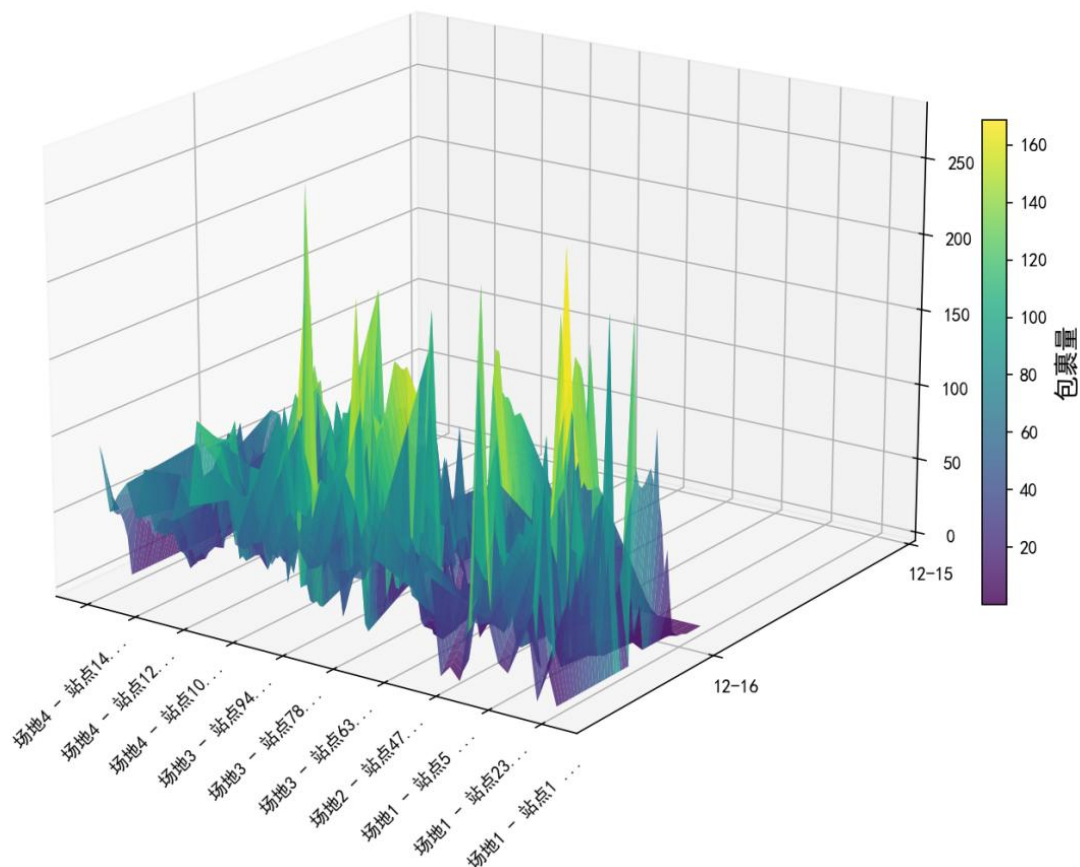


图 4 具体线路包裹量分布图

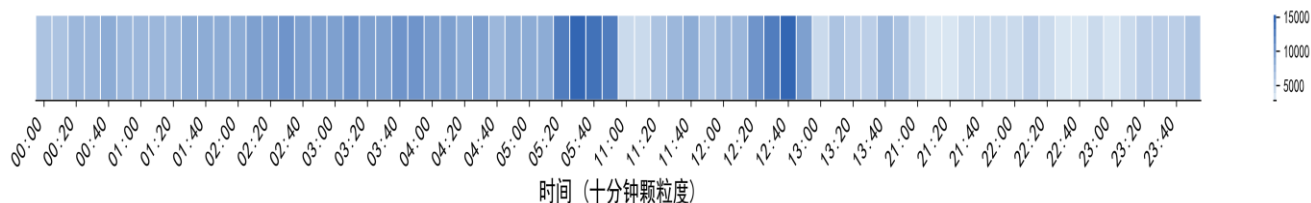


图 5 全部线路包裹量热力分布图

通过图 4 和图 5 我们可以看出总包裹数量和独立包裹产生的具体时间分布，可以确定出包裹生产的高峰期和低谷期，为我们问题二调度方案的动态选择提供合适的依据。

5.6 模型评价与敏感性分析

在模型建立和求解的过程中我们选择的是 GBRT-GBRT 模型，在求解时我们选取 $\alpha=0.3$ ，在此处我们给出通过与四个可行的预测模型进行对比及并对通过参数进行敏感性分析提供参数选择的依据，最终实验结果证明 GBRT-GBRT 模型及参数 $\alpha=0.3$ 选择均在本问预测中展示出较好的性能。

● 多预测模型比较：

实际上对于非线性和季节性的数据预测的模型有多种选择方案，如 RandomForest, SARIMA, XGBoost 和 LSTM 等，为了检验不同模型在本问题上的表现性能我们为分层预测模型建立了五种不同的组合方式（1.GBRT-GBRT；2.RandomForest-RandomForest；3.SARIMA-SARIMA；4.XGBoost-LSTM；5.XGBoost-XGBoost）各模型参数设置如表 3 所示：

表 3 各模型参数设置表

模型名称	核心参数
GBRT	树数量：100 棵 学习率：0.1 最大深度：3
RandomForest	树数量：100 棵 最大深度：10
SARIMA	季节周期：7 天（周期） 其他参数：auto_arima 自动搜索
XGBoost	树数量：100 棵 学习率：0.1 最大深度：5
LSTM	网络结构：2 层（第 1 层 24 单元，第 2 层 12 单元） Dropout：0.1 优化器：Adam 损失函数：MSE

接着以 MSE，RMSE，R2，MAE，训练时间(秒)，预测时间(秒)为指标，对于不同模型的效果进行检验并绘制相关预测折线图我们得到了以下结果如表 4、图 6 和图 7 所示：

表 4 五种模型组合预测指标结果表

模型组合	MSE	RMSE	R ²	MAE	训练时间	预测时间
1	0.0002718	0.0164854	0.9999999	0.0131459	0.7342610	0.0000001
2	19003.314	137.85251	0.9385929	125.44250	2.6252029	0.0007634
3	1974543.183	1405.18439	-5.3805160	974.68760	0.8354199	0.0017905
4	6.2374764	2.4974940	0.9999799	1.5969238	0.1124501	0.0149167
5	6.2374764	2.4974940	0.9999799	1.5969238	1.2990730	0.0171452

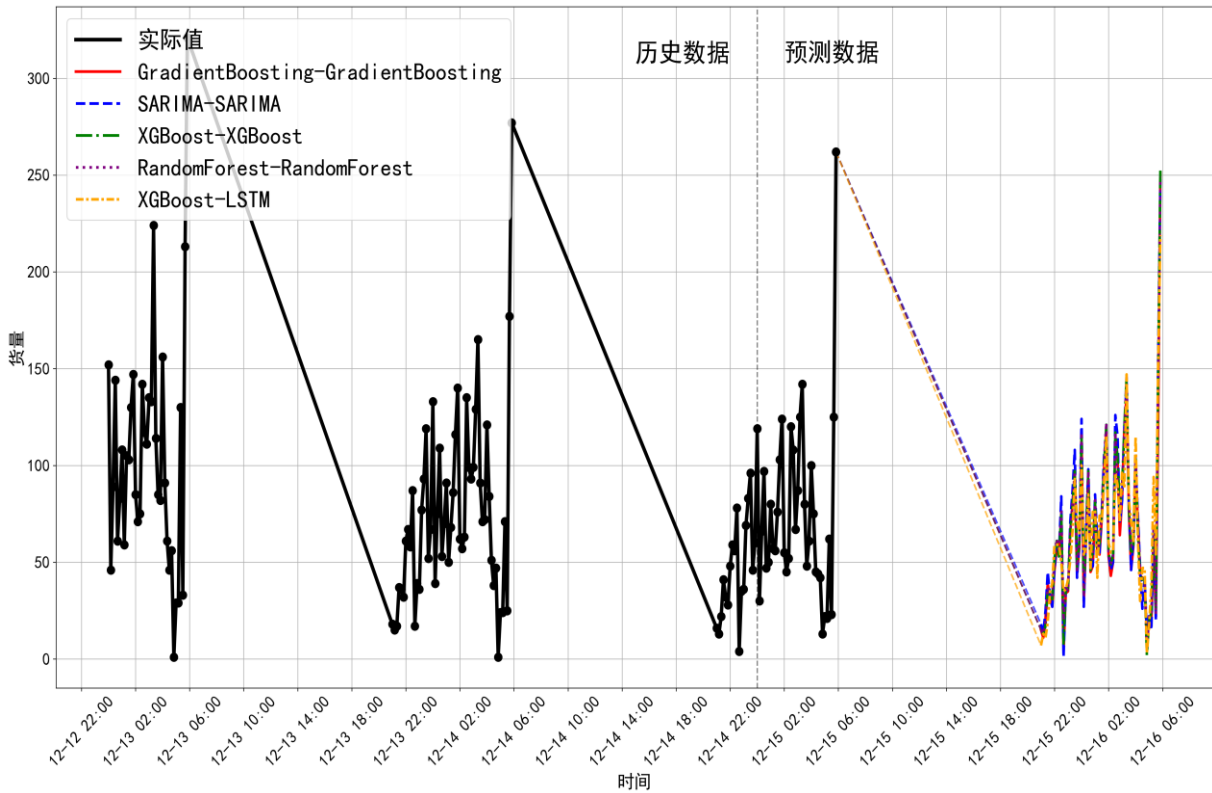


图 6 不同模型组合十分钟颗粒度预测货量折线图

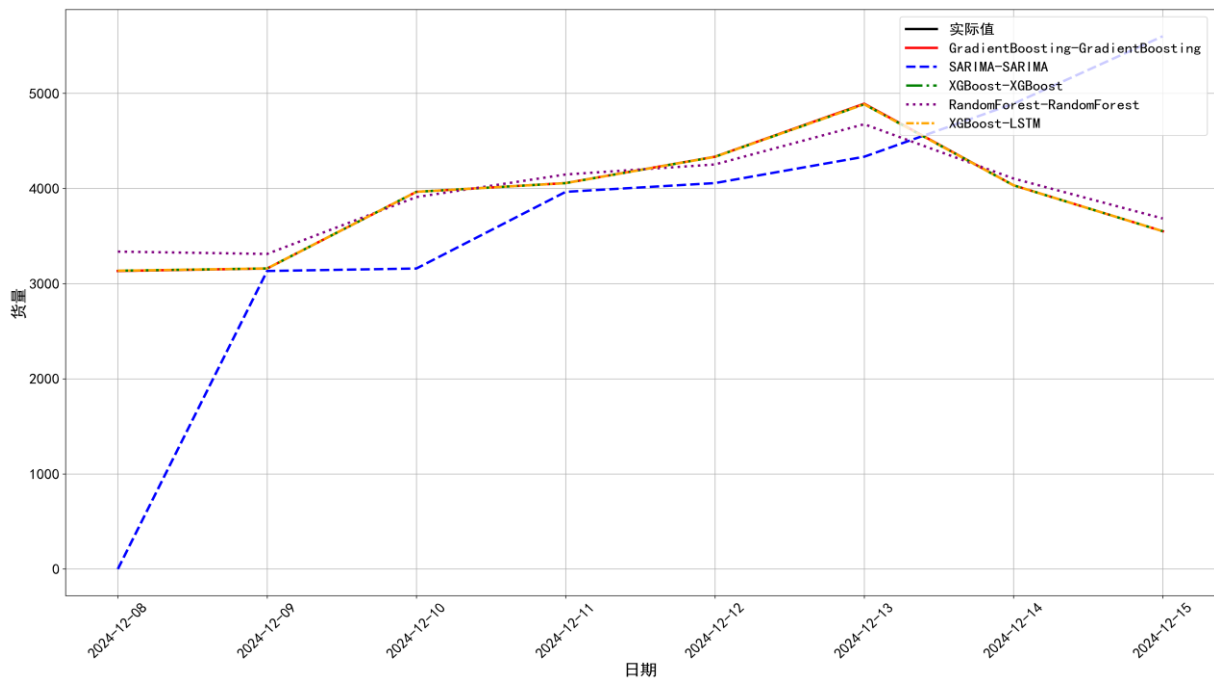


图 7 不同模型组合日总货量预测趋势图

综合表 4、图 6 和图 7 的结果来看，从预测精度角度，我们所选的 GBRT-GBRT 模型的 MSE、RMSE 显著低于其他模型， R^2 趋近于 1，MAE 极小，表明其预测值与实际值偏差极小，拟合效果较好。而其他模型如 SARIMA-SARIMA 的 R^2 为负，拟合失效。从运行效率角度，其训练时与预测时间在保持高精度的同时表现优异。综上，GBRT-GBRT 模型在预测准确性与计算效率上达到最优平衡，故为本问的最优选择。

从理论角度分析，GBRT 表现优于其他模型的核心原因在于其梯度提升机制与决策树基学习器的有机结合，能够高效适配物流货量预测中的非线性、周期性及多特征耦合场景。首先，GBRT 通过逐阶段拟合残差的梯度提升策略，可渐进式逼近复杂数据分布，对文档中提到的“季节性高峰”“预知货量校准”等非线性规律具有强建模能力，相比依赖线性假设的 SARIMA 或固定窗口的 LSTM，更能捕捉货量的不规则波动；其次，决策树基学习器天然支持类别型与连续型特征混合输入，无需人工归一化，对文档中构建的“时间分箱”“滞后特征”“预知货量比例”等多维度异构特征具有原生适配优势，能通过层级分裂自动识别关键特征组合（如跨午夜线路与日间线路的时间窗口差异）；此外，GBRT 的正则化机制（复杂度惩罚、学习率衰减）有效控制过拟合，对历史数据中的发运异常、订单取消等噪声具有鲁棒性，而 RandomForest 等模型易因基树相关性导致方差偏大，SARIMA 更因严格平稳性假设在非规则数据中拟合失效（如文档中其 R^2 为负）。

● 权重系数 α 敏感性分析

为确定最优权重系数，接下来我们探究不同历史预知货量数据权重系数对均方根误差（RMSE）和平均绝对误差（MAE）的影响得到结果如图 8 和图 9 所示：

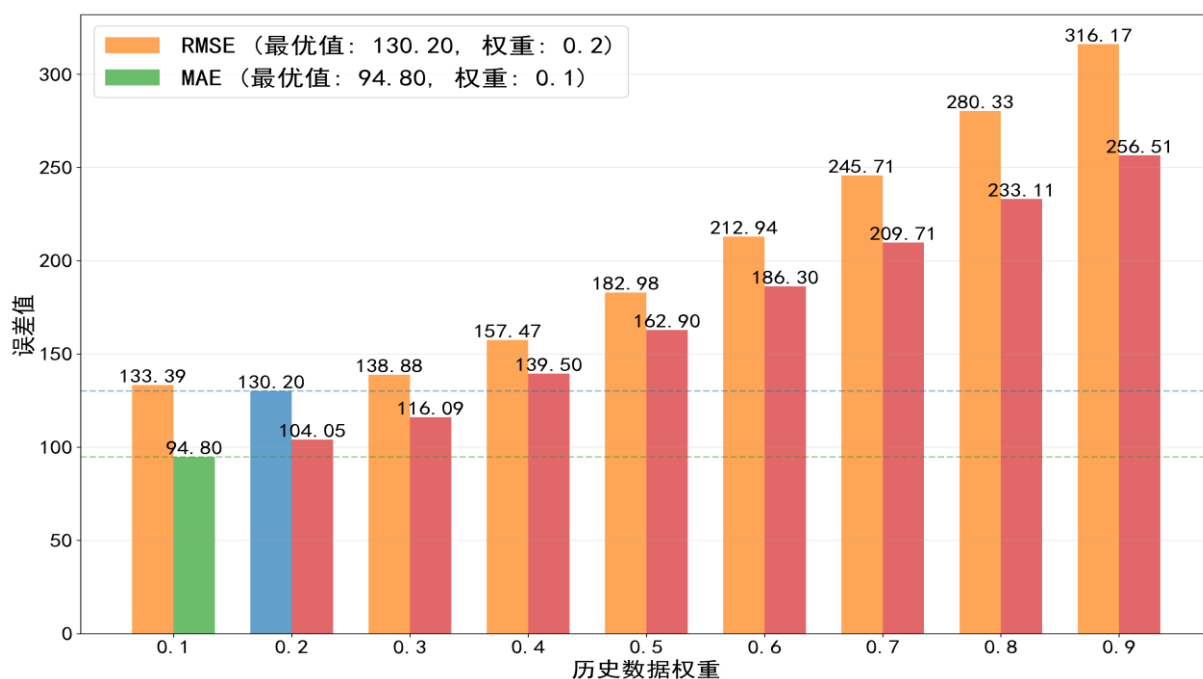


图 8 场地 3-站点 83-0600 不同权重系数误差对比图

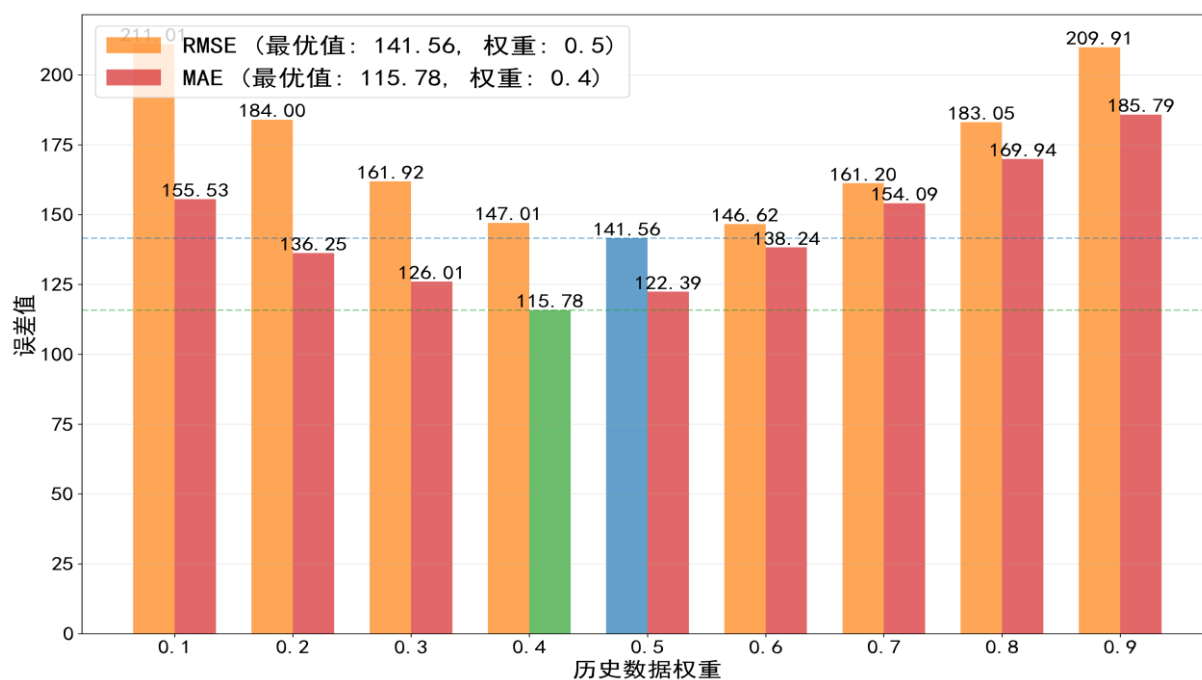


图 9 场地 3-站点 83-1400 不同权重系数误差对比图

由图 8 可知“0600”线路 RMSE 最优值为 130.20，对应权重系数为 0.2；MAE 最优值为 94.80，对应权重系数为 0.1。随着权重系数从 0.1 增至 0.9，RMSE 先减小后增大，MAE 则呈上升趋势；由图 9 可知“1400”线路 RMSE 最优值为 141.56，对应权重系数为 0.5；MAE 最优值为 115.78，对应权重系数为 0.4。随着权重系数从 0.1 增至 0.9，RMSE 和 MAE 都是先减小后增大。综合两条线路的权重系数误差对比结果图，权衡 RMSE 和 MAE 的变化情况，我们最终确定历史数据权重系数为 0.3。在此系数下，模型的 RMSE 和 MAE 相对平衡，能够较好地满足分析需求。

六、问题二模型建立与求解

问题分析：问题二是一个优化类问题，实际上可以转化为一个在一定的约束条件下的一个多目标优化问题，使得 1.自有车的周转率尽可能高 2.所有车辆均包裹尽可能高 3.总成本尽可能低，这要求我们应该选择一个合适的模型来处理不同量纲的目标值和“串点”策略以实现问题的目标。

6.1 模型建立

● 变量准备

1. 决策变量

在实际运输调度中，并非所有的运输需求都能在当前条件资源下得到满足。为明确哪些运输需求将被纳入调度方案，哪些需求由于资源限制或其他原因暂时不被安排运输，可将决策变量设置为运输需求*i*是否被安排运输，利用 0-1 变量进行如下表示：

$$X_i = \begin{cases} 0, & \text{运输需求 } i \text{ 未被安排运输} \\ 1, & \text{运输需求 } i \text{ 被安排运输,} \end{cases} \quad (9)$$

其中*i*表示不同短途运输需求。

在构建车辆调度方案时，充分利用车辆装载能力降低运输成本是重要指标之一，其中串点策略是提高运输效率的关键手段。为确定每个运输需求具体由哪辆车来承担运输任务，精确地描述车辆与运输需求之间的分配关系，我们同样利用 0-1 变量设置如下决策变量：

$$Y_{ik} = \begin{cases} 0, & \text{运输需求 } i \text{ 未由车队 } k \text{ 承运} \\ 1, & \text{运输需求 } i \text{ 由车队 } k \text{ 承运,} \end{cases} \quad (10)$$

$$S_i = \begin{cases} 0, & \text{运输需求 } i \text{ 未使用自有车承运} \\ 1, & \text{运输需求 } i \text{ 使用自有车承运,} \end{cases} \quad (11)$$

$$Z_{ij} = \begin{cases} 0, & \text{运输需求 } i \text{ 未与需求 } j \text{ 串点} \\ 1, & \text{运输需求 } i \text{ 与需求 } j \text{ 串点,} \end{cases} \quad (12)$$

其中*k*表示不同短途运输车队；*l*表示不同短途运输需求（*j* ≠ *i*）

● 确定目标

● 串点处理策略

在问题二中我们的优化目标主要有三个方面：最大化自有车周转率、最大化车辆均包裹数和最小化总成本。为此我们设计了基于图结构与贪心策略的串点处理策略，包含可串点关系建模、串点组合生成及任务分配三个核心环节，具体如下：

Step1:建立可串点关系无向图

首先我们将附件 4 中定义的可串点关系建模为无向图结构，构建可串点字典，使其能在极短时间内判断两站点是否可串点，避免重复遍历计算，为后续组合生成提供高效数据支撑。

Step2:串点组合无向图生成与筛选

接着针对同一分组内的运输需求（按起始场地、车队、发运节点分组），生成二联/三联串点组合通过相同场地和时间编码（0600 和 1400），附件 4 中站点无向连接，最终我们得到有效二连串联组合总数:174，有效三连串联组合总数:152，各“串点”关系

无向图如图 10 所示：

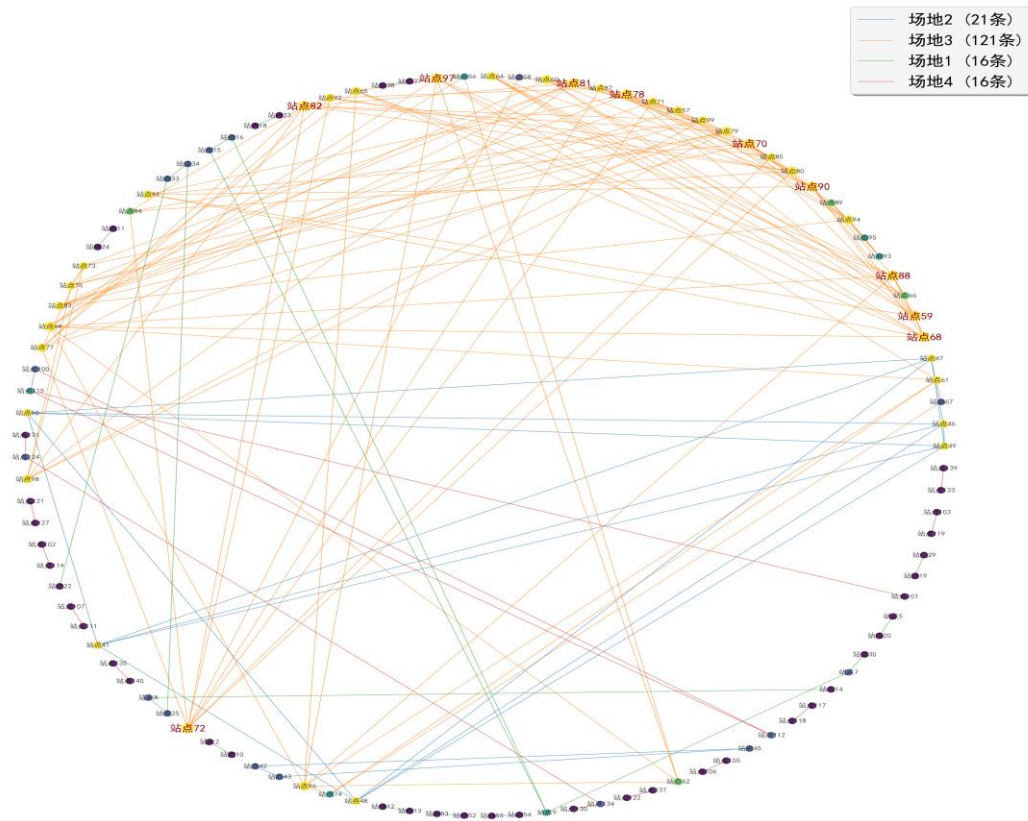


图 10 各串点关系无向图

Step3:串点任务分配

最后针对可行串点组合按照贪心排序（优先处理节约量最大的组合）排序与调度，通过贪心策略锁定当前最优解，结合完全可串点验证（三联组合）与状态动态更新，在保证计算效率的同时实现成本节约最大化。

● 优化目标函数设立

（1）最大化自有车周转率（ TR_{SV} ）

在短途运输中，自有车辆是重要的运输资源，较高的周转率意味着在单位时间内，自有车辆能够完成更多的运输任务。这不但能充分利用车辆资源，还能减少对外部承运商的依赖，从而降低成本。

$$maxTR_{SV} = \frac{\sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} Y_{ik} \cdot S_i}{\sum_{k \in \mathcal{F}} n_k}, \quad (13)$$

其中 $\mathcal{F}$ 为车队集合；运输需求集合为 $\mathcal{D}$ ； n_k 为车队 k 的自有车数量。

（2）最大化车辆均包裹率（ AV_P ）

在实际运输中，提高车辆均包裹数意味着在相同的车辆投入下，可以运输更多的货物，进而提升运输效率和经济效益。

$$maxAV_P = \frac{\sum_{i \in \mathcal{D}} q_i}{\sum_{k \in \mathcal{F}} n_k + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik}}, \quad (14)$$

其中 q_i 为需求 i 的包裹数量。

(3) 最小化总成本

自有固定成本是车队自有车辆每天的固定支出，如车辆购置成本的分摊、场地租赁等，与车辆的使用频率无关。自有变动成本与每次发运线路需求相关，例如燃油费、司机工资等，会随着运输任务的增加而增加。外部总成本则是委托外部承运商运输所产生的费用总和。总成本体现了整个运输活动的经济成本投入，降低总成本是提高运输效益的关键目标之一。

$$minTC = \begin{cases} TC_2 = \sum_{m1, m2 \in \mathcal{B}_2} \left(\sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \max\{c_{m1}^{self}, c_{m2}^{self}\} \right. \\ \quad \left. + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \max\{c_{m1}^{external}, c_{m2}^{external}\} \right) & \text{二联组合,} \\ TC_3 = \sum_{o1, o2, o3 \in \mathcal{B}_3} \left(\sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \max\{c_{o1}^{self}, c_{o2}^{self}, c_{o3}^{self}\} \right. \\ \quad \left. + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \max\{c_{o1}^{external}, c_{o2}^{external}, c_{o3}^{external}\} \right) & \text{三联组合,} \\ TC_d = \sum_{d \in \mathcal{B}_d} \left(\sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \sum c_d^{self} \right. \\ \quad \left. + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \sum c_d^{external} \right) & \text{独立组合,} \end{cases} \quad (15)$$

其中， T_{fixed} 是每辆自有车每日的固定成本； $\mathcal{B}_2$ 是可二联串点组合站点集合； $\mathcal{B}_3$ 是可三联串点组合站点集合； $c_{m_i}^{self}(i=1,2)$ 和 $c_{m_i}^{external}(i=1,2)$ 分别表示可二联串点集合线路的自有车承运的变动成本和外部车承运成本； $c_{o_i}^{self}(i=1,2,3)$ 和 $c_{o_i}^{external}(i=1,2,3)$ 分别表示可三联串点组合集合线路的自有车承运的变动成本和外部车承运成本。

● 约束条件

1. 发运时间约束

在物流运输作业中，货物的及时送达对于客户体验和物流运营的顺畅性至关重要。为确保货物的及时发出，满足物流时效性要求，设定每个运输需求的预计发货时间 t_i 不能晚于线路的发运节点 $T_{deadline}$ ，即

$$t_i \leq T_{deadline} \quad (16)$$

2. 车辆装载量约束

为保证车辆运输的安全性以及货物的完整性和运输质量，防止超载情况的发生，每辆车的装载包裹量 m_i 不能超过车辆的最大装载量 Q_{max} ，即

$$m_i \leq Q_{max} \quad (17)$$

3.自有车辆周转约束

在使用自有车辆连续执行运输任务时，为保证车辆周转的正常运行，需要确保前一个需求完成后，车辆有足够的时间返回并准备下一次运输。由此设定约束条件如下：

$$t_m + t_{loading} + 2t_{route} + t_{unloading} < t_n, \quad (18)$$

其中 t_m 表示前一个需求的发运时间； $t_{loading}$ 表示一次需求装运时间； t_{route} 表示单程在途时间； $t_{unloading}$ 表示一次需求卸车时间； t_n 表示后一个需求的发运时间。

4.串点数量约束

在实际物流操作中，虽然串点策略可以在一定程度上提高运输效率，但过多的线路串点反而会使调度变得过于复杂，影响运输效率。基于此对串点组合中的线路数量设定如下约束条件：

$$n_{string} \leq 3 \quad (19)$$

➤ 综上，多目标优化的短途车辆串点调度模型如下：

$$\left\{ \begin{array}{l} \max TR_{SV} = \frac{\sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} Y_{ik} \cdot S_i}{\sum_{k \in \mathcal{F}} n_k} \\ \max AV_P = \frac{\sum_{i \in \mathcal{D}} q_i}{\sum_{k \in \mathcal{F}} n_k + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) \cdot Y_{i,k}} \\ \min TC = \begin{cases} \sum_{m1, m2 \in \mathcal{B}_2} TC_2 = \sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \max\{c_{m1}^{self}, c_{m2}^{self}\} \\ + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \max\{c_{m1}^{external}, c_{m2}^{external}\} & \text{二联组合,} \\ \sum_{o1, o2, o3 \in \mathcal{B}_3} TC_3 = \sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \max\{c_{o1}^{self}, c_{o2}^{self}, c_{o3}^{self}\} \\ + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \max\{c_{o1}^{external}, c_{o2}^{external}, c_{o3}^{external}\} & \text{三联组合,} \\ \sum_{d \in \mathcal{B}_d} TC_d = \sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \sum c_d^{self} \\ + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \sum c_d^{external} & \text{独立组合,} \end{cases} \\ s.t. \left\{ \begin{array}{l} t_i \leq T_{deadline} \quad (\text{发运时间约束}) \\ t_m + t_{loading} + 2t_{route} + t_{unloading} < t_n \quad (\text{自有车辆周转约束}) \\ m_i \leq Q_{max} \quad (\text{车辆装载量约束}) \\ n_{string} \leq 3 \quad (\text{串点数量约束}) \end{array} \right. \end{array} \right.$$

现在问题已经被我们转换成为一个带有约束条件的多目标优化[1]问题，目前对于此类问题有很多种解决方法，如加权和整数规划法[2]， ϵ -约束法[3]，分层序列法[4]，多目标进化算法（NSGA-II，MOPSO[5]等）。在本问题中实际上对于模型的选择存在四

个难点 1.权重比选择：对于目标函数构建时权重比在很大程度上会影响结果的产生，在实际的工程应用当中，衡量权重比实际上有待欠缺 2.归一化处理：实际上问题中的三个目标优化条件的量纲并没有保持一致（周转率，包裹率，总成本），这往往需要我们对三个目标值进行归一化处理之后在进行加权的形式，但我们事先并不知道这三个数的大致范围，因此往往会使用一个简要的最大值的预测值来进行归一化处理，但每一天的波动情况都会导致目标值范围发生变动，两者综合起来看实际上对于我们的调度方案还是有较大的影响的 3.“串点”处理：对于串点策略的融入进目标函数较难处理 4.实际上对于 NSGA-II 这样的算法而言据参考一般用于处理 10-50 条线路的调度问题有较好的效果，小组对于这个模型也进行了相应的尝试，不过在种群数量调高以得到较高准确率的过程中运行的时间复杂度过高，在物流调度过程中我们不认为这是一个较好的选择。

最终我们决定利用贪心策略通过分层序列法对目标函数逐步优化，在下文的模型评价中我们可以看出，在我们的贪心策略与其他算法的比较过程中实际上在运行时间和最终结果两方面均有较好的表现。

6.2 模型求解

在观察问题一所得结果（图 4 和图 5）实际上包裹的产生是存在其峰值和平缓期的，这给我们带来启发我们的贪心策略的实现可以优先从高峰区进行串点组合和自有车辆的分配然后过度到平缓区域以达到最大的车辆周转率和包裹率最大化，实际上这一想法在路径规划，实时数据分析，网络监控，日志处理被广泛应用，可以快速的识别出包裹量高峰时期，而 Misra & Gries' s (MG) 算法[6]则是其中一个常用的算法，借助此我们可以将其与我们的贪心策略结合构建贪心-MG 算法。

算法 1: Misra & Gries's (MG) 算法

输入：数据流 $S = [s_1, s_2, \dots, s_n]$ ，整数 k

输出：候选元素集合 C

1. 初始化候选元素集合 C 为空集，每个候选元素的计数数组 $count$ 初始化为 0
 2. 对于数据流 S 中的每个元素 s :
 3. 如果 s 已经在候选元素集合 C 中:
 4. 找到 s 在 C 中的索引 i
 5. $count[i] = count[i] + 1$
 6. 否则如果候选元素集合 C 的大小小于 k :
 7. 将 s 添加到候选元素集合 C 中
 8. 初始化 $count$ 中对应 s 的计数为 1
 9. 否则:
 10. 对于候选元素集合 C 中的每个元素 c :
 11. $count[c \text{ 对应的索引}] = count[c \text{ 对应的索引}] - 1$
 12. 移除 $count$ 中计数为 0 的元素及其对应的候选元素
 13. 返回候选元素集合 C
-

最终我们的贪心 MG 算法来处理本题的整体策略如下面的流程图所示：

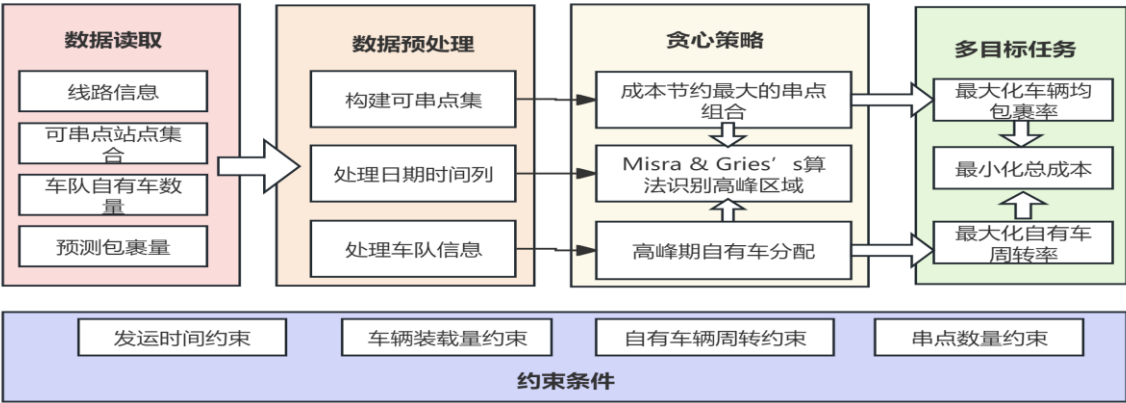


图 11 利用贪心 MG 算法处理问题二流程图

利用 Python 求解得到各线路的串点调度方案结果表如下（以场地 3-站点 83-0600 和场地 3-站点 83-1400 线路为例，完整结果见附件结果表 3）：

表 5 场地 3-站点 83-0600 线路调度结果表

线路编码	预计发运时间	承运车辆
场地 3-站点 78-0600+场地 3-站点 83-0600	2024-12-16 05:50:00	车队 3-9
场地 3-站点 83-0600	2024-12-15 22:20:00	外部
场地 3-站点 83-0600	2024-12-15 23:40:00	车队 3-23
场地 3-站点 83-0600	2024-12-16 00:50:00	外部
场地 3-站点 83-0600	2024-12-16 01:50:00	外部
场地 3-站点 83-0600	2024-12-16 03:00:00	车队 3-1
场地 3-站点 83-0600	2024-12-16 04:20:00	车队 3-1
场地 3-站点 83-0600	2024-12-16 05:10:00	车队 3-1
场地 3-站点 83-0600	2024-12-16 05:50:00	车队 3-1

表 6 场地 3-站点 83-1400 线路调度结果表

线路编码	预计发运时间	承运车辆
场地 3-站点 70-1400+场地 3-站点 83-1400	2024-12-16 13:50:00	车队 3-5
场地 3-站点 83-1400	2024-12-16 12:00:00	外部
场地 3-站点 83-1400	2024-12-16 12:30:00	外部

表 7 问题二调度结果表

调度次数	703
串点率	16.07%
自有车辆利用率	68.85%
平均周转率	408%
总耗时	5.14 秒
最终方案总成本	61880 元

观察表 5 和表 6 线路调度方案结果表我们可以看出，内部车队在单一站点运输中表现出明显的时段集中性，如车队 3-1 在“场地 3-站点 83-0600”线路中承担 2024 年 12 月 16 日 03:00 至 05:50 时段的 4 次发运任务，形成连续的高频次运输；而外部运力则主要分布在 2024 年 12 月 15 日 22:20 至 2024 年 12 月 16 日 01:50 的非连续时段，反映出“自有车为主，外部运力补充”的调度策略；同时串点组合发运模式连接多个运输需求，

节约了时间成本。

同时在选择模型时我们尝试了多种模型最终选择了贪心 MG 模型，在此处我们通过
与 1.模拟退火算法（初始温度：100.0 最终温度：0.1，温度衰减系数：0.95 每个温度下的迭代次数：100，成本权重：0.4，周转率权重：0.3，包裹率权重：0.3）2.贪心算法，
3.贪心+整数规划（成本权重：0.4，周转率权重：0.3，包裹率权重：0.3），4.NSGA-II，
5.整数规划（成本权重：0.4，周转率权重：0.3，包裹率权重：0.3）对比得出如下表 8 所示结果：

表 8 不同算法结果对比表

	调度次数	串点率	自有车辆 利用率	平均周转率	总耗时	总成本
贪心 MG	703	16.07%	68.85%	408%	5.14 秒	61880 元
模拟退火	699	15.59%	56.51%	337%	59.23 秒	71222 元
贪心算法	698	15.47%	56.45%	334%	7.70 秒	81436 元
贪心+整数 规划	683	13.62%	58.57%	364%	5.43 秒	89034 元
NSGA-II 整数规划	因模型运行时间过长不予考虑且最终调度结果一般 调度时间过长					

分析不同算法结果对比表可知，贪心 MG 算法在多项关键指标上表现突出：其自有车辆利用率、平均周转率、总耗时的表现均优于模拟退火和贪心算法，凸显其自有车辆资源利用的充分性以及货物周转的高效性；总成本均低于贪心算法与贪心结合整数规划算法，具备明显成本优势。尽管其串点率略高于部分算法，但没有 NSGA-II 和整数规划运行时间过长的的问题，且在车辆利用率、周转效率、耗时及成本等核心维度表现优异，综合性能更具实用性与高效性。

七、问题三模型建立与求解

问题分析：根据问题一得到的货量预测结果，结合问题三所给标准容器使用带来的装卸时长缩短和装载量下降情况，该问解决核心在于结合标准容器特性重新确定运输需求，决定是否使用该容器，在满足和问题二相同优化目标的前提下，平衡装卸时间和装载量变化对调度方案的影响。

7.1 模型建立

在问题二的基础上，引入标准容器作为决策变量，其核心变化为装载量与装卸时长的调整。装载量降至 800 件/车，装卸时长缩短至 10 分钟/次，需在调度中决策是否使用容器以平衡效率与成本。

● 决策变量扩展

在问题二是否被安排运输、是否由某个车队运输、是否使用自有车运输以及是否串点组合决策变量基础上，新增 0-1 变量表示是否使用标准容器：

$$L_i = \begin{cases} 0, & \text{运输需求 } i \text{ 未使用标准容器} \\ 1, & \text{运输需求 } i \text{ 使用标准容器} \end{cases} \quad (20)$$

● 串点处理策略适配

在问题二串点策略基础上增加对标准容器使用策略的校验：

1.总装载量约束:

$$\sum_{i \in \mathcal{D}} m_i \leq \sum_{i \in \mathcal{D}} 800L_i + 1000(1 - L_i) \quad (21)$$

2.成本与效率权衡: 在贪心排序时, 优先选择因装卸时间缩短(提升周转效率)与装载量下降之间净收益更高的组合。例如, 短途、小批量货物使用容器可能更优, 而大批量货物可能倾向不使用容器以减少发车次数。

● 目标函数继承

该问优化目标与问题二一致, 即结合串点组合方案, 在满足发运时间限制下, 优化自有车调度, 以尽可能提高自有车周转率和车辆均包裹数、降低总成本。

● 约束条件调整

1.车辆装载量约束

结合标准容器使用状态, 每辆车的货物装载量需满足:

$$\begin{cases} L_i = 1, & m_i \leq 800 \\ L_i = 0, & m_i \leq 1000 \end{cases} \quad (22)$$

2.自有车辆周转约束

结合标准容器使用状态, 设定自有车辆周转约束条件为:

$$t_m + t_{loading} + 2t_{route} + t_{unloading} < t_n \quad (23)$$

$$\begin{cases} t_{loading} = t_{unloading} = 10 \text{ 分钟}, & L_i = 1 \\ t_{loading} = t_{unloading} = 45 \text{ 分钟}, & L_i = 0 \end{cases} \quad (24)$$

其余约束条件(发运时间、串点数量)与问题二一致。

● 标准容器使用策略

在生成运输需求时, 问题三引入容器使用策略函数, 综合以下因素动态决定 L_i :

1.时间效益(使用容器节省装卸时间): 在自有车辆周转约束中, 装卸时间随 L_i (是否使用容器) 动态调整, 使用容器时单次装卸时间从 45 分钟缩短至 10 分钟, 能够直接减少车辆周转所需时间, 提升自有车周转率。(见公式(23))

2.包裹利用率(容器的装载率): 装载量约束根据是否使用容器设定不同上限(800 件/1000 件), 确保实际载货量不超过车辆装载能力, 避免超载或装载不足。(见公式错误!未找到引用源。)

3.路线时间(短途线路更适合使用容器): 短途线路单程在途时间 t_{route} 较短, 装卸时间占总周转时间的比例更高, 使用容器节省的装卸时间(35 分钟/次)对短途线路的时间效益更显著, 最终提升自有车周转率。(见公式(23))

4.包裹数量(包裹量少使用容器更经济): 当包裹量低于 800 件时, 使用容器不会因装载量下降导致额外车次(因原车型也需 1 车), 但能节省装卸时间; 若包裹量在 800-1000 件之间, 不使用容器可 1 车满载, 使用容器则需 1 车(不超 800 件), 此时需权衡时间节省与装载量浪费。

5.串点任务考量(串点使用容器更有效率): 在串点处理贪心排序算法中, 优先处理“装卸时间缩短带来的周转效率提升”与“装载量下降导致的车次增加成本”之间净收益更高的组合。串点任务因涉及多个站点, 装卸时间节省叠加效应更明显, 故倾向使用容器。

6.高峰期决策(发运节点优先使用容器): 每天 6 点和 14 点为固定发运节点, 高

峰期货量集中，使用容器可缩短装卸时间，增加自有车在节点前的周转次数，减少外部车使用，进而提升自有车周转率。

➤ 综上，调整后的多目标优化的短途车辆串点调度模型如下：

$$\left\{ \begin{array}{l} \max TR_{SV} = \frac{\sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} Y_{ik} \cdot S_i}{\sum_{k \in \mathcal{F}} n_k} \\ \max AV_P = \frac{\sum_{i \in \mathcal{D}} q_i}{\sum_{k \in \mathcal{F}} n_k + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) \cdot Y_{i,k}} \\ \min TC = \left\{ \begin{array}{l} \sum_{m1, m2 \in \mathcal{B}_2} TC_2 = \sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \max\{c_{m1}^{self}, c_{m2}^{self}\} \\ + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \max\{c_{m1}^{external}, c_{m2}^{external}\} \quad \text{二联组合,} \\ \sum_{o1, o2, o3 \in \mathcal{B}_3} TC_3 = \sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \max\{c_{o1}^{self}, c_{o2}^{self}, c_{o3}^{self}\} \\ + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \max\{c_{o1}^{external}, c_{o2}^{external}, c_{o3}^{external}\} \quad \text{三联组合,} \\ \sum_{d \in \mathcal{B}_4} TC_d = \sum_{k \in \mathcal{F}} n_k T_{fixed} + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} S_i Y_{ik} \sum c_d^{self} \\ + \sum_{k \in \mathcal{F}} \sum_{i \in \mathcal{D}} (1 - S_i) Y_{ik} \sum c_d^{external} \quad \text{独立组合,} \end{array} \right. \\ s.t. \left\{ \begin{array}{l} t_i \leq T_{deadline} \quad (\text{发运时间约束}) \\ t_m + t_{loading} + 2t_{route} + t_{unloading} < t_n \\ \begin{cases} t_{loading} = t_{unloading} = 10 \text{ 分钟}, L_i = 1 \\ t_{loading} = t_{unloading} = 45 \text{ 分钟}, L_i = 0 \end{cases} \quad (\text{自有车辆周转约束}) \\ m_i \leq 800L_i + 1000(1 - L_i) \\ \begin{cases} L_i = 1, m_i \leq 800 \\ L_i = 0, m_i \leq 1000 \end{cases} \quad (\text{车辆装载量约束}) \\ n_{string} \leq 3 \quad (\text{串点数量约束}) \end{array} \right. \end{array} \right.$$

(25)

7.2 模型求解

同样地，我们采取贪心-MG 算法利用 Python 求解得到各线路的串点新调度方案结果表如下（以场地 3-站点 83-0600 和场地 3-站点 83-1400 线路为例，完整结果见附件结果表 4）：

表 9 场地 3-站点 83-0600 新调度结果表

线路编码	预计发运时间	是否使用 容器	发运车辆
场地 3-站点 78-0600+场地 3-站点 83-0600	2024-12-16 05:50:00	是	车队 3-9
场地 3-站点 83-0600	2024-12-15 22:20:00	否	外部
场地 3-站点 83-0600	2024-12-15 23:40:00	否	车队 3-25
场地 3-站点 83-0600	2024-12-16 00:50:00	否	外部
场地 3-站点 83-0600	2024-12-16 01:50:00	否	外部
场地 3-站点 83-0600	2024-12-16 03:00:00	否	车队 3-1
场地 3-站点 83-0600	2024-12-16 04:20:00	否	车队 3-1
场地 3-站点 83-0600	2024-12-16 05:10:00	否	车队 3-1
场地 3-站点 83-0600	2024-12-16 05:50:00	否	车队 3-1

表 10 场地 3-站点 83-1400 新调度结果表

线路编码	预计发运时间	是否使用 容器	发运车辆
场地 3-站点 70-1400+场地 3-站点 83-1400	2024-12-16 13:50:00	否	车队 3-5
场地 3-站点 83-1400	2024-12-16 12:00:00	否	外部
场地 3-站点 83-1400	2024-12-16 12:30:00	否	外部

表 11 问题三新调度结果表

调度次数	703
串点率	16.07%
自有车辆利用率	72.83%
平均周转率	435%
总耗时	6.52 秒
最终方案总成本	54158 元
容器使用占比	21.48%

观察表 9 和表 10 的新调度方案结果表我们可以看出，问题三的调度结果通过标准容器的使用有效提升了自有车的周转效率，并通过串点策略整合零散货量，在满足时效约束的前提下，平衡了自有车利用率与成本优化目标。

观察表 11 我们可以看出，最终我们使用标准容器的占比为 21.48%，在引入标准容器之后，相对于问题二我们所求解的结果在自有车辆利用率，平均周转率，最终方案总成本均有所改进，说明标准容器对于我们目标的进一步优化的构建十分有效果。

八、 问题四模型建立与求解

基于问题一的货量预测结果，结合问题三求得的新调度模型，该问解决核心在于评估当货量预测结果出现偏差时，对调度模型中自有车周转率、车辆均包裹数和总成本等优化结果的影响，同时分析不同偏差情况可能带来的车辆调度变化及指标波动。

8.1 预测误差扰动模型构建

1.随机误差扰动模型

假设货量偏差服从正态分布，模拟小概率随机扰动得到扰动公式如下：

$$V'_t = V_t(1 + \epsilon) \quad \epsilon \sim \mathcal{N}(0, \sigma^2), \tag{26}$$

其中 ϵ 为随机扰动项，均值为0，方差为历史货量数据的波动水平； V_t 为原始货量； V'_t 为扰动后的货量（ $V'_t \geq 0$ ）。

2. 系统性误差扰动模型

假设所有时段货量按照统一比例产生偏差，模拟得到扰动公式如下：

$$V'_t = V_t(1 + bias), \quad (27)$$

其中 $bias$ 表示实际货量对于预测货量的相对误差，当 $bias > 0$ 时表示模型低估了需求货量，即实际货量 $>$ 预测货量；当 $bias < 0$ 时表示模型高估了需求货量，即实际货量 $<$ 预测货量。

3. 周期性误差扰动模型

由题中“6点和14点发运节点对应的最早货量分别为前一天21点和当天11点”，即每天6-11点、14-21点为非生产时段，但实际可能存在周期性波动（如早晚高峰货量突增）。因此我们采用正弦函数模拟24小时内的周期性偏差，模拟得到扰动公式如下：

$$\text{periodic_error} = A \sin\left(\frac{2\pi t_{\text{index}}}{T} + \phi\right) V'_t = V_t(1 + \text{periodic_error}), \quad (28)$$

$$t_{\text{index}} = \frac{(dt - t_{\min})\text{total_seconds}()}{600}, \quad (29)$$

其中 $t_{\min}$ 为预测时间范围起点，单位为十分钟颗粒度；非生产时段的 periodic_error 绝对值 ≤ 0.1 ； A 为最大相对误差； T 是时间周期。

4. 时间偏移误差扰动模型

由题中可能存在的“计划流经线路时间与实际不符”问题，如实际高峰期提前/延后，导致货量在时间颗粒度上的错配。我们按照“线路+日期”分组，对货量序列 V_t 进行时间偏移得到如下扰动公式：

$$V'_t = \begin{cases} \text{group}[V_t].\text{shift}(k) & t \notin \text{边界时段(避免越界)} \\ \text{group}[V_t].\text{mean}() & \text{越界时用组内均值填充,} \end{cases} \quad (30)$$

其中偏移单位 k 取值正负，对应模拟高峰期提前和延后。

5. 突发性误差扰动模型

由题中可能存在“已下单货量可能取消”或突发新增订单等问题导致在特点时间段内货量骤增/骤减，假设突发时段为8:00-9:00、17:00-18:00，即非生产时段外的潜在高峰，模拟得到如下扰动模型：

$$V'_t = \begin{cases} V_t \beta & t \in [t_{\text{start}}, t_{\text{start}} + D \times 10 \text{ 分钟}] \\ V_t & \text{其他时段,} \end{cases} \quad (31)$$

其中 β 为突发因子表示突发时段货量增加或减少； D 表示突发事件持续时长； t_{start} 表示突发事件开始时间。

6. 蒙特卡洛模拟复合扰动模型

为了进一步量化不确定性影响，我们通过随机采样随机误差 σ 和系统性误差 b ，模拟生成多组复合扰动场景，具体公式如下：

$$V_t^{(i)} = V_t(1 + \epsilon_t^{(i)})(1 + b^{(i)}), \quad \begin{cases} \epsilon_t^{(i)} \sim \mathcal{N}(0, \sigma_i^2), & \sigma_i \sim \mathcal{U}(\sigma_{\min}, \sigma_{\max}) \\ b^{(i)} \sim \mathcal{U}(b_{\min}, b_{\max}), \end{cases} \quad (32)$$

其中 $V_t^{(i)}$ 表示时间段为 t 时第 i 次扰动后货量； $\epsilon_t^{(i)} \in [0.05 - 0.3]$ 表示生成的高斯噪声；

$b^{(i)} \in [-0.2 - 0.2]$ 表示第 i 次的系统性误差； $\mathcal{N}$ 表示模拟重复次数。

8.2 模型求解

在随机误差扰动中设置货量 10% 正态波动来模拟传感器噪声；在系统性误差扰动中设置 $bias = 0.2$ ；在周期性误差扰动中设置 $A = 0.2$ （ $\pm 20\%$ 振幅）、 $T = 144$ （24 小时周期，时间单位 10 分钟）的正弦函数模拟日波动，时间偏移设定 $k = 6$ 小时来模拟高峰时段偏差，突发误差默认 $\beta = 3.0$ （3 倍激增）、 $D = 6$ 单位时长；在蒙特卡洛模拟复合扰动中集成 $\epsilon_t^{(i)} \in [0.05 - 0.3]$ （5%-30% 随机波动）和 $b^{(i)} \in [-0.2 - 0.2]$ （ $\pm 20\%$ 全局偏移）的 100 次复合扰动。所有模型均作用于预测货量列，利用 Python 求解得到如下扰动后货量数据结果对比图：

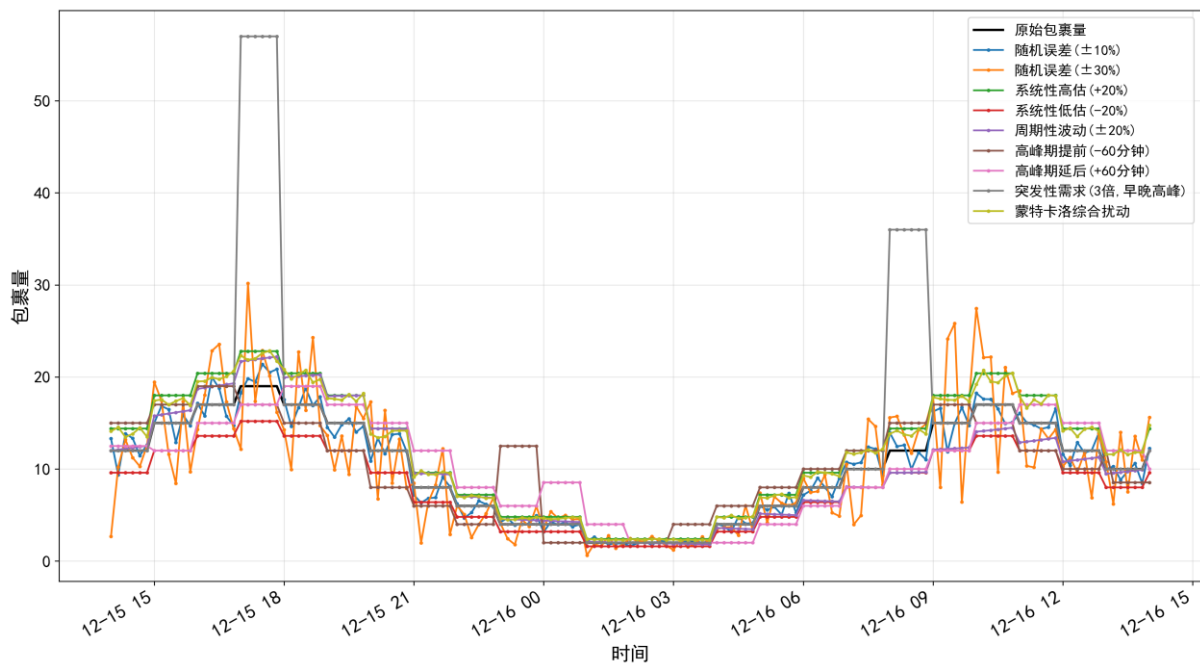


图 12 不同扰动作用下的预测货量折线图

从上图不同扰动方式下预测货量变化折线图我们可以发现，原始包裹量具有的季节性变化在不同扰动方式下得到不同程度的继承：随机扰动围绕原始值小幅波动，影响幅度较小；系统性扰动高估与低估线条走势整体高于或低于原始值，表现出一致性的偏差；周期性扰动线条呈现规律起伏，与原始包裹量周期性特征相呼应；高峰期的提前与延迟使高峰位置产生相应偏移；突发性需求的激增使货量在特定时段出现显著高峰，突出其对货量的强烈冲击；蒙特卡洛复合扰动综合多种因素，波动较为复杂。总的来说，该图呈现的不同扰动因素对包裹量预测的影响方式与程度有助于深入分析各因素在物流预测中的作用，为后续优化预测模型、提升物流运作效率提供支撑。

接着我们根据不同扰动方式作用后的预测货量数据进一步分析对不同线路运输需求、调度优化目标（自有车使用率、自有车周转率、运输成本）、串点率、标准容器使用率等的影响变化，利用 Python 得到如图 13 和图 14 所示：

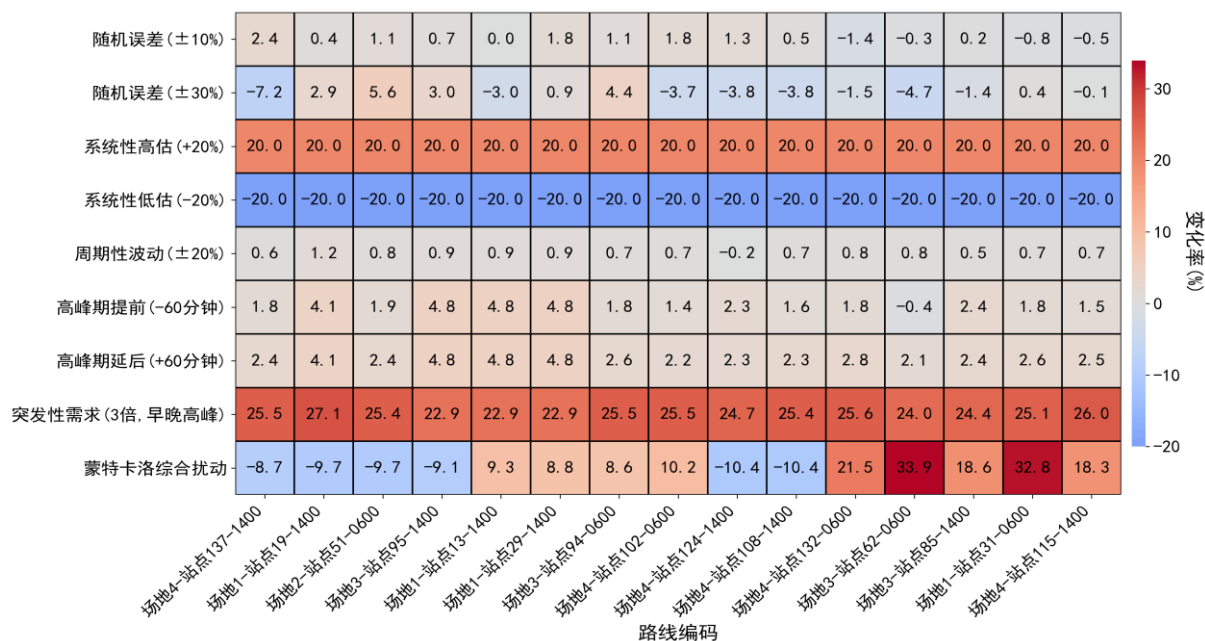


图 13 扰动后的预测货量对各线路运输量的影响热力矩阵图

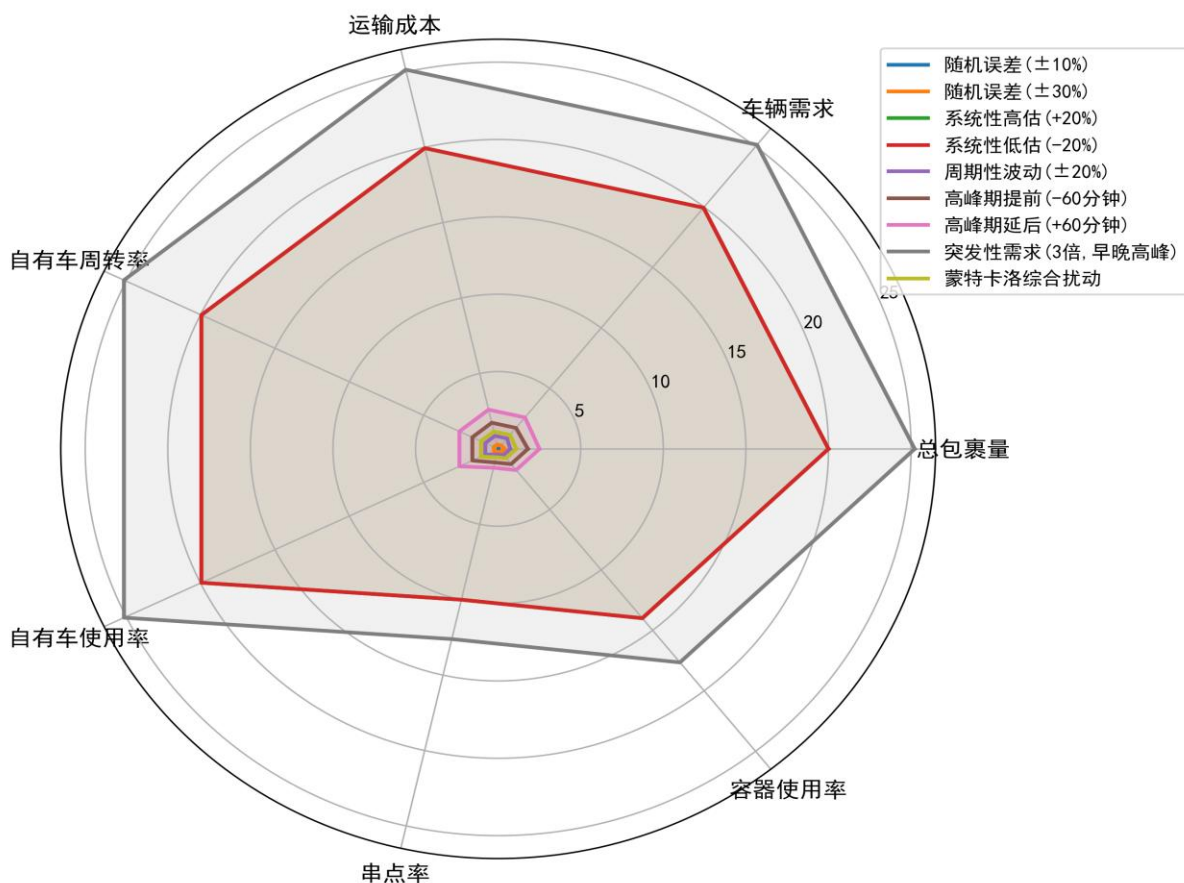


图 14 扰动后的预测货量对调度优化目标的影响雷达图

从图 13 和图 14 我们可以发现，所有评估指标平均敏感性均不超过 10%，其中总包裹量对扰动最为敏感，在雷达图中其受扰动影响的幅度相对显著；串点率敏感度最低，受扰动影响的变化幅度较小。这表明尽管不同扰动对各指标存在一定影响，但整体模型

在多种扰动下仍能保持相对稳定，鲁棒性较强，为物流调度优化提供了可靠支撑，确保在复杂扰动场景下仍具备较好的适应性与稳定性。

九、模型评价

● 模型优点：

1.**特征工程丰富**：融合多维度时间特征、滞后特征和预知数据比例，对跨午夜线路和日间线路的差异化货量模式建模能力强，适配物流场景的复杂约束。

2.**高峰区域识别**：问题二结合 MG 算法动态筛选高频货量时段，优先分配自有车辆处理高峰需求，提升周转率和包裹率。相较于传统贪心算法，MG 算法在处理大规模数据流时内存占用低，且能快速定位关键运输需求，调度总成本降低 24% 以上。

3.**多类型扰动模拟**：问题四构建随机误差、系统性误差、周期性误差等 6 类扰动模型，覆盖货量预测偏差的主要场景（如突发订单、高峰期偏移），通过蒙特卡洛模拟量化不确定性对调度指标的影响，为实际应用中的风险控制提供依据。

4.**求解速度快，精度高**：本论文所提出的贪心 MG 算法对于大批量的数据可以实现快速的调度分配，同时还能够保持较高的目标完成率，对于实际的物流调度的分配有很大帮助。

● 模型缺点：

1.**可能并非最优解**：考虑到实际应用中求解大批量线路的最优解会消耗大量的时间成本，本文中的调度模型实际上采用的是以贪心策略为基石所提出的算法，而这样往往可能无法保证模型会求解到全局最优的情况，但其实很多实际问题我们选取一个较优的解已经有所帮助，无需追求最优解。

2.**预测模型的假设依赖性**：依赖历史数据的稳定性，若未来货量模式因促销活动、天气突变等出现非历史异常波动，校准后的预知货量可能失效，导致总货量预测偏差扩大。

3.**实际场景简化的局限性**：忽略交通拥堵、车辆故障等动态干扰因素，自有车周转约束假设“装卸+往返时间”严格可计算，而现实中不确定性可能导致调度方案不可行。

该模型适用于物流末端运输场景，通过分层预测与贪心算法生成精准调度计划，实现自有车效率提升、运输成本降低。可辅助企业评估标准容器等新型装备的引入价值，在高频小批量运输中平衡时间与装载效率。同时，基于误差扰动模型预设风险应对策略，动态优化运力分配，形成“预测-调度-应对”闭环，助力物流企业实现成本、效率与风险的协同优化。

十、参考文献

- [1].Coello, C. A. C., & Lechuga, M. S. "MOPSO: A proposal for multiple objective particle swarm optimization." Proceedings of the 2002 Congress on Evolutionary Computation, 2: 1051-1056, 2002.
- [2].Miettinen, K. "Nonlinear Multiobjective Optimization." Boston: Kluwer Academic Publishers, 1999.
- [3].Haimes, Y. Y., Lasdon, L. S., & Wismer, D. A. "On a bicriterion formulation of the problems of integrated system identification and system optimization." IEEE Transactions on Systems, Man, and Cybernetics, 1(3): 296-297, 1971.

- [4].Tzeng, G. H., & Huang, J. J. "Multiple Attribute Decision Making: Methods and Applications." Boca Raton: CRC Press, 2011.
- [5].Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. "A fast and elitist multiobjective genetic algorithm: NSGA-II." IEEE Transactions on Evolutionary Computation, 6(2): 182-197, 2002.
- [6].Misra, J., & Gries, D. "Finding repeated elements." Science of Computer Programming, 2(2): 143-152, 1982.

十一、附录

问题一模型评价代码

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
from datetime import datetime, timedelta
import time
import warnings
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import xgboost as xgb
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
# import pmdarima as pm
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose

# 深度学习模型
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.preprocessing import MinMaxScaler

# 忽略不必要的警告信息
warnings.filterwarnings('ignore')

# 配置中文字体与显示设置
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用于显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用于正确显示负号
```

```

def import_dataset():
    """导入数据源文件"""
    try:
        # 路线基础信息
        routes_data = pd.read_excel(r'C:\Users\16878\Desktop\D 题\附件\附件 1.xlsx')

        # 历史需求数据
        historical_volume = pd.read_excel(r'C:\Users\16878\Desktop\D 题\附件\附件
2.xlsx')

        # 已知需求数据
        known_volume = pd.read_excel(r'C:\Users\16878\Desktop\D 题\附件\附件 3.xlsx')

        # 生成历史需求数据的时间列
        historical_volume['时间戳'] = pd.to_datetime(historical_volume['日期'].astype(str)
+ '' +
                                                historical_volume['分钟起始
'].astype(str))

        # 清除线路编码空格
        historical_volume['线路编码'] = historical_volume['线路编码'].str.replace(' ', '')
        if '线路编码' in known_volume.columns:
            known_volume['线路编码'] = known_volume['线路编码'].str.replace(' ', '')

        return routes_data, historical_volume, known_volume
    except Exception as e:
        print(f'数据导入异常: {e}')
        return None, None, None

def extract_temporal_features(df, time_column):
    """提取时间特征"""
    df['hour'] = df[time_column].dt.hour
    df['minute'] = df[time_column].dt.minute
    df['weekday'] = df[time_column].dt.dayofweek
    df['day'] = df[time_column].dt.day
    df['month'] = df[time_column].dt.month
    df['is_weekend'] = df['weekday'].apply(lambda x: 1 if x >= 5 else 0)

    # 区分不同时段
    df['day_period'] = df['hour'].apply(lambda x:
                                                'morning' if 6 <= x < 12 else

```

```

        'afternoon' if 12 <= x < 18 else
        'evening' if 18 <= x < 22 else
        'night')

# 区分工作和非工作时段
df['business_hours'] = df['hour'].apply(lambda x: 1 if 8 <= x < 20 else 0)

return df

def generate_lag_features(df, group_col, target_col, time_col, lags=[1, 7]):
    """创建滞后特征 - 例如前 1 天、前 7 天同时段的数据"""
    df = df.sort_values([group_col, time_col])

    # 收集日期
    unique_dates = df[time_col].dt.date.unique()
    unique_dates = sorted(unique_dates)

    # 为每个滞后周期创建特征
    for lag in lags:
        lag_col_name = f'lag_{lag}d'
        df[lag_col_name] = np.nan

    # 按分组处理
    for group in df[group_col].unique():
        group_data = df[df[group_col] == group]

        # 处理每个日期
        for i, current_date in enumerate(unique_dates):
            if i >= lag:
                # 当前日期
                current_mask = (df[group_col] == group) & (df[time_col].dt.date ==
current_date)

                # 滞后日期
                lag_date = unique_dates[i - lag]
                lag_mask = (df[group_col] == group) & (df[time_col].dt.date ==
lag_date)

                # 匹配相同时段数据
                if sum(lag_mask) > 0:
                    lag_data = df.loc[lag_mask, [time_col, target_col]].copy()
                    lag_data['time_minutes'] = lag_data[time_col].dt.hour * 60 +

```

```

lag_data[time_col].dt.minute

        current_data = df.loc[current_mask, [time_col]].copy()
        current_data['time_minutes'] = current_data[time_col].dt.hour *
60 + current_data[time_col].dt.minute

        # 为每个当前时间点找到对应的滞后时间点
        for idx in current_data.index:
            current_minutes = current_data.loc[idx, 'time_minutes']
            closest_idx      =      (lag_data['time_minutes']      -
current_minutes).abs().idxmin()

            lag_value = df.loc[closest_idx, target_col]
            df.loc[idx, lag_col_name] = lag_value

# 处理缺失值
df = df.fillna(method='ffill')
df = df.fillna(method='bfill')

return df

def identify_route_time_window(route_code):
    """识别路线的操作时间窗口"""
    # 解析路线编码中的时间标识
    segments = route_code.split('-')
    if len(segments) >= 3:
        time_code = segments[2]

        # 处理早班路线（0600）
        if time_code == '0600':
            return {
                'start_hour': 21, # 前一天晚上 21 点
                'start_minute': 0,
                'end_hour': 5,    # 当天早上 5 点 50 分
                'end_minute': 50,
                'spans_midnight': True # 跨越午夜
            }

        # 处理午班路线（1400）
        elif time_code == '1400':
            return {
                'start_hour': 11, # 上午 11 点
                'start_minute': 0,
                'end_hour': 13,   # 下午 13 点 50 分

```

```
        'end_minute': 50,
        'spans_midnight': False # 不跨越午夜
    }
# 如果不是标准编码, 尝试判断类型
else:
    # 检查是否包含特定时间标记
    if '0600' in time_code:
        return {
            'start_hour': 21,
            'start_minute': 0,
            'end_hour': 5,
            'end_minute': 50,
            'spans_midnight': True
        }
    elif '1400' in time_code:
        return {
            'start_hour': 11,
            'start_minute': 0,
            'end_hour': 13,
            'end_minute': 50,
            'spans_midnight': False
        }
# 尝试根据前两位数字判断
try:
    hour = int(time_code[:2])
    if 0 <= hour <= 10: # 早间路线视为 0600 类型
        return {
            'start_hour': 21,
            'start_minute': 0,
            'end_hour': 5,
            'end_minute': 50,
            'spans_midnight': True
        }
    else: # 其他时间段路线视为 1400 类型
        return {
            'start_hour': 11,
            'start_minute': 0,
            'end_hour': 13,
            'end_minute': 50,
            'spans_midnight': False
        }
except:
```

```

        # 解析失败时，默认为早班类型
        print(f'提示: 无法确定路线 {route_code} 的时间窗口类型, 使用早班
类型作为默认值")
        return {
            'start_hour': 21,
            'start_minute': 0,
            'end_hour': 5,
            'end_minute': 50,
            'spans_midnight': True
        }

# 编码格式异常时的默认处理
print(f'提示: 路线编码 {route_code} 格式异常, 使用早班类型作为默认值")
return {
    'start_hour': 21,
    'start_minute': 0,
    'end_hour': 5,
    'end_minute': 50,
    'spans_midnight': True
}

def compile_daily_data(historical_data, known_data, route_code, prediction_date):
    """准备日级别预测所需的数据"""
    # 获取路线时间窗口类型
    time_window = identify_route_time_window(route_code)
    spans_midnight = time_window['spans_midnight']

    # 筛选目标路线数据
    daily_data = historical_data[historical_data['线路编码'] == route_code].copy()

    if spans_midnight:
        # 早班路线: 前一晚 21:00 到次日早上 5:50
        daily_data['数据日期'] = daily_data['时间戳'].apply(
            lambda x: x.date() if 0 <= x.hour < 6 else (x + timedelta(days=1)).date()
        )
        # 筛选相关时间段
        daily_data = daily_data[
            ((daily_data['时间戳'].dt.hour >= 21) & (daily_data['时间戳'].dt.hour <= 23)) |
            ((daily_data['时间戳'].dt.hour >= 0) & (daily_data['时间戳'].dt.hour < 6))
        ]
    else:
        # 午班路线: 当天 11:00-13:50

```

```
daily_data['数据日期'] = daily_data['时间戳'].dt.date

# 按日期汇总
daily_totals = daily_data.groupby('数据日期')['包裹量'].sum().reset_index()
daily_totals.rename(columns={'数据日期': '日期'}, inplace=True)

# 添加日期特征
daily_totals['datetime'] = pd.to_datetime(daily_totals['日期'])
daily_totals['weekday'] = daily_totals['datetime'].dt.dayofweek
daily_totals['day'] = daily_totals['datetime'].dt.day
daily_totals['month'] = daily_totals['datetime'].dt.month
daily_totals['is_weekend'] = daily_totals['weekday'].apply(lambda x: 1 if x >= 5 else 0)

# 添加滞后特征
for lag in [1, 7]:
    daily_totals[f'lag_{lag}d'] = daily_totals['包裹量'].shift(lag)

# 处理缺失值
daily_totals = daily_totals.dropna()

return daily_totals

def train_gbr_daily_model(historical_data, known_data, line_code, prediction_date):
    """使用梯度提升回归(GBR)模型预测每日总量"""
    daily_totals, X, y, pred_features, known_amount = prepare_daily_data(historical_data,
known_data, line_code, prediction_date)

    start_time = time.time()

    # 训练梯度提升回归模型
    model = GradientBoostingRegressor(n_estimators=100, random_state=42)
    model.fit(X, y)

    # 预测
    history_based_prediction = model.predict(pred_features)[0]

    end_time = time.time()
    training_time = end_time - start_time

    # 计算模型评估指标
    y_pred = model.predict(X)
    mse = mean_squared_error(y, y_pred)
```

```

rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred)
mae = mean_absolute_error(y, y_pred)

# 如果有预知数据，计算加权预测
if known_amount > 0:
    # 计算历史上预知数据与实际数据的平均比例
    known_actual_ratios = []
    date_column = '预知日期' if '预知日期' in known_data.columns else '日期'
    amount_column = '预知包裹量' if '预知包裹量' in known_data.columns else '包裹
量'

    for index, row in daily_totals.iterrows():
        date = row['日期']
        actual = row['包裹量']
        known_row = known_data[(known_data['线路编码'] == line_code) &
                                (known_data[date_column] == date)]
        if not known_row.empty and amount_column in known_row.columns:
            known = known_row[amount_column].iloc[0]
            if actual > 0: # 避免除零错误
                known_actual_ratios.append(known / actual)

    if known_actual_ratios:
        avg_ratio = np.mean(known_actual_ratios)
        known_adjusted = known_amount / avg_ratio
    else:
        known_adjusted = known_amount

    # 加权平均，给予预知数据更高权重
    final_prediction = 0.3 * history_based_prediction + 0.7 * known_adjusted
else:
    final_prediction = history_based_prediction

# 保存模型评估结果
evaluation = {
    'model_type': 'GradientBoostingRegressor',
    'mse': mse,
    'rmse': rmse,
    'r2': r2,
    'mae': mae,
    'training_time': training_time,
    'prediction': max(0, round(final_prediction)),

```

```

        'y_pred': y_pred # 存储训练集的预测值
    }

    # 保存用于绘图的信息
    model_info = {
        'daily_model': model,
        'daily_eval': evaluation,
        'X': X # 保存特征数据，用于绘制拟合图像
    }

    return model_info, evaluation, max(0, round(final_prediction)), daily_totals

def train_rf_daily_model(historical_data, known_data, line_code, prediction_date):
    """使用随机森林(RF)模型预测每日总量"""
    daily_totals, X, y, pred_features, known_amount = prepare_daily_data(historical_data,
    known_data, line_code, prediction_date)

    start_time = time.time()

    # 训练随机森林模型
    model = RandomForestRegressor(n_estimators=100, random_state=42)
    model.fit(X, y)

    # 预测
    history_based_prediction = model.predict(pred_features)[0]

    end_time = time.time()
    training_time = end_time - start_time

    # 计算模型评估指标
    y_pred = model.predict(X)
    mse = mean_squared_error(y, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y, y_pred)
    mae = mean_absolute_error(y, y_pred)

    # 如果有预知数据，计算加权预测
    if known_amount > 0:
        # 计算历史上预知数据与实际数据的平均比例
        known_actual_ratios = []
        date_column = '预知日期' if '预知日期' in known_data.columns else '日期'
        amount_column = '预知包裹量' if '预知包裹量' in known_data.columns else '包裹

```

量'

```
for index, row in daily_totals.iterrows():
    date = row['日期']
    actual = row['包裹量']
    known_row = known_data[(known_data['线路编码'] == line_code) &
                           (known_data[date_column] == date)]
    if not known_row.empty and amount_column in known_row.columns:
        known = known_row[amount_column].iloc[0]
        if actual > 0: # 避免除零错误
            known_actual_ratios.append(known / actual)

    if known_actual_ratios:
        avg_ratio = np.mean(known_actual_ratios)
        known_adjusted = known_amount / avg_ratio
    else:
        known_adjusted = known_amount

    # 加权平均，给予预知数据更高权重
    final_prediction = 0.3 * history_based_prediction + 0.7 * known_adjusted
else:
    final_prediction = history_based_prediction

# 保存模型评估结果
evaluation = {
    'model_type': 'RandomForestRegressor',
    'mse': mse,
    'rmse': rmse,
    'r2': r2,
    'mae': mae,
    'training_time': training_time,
    'prediction': max(0, round(final_prediction)),
    'y_pred': y_pred # 存储训练集的预测值
}

# 保存用于绘图的信息
model_info = {
    'daily_model': model,
    'daily_eval': evaluation,
    'X': X # 保存特征数据，用于绘制拟合图像
}
```

```

    return model_info, evaluation, max(0, round(final_prediction)), daily_totals

def train_xgb_daily_model(historical_data, known_data, line_code, prediction_date):
    """使用 XGBoost 模型预测每日总量"""
    daily_totals, X, y, pred_features, known_amount = prepare_daily_data(historical_data,
    known_data, line_code, prediction_date)

    start_time = time.time()

    # 训练 XGBoost 模型
    model = xgb.XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=5,
    random_state=42)
    model.fit(X, y)

    # 预测
    history_based_prediction = model.predict(pred_features)[0]

    end_time = time.time()
    training_time = end_time - start_time

    # 计算模型评估指标
    y_pred = model.predict(X)
    mse = mean_squared_error(y, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y, y_pred)
    mae = mean_absolute_error(y, y_pred)

    # 如果有预知数据，计算加权预测
    if known_amount > 0:
        # 计算历史上预知数据与实际数据的平均比例
        known_actual_ratios = []
        date_column = '预知日期' if '预知日期' in known_data.columns else '日期'
        amount_column = '预知包裹量' if '预知包裹量' in known_data.columns else '包裹
量'

        for index, row in daily_totals.iterrows():
            date = row['日期']
            actual = row['包裹量']
            known_row = known_data[(known_data['线路编码'] == line_code) &
            (known_data[date_column] == date)]
            if not known_row.empty and amount_column in known_row.columns:
                known = known_row[amount_column].iloc[0]

```

```

        if actual > 0: # 避免除零错误
            known_actual_ratios.append(known / actual)

    if known_actual_ratios:
        avg_ratio = np.mean(known_actual_ratios)
        known_adjusted = known_amount / avg_ratio
    else:
        known_adjusted = known_amount

    # 加权平均，给予预知数据更高权重
    final_prediction = 0.3 * history_based_prediction + 0.7 * known_adjusted
else:
    final_prediction = history_based_prediction

# 保存模型评估结果
evaluation = {
    'model_type': 'XGBRegressor',
    'mse': mse,
    'rmse': rmse,
    'r2': r2,
    'mae': mae,
    'training_time': training_time,
    'prediction': max(0, round(final_prediction)),
    'y_pred': y_pred # 存储训练集的预测值
}

# 保存用于绘图的信息
model_info = {
    'daily_model': model,
    'daily_eval': evaluation,
    'X': X # 保存特征数据，用于绘制拟合图像
}

return model_info, evaluation, max(0, round(final_prediction)), daily_totals

def train_sarima_daily_model(historical_data, known_data, line_code, prediction_date):
    """使用 SARIMA 模型预测每日总量"""
    daily_totals, X, y, pred_features, known_amount = prepare_daily_data(historical_data,
    known_data, line_code, prediction_date)

    start_time = time.time()

```

```

# 将数据转换为时间序列
daily_totals_ts = daily_totals.set_index('日期')['包裹量']

try:
    # 使用简单的 auto_arima 找到最佳参数
    model_fit, best_order, best_seasonal_order = simple_auto_arima(daily_totals_ts,
seasonal=True, m=7)

    # 预测
    forecast = model_fit.forecast(steps=1)
    history_based_prediction = forecast[0]

    # 计算模型评估指标 - 在训练数据上的拟合情况
    y_pred = model_fit.predict(start=0, end=len(daily_totals_ts)-1)
    mse = mean_squared_error(daily_totals_ts, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(daily_totals_ts, y_pred)
    mae = mean_absolute_error(daily_totals_ts, y_pred)

except Exception as e:
    print(f'SARIMA 模型训练失败({line_code}): {str(e)}")
    # 使用备用方法 - 简单的移动平均
    y_pred = daily_totals_ts.rolling(window=7, min_periods=1).mean()
    mse = mean_squared_error(daily_totals_ts[6:], y_pred[6:]) # 跳过前 6 个 NaN 值
    rmse = np.sqrt(mse)
    r2 = r2_score(daily_totals_ts[6:], y_pred[6:])
    mae = mean_absolute_error(daily_totals_ts[6:], y_pred[6:])

    # 使用移动平均预测
    history_based_prediction = y_pred.iloc[-1]
    model_fit = None

end_time = time.time()
training_time = end_time - start_time

# 如果有预知数据，计算加权预测
if known_amount > 0:
    # 计算历史上预知数据与实际数据的平均比例
    known_actual_ratios = []
    date_column = '预知日期' if '预知日期' in known_data.columns else '日期'
    amount_column = '预知包裹量' if '预知包裹量' in known_data.columns else '包裹
量'

```

```

for index, row in daily_totals.iterrows():
    date = row['日期']
    actual = row['包裹量']
    known_row = known_data[(known_data['线路编码'] == line_code) &
                             (known_data[date_column] == date)]
    if not known_row.empty and amount_column in known_row.columns:
        known = known_row[amount_column].iloc[0]
        if actual > 0: # 避免除零错误
            known_actual_ratios.append(known / actual)

if known_actual_ratios:
    avg_ratio = np.mean(known_actual_ratios)
    known_adjusted = known_amount / avg_ratio
else:
    known_adjusted = known_amount

# 加权平均, 给予预知数据更高权重
final_prediction = 0.3 * history_based_prediction + 0.7 * known_adjusted
else:
    final_prediction = history_based_prediction

# 保存模型评估结果
evaluation = {
    'model_type': 'SARIMA',
    'mse': mse,
    'rmse': rmse,
    'r2': r2,
    'mae': mae,
    'training_time': training_time,
    'prediction': max(0, round(final_prediction)),
    'y_pred': y_pred.values if hasattr(y_pred, 'values') else y_pred # 存储训练集的预
测值
}

# 保存用于绘图的信息
model_info = {
    'daily_model': model_fit,
    'daily_eval': evaluation,
    'daily_totals_ts': daily_totals_ts # 保存时间序列数据, 用于 SARIMA 模型的拟合图像
}

```

```
return model_info, evaluation, max(0, round(final_prediction)), daily_totals
```

```
# 时间分布预测模型 - 四种不同的实现
```

```
def prepare_distribution_data(historical_data, line_code):
    """准备时间分布预测的数据"""
    # 获取线路时间窗口类型
    time_window = get_time_window_for_line(line_code)
    spans_midnight = time_window['spans_midnight']

    # 筛选特定线路的历史数据
    line_data = historical_data[historical_data['线路编码'] == line_code].copy()

    if line_data.empty:
        return None, None, None

    # 筛选相关时间窗口的数据
    if spans_midnight:
        # 对于 0600 结尾的线路，只考虑前一天晚上 21:00 到当天早上 5:50 的数据
        line_data = line_data[
            ((line_data['时间'].dt.hour >= 21) & (line_data['时间'].dt.hour <= 23)) |
            ((line_data['时间'].dt.hour >= 0) & (line_data['时间'].dt.hour < 6))
        ]
        # 为跨午夜线路创建处理日期：如果是 0-6 点，使用当天日期；如果是 21-23
        # 点，使用次日日期
        line_data['处理日期'] = line_data['时间'].apply(
            lambda x: x.date() if 0 <= x.hour < 6 else (x + timedelta(days=1)).date()
        )
    else:
        # 对于 1400 结尾的线路，只考虑 11:00-13:50 的数据
        line_data = line_data[
            (line_data['时间'].dt.hour >= 11) & (line_data['时间'].dt.hour <= 13)
        ]
        line_data['处理日期'] = line_data['时间'].dt.date

    # 按工作日/周末和日期分组，计算每个时间点占当日总量的比例
    line_data['day_of_week'] = line_data['时间'].dt.dayofweek
    line_data['is_weekend'] = line_data['day_of_week'] >= 5
    line_data['time_bin'] = line_data['时间'].dt.hour * 6 + line_data['时间'].dt.minute // 10

    # 计算每天的总包裹量
```

```

daily_totals = line_data.groupby(['处理日期'])['包裹量'].sum().reset_index()
daily_totals.rename(columns={'包裹量': '日总量'}, inplace=True)

# 合并回原数据
line_data = pd.merge(line_data, daily_totals, on='处理日期')

# 计算每个时间点占当日总量的比例
line_data['占比'] = line_data['包裹量'] / line_data['日总量']

# 准备时间段到占比的训练数据
time_bin_data = line_data.groupby(['处理日期', 'time_bin', 'day_of_week',
'is_weekend'])['占比'].mean().reset_index()
time_bin_data = time_bin_data.dropna()

# 提取所有有效的时间段
valid_time_bins = sorted(time_bin_data['time_bin'].unique())

return line_data, time_bin_data, valid_time_bins, spans_midnight

def train_gbr_distribution_model(historical_data, line_code, prediction_date):
    """使用梯度提升回归(GBR)模型预测时间分布"""
    # 获取预测日期的特征
    pred_date = pd.to_datetime(prediction_date)
    day_of_week = pred_date.dayofweek
    is_weekend = 1 if day_of_week >= 5 else 0

    # 获取时间窗口
    time_window = get_time_window_for_line(line_code)
    spans_midnight = time_window['spans_midnight']
    start_hour = time_window['start_hour']
    end_hour = time_window['end_hour']

    # 准备训练数据
    line_data, time_bin_data, valid_time_bins, spans_midnight =
prepare_distribution_data(historical_data, line_code)

    if time_bin_data is None or time_bin_data.empty:
        return None, None, {}

    # 存储每个时间段的预测模型
    time_bin_models = {}

```

```

# 存储所有时间段的实际分布，用于绘图
actual_distribution = {}

# 存储预测的时间分布
time_distribution = {}

start_time = time.time()

# 对每个时间段训练一个模型
for time_bin in valid_time_bins:
    group = time_bin_data[time_bin_data['time_bin'] == time_bin].copy()

    # 对于 0600 结尾的线路，只考虑 21:00-23:50 和 00:00-05:50 的时间段
    hour = time_bin // 6
    if spans_midnight:
        if not (21 <= hour <= 23 or 0 <= hour < 6):
            continue
    # 对于 1400 结尾的线路，只考虑 11:00-13:50 的时间段
    else:
        if not (11 <= hour <= 13):
            continue

    # 至少需要 5 个数据点才能训练模型
    if len(group) < 5:
        # 对于数据不足的时段，使用简单平均
        avg_ratio = group['占比'].mean()
        if pd.isna(avg_ratio):
            avg_ratio = 0

        # 获取前一天和前一周的比例
        prev_day_ratio = group[group['处理日期'] == (pred_date -
pd.Timedelta(days=1)).date())['占比'].mean() \
            if (pred_date - pd.Timedelta(days=1)).date() in group['处理日期'].values
        else avg_ratio
        prev_week_ratio = group[group['处理日期'] == (pred_date -
pd.Timedelta(days=7)).date())['占比'].mean() \
            if (pred_date - pd.Timedelta(days=7)).date() in group['处理日期'].values
        else avg_ratio

        if pd.isna(prev_day_ratio):
            prev_day_ratio = avg_ratio
        if pd.isna(prev_week_ratio):

```

```

        prev_week_ratio = avg_ratio

    time_bin_models[time_bin] = {
        'avg_ratio': avg_ratio,
        'latest_ratio': group['占比'].iloc[-1] if not group.empty else 0,
        'prev_day_ratio': prev_day_ratio,
        'prev_week_ratio': prev_week_ratio,
        'model_type': 'avg'
    }

    # 保存该时间段的实际分布
    actual_distribution[time_bin] = avg_ratio

    # 保存预测分布
    time_distribution[time_bin] = avg_ratio

    continue

# 添加滞后特征（前一天和前一周同一时间点的占比）
group = group.sort_values(['处理日期'])
group['prev_day_ratio'] = group['占比'].shift(1)
group['prev_week_ratio'] = group['占比'].shift(7)
group = group.dropna()

if len(group) < 3: # 滞后特征可能导致一些行被删除
    continue

# 准备训练特征
X = group[['day_of_week', 'is_weekend', 'prev_day_ratio', 'prev_week_ratio']]
y = group['占比']

# 训练模型
model = GradientBoostingRegressor(n_estimators=50, learning_rate=0.1,
max_depth=3, random_state=42)
model.fit(X, y)

# 获取前一天和前一周的比例
prev_day_ratio = group[group['处理日期'] == (pred_date -
pd.Timedelta(days=1)).date()]['占比'].mean() \
    if (pred_date - pd.Timedelta(days=1)).date() in group['处理日期'].values else
y.mean()
prev_week_ratio = group[group['处理日期'] == (pred_date -

```

```

pd.Timedelta(days=7)).date()]['占比'].mean() \
    if (pred_date - pd.Timedelta(days=7)).date() in group['处理日期'].values else
y.mean()

if pd.isna(prev_day_ratio):
    prev_day_ratio = y.mean()
if pd.isna(prev_week_ratio):
    prev_week_ratio = y.mean()

# 准备预测特征
pred_features = pd.DataFrame({
    'day_of_week': [day_of_week],
    'is_weekend': [is_weekend],
    'prev_day_ratio': [prev_day_ratio],
    'prev_week_ratio': [prev_week_ratio]
})

# 预测该时间段的比例
predicted_ratio = model.predict(pred_features)[0]

# 确保比例在合理范围内
predicted_ratio = max(0.0, min(1.0, predicted_ratio))

# 计算训练集上的预测值，用于评估
y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)

# 保存模型和相关信息
time_bin_models[time_bin] = {
    'model': model,
    'avg_ratio': y.mean(),
    'latest_ratio': y.iloc[-1] if not y.empty else 0,
    'prev_day_ratio': prev_day_ratio,
    'prev_week_ratio': prev_week_ratio,
    'model_type': 'gbr',
    'mse': mse,
    'y_pred': y_pred
}

# 保存该时间段的实际分布
actual_distribution[time_bin] = y.mean()

```

```

        # 保存预测分布
        time_distribution[time_bin] = predicted_ratio

    end_time = time.time()
    training_time = end_time - start_time

    # 准备评估结果
    evaluation = {
        'model_type': 'GradientBoostingRegressor',
        'training_time': training_time,
        'time_distribution': time_distribution,
        'actual_distribution': actual_distribution
    }

    # 确保时间分布的总和为 1
    total = sum(time_distribution.values())
    if total > 0:
        for time_bin in time_distribution:
            time_distribution[time_bin] /= total

    return time_bin_models, evaluation, time_distribution

def train_rf_distribution_model(historical_data, line_code, prediction_date):
    """使用随机森林(RF)模型预测时间分布"""
    line_data, time_bin_data, valid_time_bins, spans_midnight = \
    prepare_distribution_data(historical_data, line_code)

    if time_bin_data is None or time_bin_data.empty:
        return None, None, {}

    start_time = time.time()

    # 为每个时间点创建预测模型
    time_bin_models = {}
    time_bin_evaluations = {}

    # 对每个时间点进行建模
    for time_bin in valid_time_bins:
        bin_data = time_bin_data[time_bin_data['time_bin'] == time_bin].copy()

        if len(bin_data) < 5: # 如果数据太少，使用全局平均值
            continue

```

```

# 添加滞后特征（前一天和前一周同一时间点的占比）
bin_data = bin_data.sort_values(['处理日期'])
bin_data['prev_day_ratio'] = bin_data['占比'].shift(1)
bin_data['prev_week_ratio'] = bin_data['占比'].shift(7)
bin_data = bin_data.dropna()

if len(bin_data) < 3: # 滞后特征可能导致一些行被删除
    continue

# 准备训练特征
features = ['day_of_week', 'is_weekend', 'prev_day_ratio', 'prev_week_ratio']
X = bin_data[features]
y = bin_data['占比']

# 训练随机森林模型
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X, y)

# 计算评估指标
y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred) if len(y) > 1 else 0
mae = mean_absolute_error(y, y_pred)

# 存储模型和评估结果
time_bin_models[time_bin] = {
    'model': model,
    'avg_ratio': bin_data['占比'].mean(), # 备用：平均值
    'latest_ratio': bin_data['占比'].iloc[-1], # 备用：最新值
    'prev_day_ratio': bin_data['占比'].iloc[-1], # 用于预测的前一天比例
    'prev_week_ratio': bin_data['占比'].iloc[-7] if len(bin_data) >= 7 else
bin_data['占比'].mean() # 用于预测的前一周比例
}

time_bin_evaluations[time_bin] = {
    'mse': mse,
    'rmse': rmse,
    'r2': r2,
    'mae': mae
}

```

```
end_time = time.time()
training_time = end_time - start_time

# 预测指定日期的时间分布
pred_date = pd.to_datetime(prediction_date)
day_of_week = pred_date.dayofweek
is_weekend = 1 if day_of_week >= 5 else 0

# 生成预测比例
predicted_ratios = {}

# 对于每个时间段，使用训练好的模型预测占比
for time_bin in range(144): # 24 小时 * 6 个 10 分钟时段
    # 对于 0600 结尾的线路，只考虑 21:00-23:50 和 00:00-05:50 的时间段
    if spans_midnight:
        hour = time_bin // 6
        if not (21 <= hour <= 23 or 0 <= hour < 6):
            continue
    # 对于 1400 结尾的线路，只考虑 11:00-13:50 的时间段
    else:
        hour = time_bin // 6
        if not (11 <= hour <= 13):
            continue

    if time_bin in time_bin_models:
        model_info = time_bin_models[time_bin]

        # 准备预测特征
        pred_features = pd.DataFrame({
            'day_of_week': [day_of_week],
            'is_weekend': [is_weekend],
            'prev_day_ratio': [model_info['prev_day_ratio']],
            'prev_week_ratio': [model_info['prev_week_ratio']]
        })

        # 预测占比
        try:
            ratio = model_info['model'].predict(pred_features)[0]
            # 确保比例在合理范围内
            ratio = max(0.0, min(1.0, ratio))
        except:
```

```

        # 如果预测失败，使用备用平均值
        ratio = model_info['avg_ratio']
    else:
        # 如果没有该时间点的模型，使用时间段的全局平均值
        similar_time_bins = [tb for tb in time_bin_models.keys()
                             if abs(tb - time_bin) <= 6] # 在同一小时左右的时
间点

        if similar_time_bins:
            ratio = np.mean([time_bin_models[tb]['avg_ratio'] for tb in
similar_time_bins])
        else:
            # 如果没有类似时间点，使用全局平均值
            if spans_midnight:
                # 跨午夜的线路有 9 小时有效时间（21:00-05:50）
                ratio = 1.0 / (9 * 6) # 9 小时，每小时 6 个 10 分钟时段
            else:
                # 非跨午夜的线路有 3 小时有效时间（11:00-13:50）
                ratio = 1.0 / (3 * 6) # 3 小时，每小时 6 个 10 分钟时段

    predicted_ratios[time_bin] = ratio

# 归一化比例，确保总和为 1
total_ratio = sum(predicted_ratios.values())
if total_ratio > 0:
    normalized_ratios = {tb: r/total_ratio for tb, r in predicted_ratios.items()}
else:
    print(f"警告：线路 {line_code} 的总预测比例为 0，使用均匀分布")
    # 创建均匀分布
    valid_time_bins_count = len(predicted_ratios)
    normalized_ratios = {tb: 1.0/valid_time_bins_count for tb in
predicted_ratios.keys()} if valid_time_bins_count > 0 else {}

# 总体评估指标（所有时间段的平均）
overall_evaluation = {
    'model_type': 'RandomForestRegressor',
    'mse': np.mean([eval_dict['mse'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'rmse': np.mean([eval_dict['rmse'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'r2': np.mean([eval_dict['r2'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,

```

```

        'mae': np.mean([eval_dict['mae'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
        'training_time': training_time
    }

    return time_bin_models, overall_evaluation, normalized_ratios

def train_xgb_distribution_model(historical_data, line_code, prediction_date):
    """使用 XGBoost 模型预测时间分布"""
    line_data, time_bin_data, valid_time_bins, spans_midnight =
prepare_distribution_data(historical_data, line_code)

    if time_bin_data is None or time_bin_data.empty:
        return None, None, {}

    start_time = time.time()

    # 为每个时间点创建预测模型
    time_bin_models = {}
    time_bin_evaluations = {}

    # 对每个时间点进行建模
    for time_bin in valid_time_bins:
        bin_data = time_bin_data[time_bin_data['time_bin'] == time_bin].copy()

        if len(bin_data) < 5: # 如果数据太少，使用全局平均值
            continue

        # 添加滞后特征（前一天和前一周同一时间点的占比）
        bin_data = bin_data.sort_values(['处理日期'])
        bin_data['prev_day_ratio'] = bin_data['占比'].shift(1)
        bin_data['prev_week_ratio'] = bin_data['占比'].shift(7)
        bin_data = bin_data.dropna()

        if len(bin_data) < 3: # 滞后特征可能导致一些行被删除
            continue

        # 准备训练特征
        features = ['day_of_week', 'is_weekend', 'prev_day_ratio', 'prev_week_ratio']
        X = bin_data[features]
        y = bin_data['占比']

```

```

# 训练 XGBoost 模型
model = xgb.XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=5,
random_state=42)
model.fit(X, y)

# 计算评估指标
y_pred = model.predict(X)
mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred) if len(y) > 1 else 0
mae = mean_absolute_error(y, y_pred)

# 存储模型和评估结果
time_bin_models[time_bin] = {
    'model': model,
    'avg_ratio': bin_data['占比'].mean(), # 备用：平均值
    'latest_ratio': bin_data['占比'].iloc[-1], # 备用：最新值
    'prev_day_ratio': bin_data['占比'].iloc[-1], # 用于预测的前一天比例
    'prev_week_ratio': bin_data['占比'].iloc[-7] if len(bin_data) >= 7 else
bin_data['占比'].mean() # 用于预测的前一周比例
}

time_bin_evaluations[time_bin] = {
    'mse': mse,
    'rmse': rmse,
    'r2': r2,
    'mae': mae
}

end_time = time.time()
training_time = end_time - start_time

# 预测指定日期的时间分布
pred_date = pd.to_datetime(prediction_date)
day_of_week = pred_date.dayofweek
is_weekend = 1 if day_of_week >= 5 else 0

# 生成预测比例
predicted_ratios = {}

# 对于每个时间段，使用训练好的模型预测占比
for time_bin in range(144): # 24 小时 * 6 个 10 分钟时段

```

```

# 对于 0600 结尾的线路，只考虑 21:00-23:50 和 00:00-05:50 的时间段
if spans_midnight:
    hour = time_bin // 6
    if not (21 <= hour <= 23 or 0 <= hour < 6):
        continue
# 对于 1400 结尾的线路，只考虑 11:00-13:50 的时间段
else:
    hour = time_bin // 6
    if not (11 <= hour <= 13):
        continue

if time_bin in time_bin_models:
    model_info = time_bin_models[time_bin]

    # 准备预测特征
    pred_features = pd.DataFrame({
        'day_of_week': [day_of_week],
        'is_weekend': [is_weekend],
        'prev_day_ratio': [model_info['prev_day_ratio']],
        'prev_week_ratio': [model_info['prev_week_ratio']]
    })

    # 预测占比
    try:
        ratio = model_info['model'].predict(pred_features)[0]
        # 确保比例在合理范围内
        ratio = max(0.0, min(1.0, ratio))
    except:
        # 如果预测失败，使用备用平均值
        ratio = model_info['avg_ratio']
else:
    # 如果没有该时间点的模型，使用时间段的全局平均值
    similar_time_bins = [tb for tb in time_bin_models.keys()
                        if abs(tb - time_bin) <= 6] # 在同一小时左右的时
间点

    if similar_time_bins:
        ratio = np.mean([time_bin_models[tb]['avg_ratio'] for tb in
similar_time_bins])
    else:
        # 如果没有类似时间点，使用全局平均值
        if spans_midnight:

```

```

        # 跨午夜的线路有 9 小时有效时间（21:00-05:50）
        ratio = 1.0 / (9 * 6) # 9 小时，每小时 6 个 10 分钟时段
    else:
        # 非跨午夜的线路有 3 小时有效时间（11:00-13:50）
        ratio = 1.0 / (3 * 6) # 3 小时，每小时 6 个 10 分钟时段

    predicted_ratios[time_bin] = ratio

# 归一化比例，确保总和为 1
total_ratio = sum(predicted_ratios.values())
if total_ratio > 0:
    normalized_ratios = {tb: r/total_ratio for tb, r in predicted_ratios.items()}
else:
    print(f"警告：线路 {line_code} 的总预测比例为 0，使用均匀分布")
    # 创建均匀分布
    valid_time_bins_count = len(predicted_ratios)
    normalized_ratios = {tb: 1.0/valid_time_bins_count for tb in
predicted_ratios.keys()} if valid_time_bins_count > 0 else {}

# 总体评估指标（所有时间段的平均）
overall_evaluation = {
    'model_type': 'XGBRegressor',
    'mse': np.mean([eval_dict['mse'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'rmse': np.mean([eval_dict['rmse'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'r2': np.mean([eval_dict['r2'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'mae': np.mean([eval_dict['mae'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'training_time': training_time
}

return time_bin_models, overall_evaluation, normalized_ratios

def predict_10min_distribution(total_prediction, time_distribution, start_time, end_time,
line_code):
    """生成 10 分钟颗粒度预测"""
    # 获取该线路的有效时间窗口
    time_window = get_time_window_for_line(line_code)
    spans_midnight = time_window['spans_midnight']

```

```

# 创建有效时间点列表
valid_time_points = []

# 确定预测范围内的日期
dates_in_range = []
current_date = start_time.date()
while current_date <= end_time.date():
    dates_in_range.append(current_date)
    current_date += timedelta(days=1)

# 对于 0600 结尾的线路（跨午夜）
if spans_midnight:
    for current_date in dates_in_range:
        # 晚上时间段 (21:00-23:50)
        evening_start = datetime.combine(current_date, datetime.min.time()).replace(
            hour=time_window['start_hour'],
            minute=time_window['start_minute']
        )
        midnight = datetime.combine(current_date + timedelta(days=1),
datetime.min.time())

        evening_points = pd.date_range(
            start=evening_start,
            end=midnight - timedelta(minutes=10), # 直到 23:50
            freq='10min'
        )

        # 早上时间段 (00:00-05:50)
        morning_start = datetime.combine(current_date, datetime.min.time())
        morning_end = datetime.combine(current_date, datetime.min.time()).replace(
            hour=time_window['end_hour'],
            minute=time_window['end_minute']
        )

        morning_points = pd.date_range(
            start=morning_start,
            end=morning_end,
            freq='10min'
        )

        # 合并当天有效的时间点
        day_valid_points = []

```

据

```
# 只有当当前日期的晚上在预测时间范围内时，才添加晚上的时间点
for tp in evening_points:
    if start_time <= tp < end_time:
        day_valid_points.append(tp)

# 只有当第二天的早上在预测时间范围内时，才添加早上的时间点
for tp in morning_points:
    # 注意：这里的 morning_points 是当天的早上，对应前一天晚上的数

    # 所以我们要检查的是当天而不是下一天
    if start_time <= tp < end_time:
        day_valid_points.append(tp)

valid_time_points.extend(day_valid_points)

# 对于 1400 结尾的线路（非跨午夜）
else:
    for current_date in dates_in_range:
        # 创建当天的时间窗口（11:00-13:50）
        day_start = datetime.combine(current_date, datetime.min.time().replace(
            hour=time_window['start_hour'],
            minute=time_window['start_minute']
        ))
        day_end = datetime.combine(current_date, datetime.min.time().replace(
            hour=time_window['end_hour'],
            minute=time_window['end_minute']
        ))

        day_points = pd.date_range(
            start=day_start,
            end=day_end,
            freq='10min'
        )

        # 只保留在预测范围内的时间点
        day_valid_points = []
        for tp in day_points:
            if start_time <= tp < end_time:
                day_valid_points.append(tp)

        valid_time_points.extend(day_valid_points)
```

```

# 排序时间点
valid_time_points = sorted(valid_time_points)

# 如果没有有效时间点，直接返回空 DataFrame
if not valid_time_points:
    return pd.DataFrame()

# 创建结果 DataFrame
result_df = pd.DataFrame()

# 计算这些时间点在总体预测中应占的比例
total_prediction_amount = 0
time_bin_to_amount = {}

# 对每个有效时间点进行预测
for tp in valid_time_points:
    # 计算时间点对应的 bin
    time_bin = tp.hour * 6 + tp.minute // 10

    # 获取该时间点的预测比例
    if time_distribution and time_bin in time_distribution:
        ratio = time_distribution[time_bin]
    else:
        # 如果没有该时间点的分布信息，使用均匀分布
        ratio = 1.0 / len(valid_time_points)

    # 计算该时间点的预测量
    predicted_amount = total_prediction * ratio
    time_bin_to_amount[time_bin] = predicted_amount
    total_prediction_amount += predicted_amount

# 调整各时间点的预测量，确保总和等于总预测量
adjustment_factor = total_prediction / total_prediction_amount if
total_prediction_amount > 0 else 0

# 对每个有效时间点创建结果行
for tp in valid_time_points:
    time_bin = tp.hour * 6 + tp.minute // 10
    adjusted_amount = time_bin_to_amount.get(time_bin, 0) * adjustment_factor

# 添加到结果

```

```

        new_row = pd.DataFrame({
            '日期': [tp.date()],
            '时间段开始': [f'{tp.hour:02d}:{tp.minute:02d}'],
            '时间段结束': [f'{(tp + pd.Timedelta(minutes=10)).hour:02d}:{(tp +
pd.Timedelta(minutes=10)).minute:02d}'],
            '预测货量': [round(adjusted_amount)]
        })
        result_df = pd.concat([result_df, new_row], ignore_index=True)

# 最终调整，确保总和精确等于总预测量
current_total = result_df['预测货量'].sum()
if current_total != total_prediction and len(result_df) > 0:
    # 计算需要调整的量
    adjustment = total_prediction - current_total

    # 找出最主要的时段进行调整
    if adjustment > 0:
        # 按预测货量降序排序，增加主要时段的货量
        sorted_indices = result_df['预测货量'].sort_values(ascending=False).index
    else:
        # 按预测货量升序排序，减少次要时段的货量
        sorted_indices = result_df['预测货量'].sort_values(ascending=True).index

    # 逐个时间点调整，直到达到总量
    adjustment_left = abs(adjustment)
    for idx in sorted_indices:
        if adjustment_left == 0:
            break

        if adjustment > 0:
            result_df.loc[idx, '预测货量'] += 1
        else:
            # 确保不会减到负值
            if result_df.loc[idx, '预测货量'] > 0:
                result_df.loc[idx, '预测货量'] -= 1

        adjustment_left -= 1

    return result_df

def plot_results(line_code, daily_data, model_evaluations, model_names, prediction_date):
    """绘制日粒度的预测结果对比"""

```

```
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

# 创建一个大图
fig = plt.figure(figsize=(20, 12))
ax = fig.add_subplot(111)

# 使用原始数据中的日期和实际值
dates = daily_data['日期']
actual_values = daily_data['包裹量']

# 绘制实际值
ax.plot(dates, actual_values, 'k-', linewidth=3, label='实际值')

# 颜色和线型
colors = ['red', 'blue', 'green', 'purple', 'orange', 'brown', 'pink']
linestyles = ['-', '--', '-.', ':', (0, (3, 1, 1, 1))]

# 绘制每个模型的预测值
for i, model_name in enumerate(model_names):
    if model_name in model_evaluations:
        daily_eval = model_evaluations[model_name].get('daily_eval', {})

        # 选择颜色和线型
        color = colors[i % len(colors)]
        linestyle = linestyles[i % len(linestyles)]

        # 如果有训练集上的预测值，绘制拟合曲线
        if 'y_pred' in daily_eval:
            ax.plot(dates, daily_eval['y_pred'], linestyle=linestyle, color=color,
                    linewidth=3, label=f'{model_name}')

# 设置坐标轴标签
ax.set_xlabel('日期', fontsize=20)
ax.set_ylabel('货量', fontsize=20)
ax.grid(True)

# 调整刻度字体大小
ax.tick_params(axis='both', which='major', labelsize=18)

# 设置合适的日期格式
date_format = mdates.DateFormatter('%Y-%m-%d')
```

```

ax.xaxis.set_major_formatter(date_format)

# 根据日期数量确定合适的刻度间隔
if len(dates) > 30:
    interval = len(dates) // 15 # 设置约 15 个刻度
    ax.xaxis.set_major_locator(mdates.DayLocator(interval=max(1, interval)))
else:
    ax.xaxis.set_major_locator(mdates.DayLocator(interval=max(1, len(dates)//10)))

plt.xticks(rotation=45)

# 添加图例
ax.legend(loc='upper right', fontsize=18)

plt.tight_layout()
plt.savefig(f'线路{line_code}_所有模型对比.png', dpi=300, bbox_inches='tight')
plt.close()

return fig

def evaluate_models(line_codes, historical_packages, known_packages):
    """评估不同模型组合的性能"""
    # 设置预测的时间范围和日期
    prediction_date = datetime(2024, 12, 15).date()
    prediction_start = datetime(2024, 12, 15, 14, 0)
    prediction_end = datetime(2024, 12, 16, 14, 0)

    # 模型组合
    model_combinations = [
        ('GradientBoosting', 'GradientBoosting'), # GBR 日总量 + GBR 时间分布
        ('SARIMA', 'SARIMA'), # SARIMA 日总量 +
SARIMA 时间分布
        ('XGBoost', 'XGBoost'), # XGBoost 日总量 + XGBoost 时
间分布
        ('RandomForest', 'RandomForest'), # RF 日总量 + RF 时间分布
        ('XGBoost', 'LSTM') # XGBoost 日总量 + LSTM 时
间分布
    ]

    # 存储评估结果
    all_evaluations = {}

```

```

# 每个模型组合的整体指标
overall_metrics = pd.DataFrame(columns=['模型组合', 'MSE', 'RMSE', 'R2', 'MAE', '训练时间(秒)', '预测时间(秒)'])

for line_code in line_codes:
    print(f"\n 开始评估线路 {line_code} 的不同模型组合...")

    line_evaluations = {}
    daily_data = None # 用于存储最后一次训练的日总量数据，供绘图使用

    # 对每个模型组合进行训练和评估
    for daily_model_name, distribution_model_name in model_combinations:
        model_key = f"{daily_model_name}-{distribution_model_name}"
        print(f" 正在训练模型组合: {model_key}...")

        start_time = time.time()

        # 训练日总量预测模型
        if daily_model_name == 'GradientBoosting':
            daily_model_info, daily_eval, total_prediction, daily_data =
train_gbr_daily_model(
                historical_packages, known_packages, line_code, prediction_date
            )
        elif daily_model_name == 'RandomForest':
            daily_model_info, daily_eval, total_prediction, daily_data =
train_rf_daily_model(
                historical_packages, known_packages, line_code, prediction_date
            )
        elif daily_model_name == 'XGBoost':
            daily_model_info, daily_eval, total_prediction, daily_data =
train_xgb_daily_model(
                historical_packages, known_packages, line_code, prediction_date
            )
        elif daily_model_name == 'SARIMA':
            daily_model_info, daily_eval, total_prediction, daily_data =
train_sarima_daily_model(
                historical_packages, known_packages, line_code, prediction_date
            )

        # 训练时间分布预测模型
        if distribution_model_name == 'GradientBoosting':
            _, distribution_eval, time_distribution = train_gbr_distribution_model(

```

```

        historical_packages, line_code, prediction_date
    )
    elif distribution_model_name == 'RandomForest':
        _, distribution_eval, time_distribution = train_rf_distribution_model(
            historical_packages, line_code, prediction_date
        )
    elif distribution_model_name == 'XGBoost':
        _, distribution_eval, time_distribution = train_xgb_distribution_model(
            historical_packages, line_code, prediction_date
        )
    elif distribution_model_name == 'SARIMA':
        _, distribution_eval, time_distribution = train_sarima_distribution_model(
            historical_packages, line_code, prediction_date
        )
    elif distribution_model_name == 'LSTM':
        _, distribution_eval, time_distribution = train_lstm_distribution_model(
            historical_packages, line_code, prediction_date
        )

    # 生成 10 分钟颗粒度预测
    prediction_start_time = time.time()
    detailed_predictions = predict_10min_distribution(
        total_prediction, time_distribution, prediction_start, prediction_end,
line_code
    )
    prediction_time = time.time() - prediction_start_time

    training_end_time = time.time()
    total_training_time = training_end_time - start_time

    # 确保评估指标不是 NaN
    for metric in ['mse', 'rmse', 'r2', 'mae']:
        if np.isnan(daily_eval.get(metric, np.nan)):
            daily_eval[metric] = 0 # 替换 NaN 为 0, 避免汇总时出现 NaN

    # 保存评估结果
    line_evaluations[model_key] = {
        'daily_model': daily_model_info.get('daily_model') if
isinstance(daily_model_info, dict) else daily_model_info,
        'daily_eval': daily_eval,
        'distribution_eval': distribution_eval,
        'total_prediction': total_prediction,

```

```

        'detailed_predictions': detailed_predictions,
        'training_time': total_training_time,
        'prediction_time': prediction_time,
        'X': daily_model_info.get('X') if isinstance(daily_model_info, dict) else
None
    }

    print(f" {model_key} 评估完成: 日总量预测={total_prediction}, 训练时
间={total_training_time:.2f}秒")

    # 绘制模型评估结果 - 修改这里, 传递完整的模型组合键
    plot_results(line_code, daily_data, line_evaluations, [f"{combo[0]}-{combo[1]}"
for combo in model_combinations], prediction_date)

    # 保存该线路的评估结果
    all_evaluations[line_code] = line_evaluations

    # 更新整体指标
    for daily_model_name, distribution_model_name in model_combinations:
        model_key = f"{daily_model_name}-{distribution_model_name}"
        eval_data = line_evaluations[model_key]

        # 确保所有指标都有有效值
        mse = eval_data['daily_eval'].get('mse', 0)
        mse = 0 if np.isnan(mse) else mse

        rmse = eval_data['daily_eval'].get('rmse', 0)
        rmse = 0 if np.isnan(rmse) else rmse

        r2 = eval_data['daily_eval'].get('r2', 0)
        r2 = 0 if np.isnan(r2) else r2

        mae = eval_data['daily_eval'].get('mae', 0)
        mae = 0 if np.isnan(mae) else mae

        new_row = {
            '模型组合': model_key,
            'MSE': mse,
            'RMSE': rmse,
            'R2': r2,
            'MAE': mae,
            '训练时间(秒)': eval_data['training_time'],

```

```

        '预测时间(秒)': eval_data['prediction_time']
    }

    overall_metrics = pd.concat([overall_metrics, pd.DataFrame([new_row])],
                                ignore_index=True)

    # 计算每个模型组合的平均指标
    model_avg_metrics = overall_metrics.groupby('模型组合').mean().reset_index()

    # 选择最佳模型组合
    best_model_by_mse = model_avg_metrics.loc[model_avg_metrics['MSE'].idxmin()]
    best_model_by_rmse = model_avg_metrics.loc[model_avg_metrics['RMSE'].idxmin()]
    best_model_by_r2 = model_avg_metrics.loc[model_avg_metrics['R2'].idxmax()]

    print("\n 模型评估结果汇总:")
    print(model_avg_metrics)

    print(f"\n 按 MSE 最小选择的最佳模型: {best_model_by_mse['模型组合']}")
    print(f"按 RMSE 最小选择的最佳模型: {best_model_by_rmse['模型组合']}")
    print(f"按 R2 最大选择的最佳模型: {best_model_by_r2['模型组合']}")

    # 将评估结果保存到 excel
    with pd.ExcelWriter('模型评估结果.xlsx') as writer:
        # 保存总评估结果
        model_avg_metrics.to_excel(writer, sheet_name='总体评估', index=False)

        # 为每个线路创建一个工作表
        for line_code in all_evaluations:
            line_data = []
            for model_key, eval_data in all_evaluations[line_code].items():
                line_data.append({
                    '模型组合': model_key,
                    'MSE': eval_data['daily_eval'].get('mse', 0),
                    'RMSE': eval_data['daily_eval'].get('rmse', 0),
                    'R2': eval_data['daily_eval'].get('r2', 0),
                    'MAE': eval_data['daily_eval'].get('mae', 0),
                    '预测总量': eval_data['total_prediction'],
                    '训练时间(秒)': eval_data['training_time'],
                    '预测时间(秒)': eval_data['prediction_time']
                })

    line_df = pd.DataFrame(line_data)

```

```

        line_df.to_excel(writer, sheet_name=f'线路{line_code[:10]}', index=False)

    print("\n 评估结果已保存到 '模型评估结果.xlsx'")

    return all_evaluations, model_avg_metrics

def plot_10min_distributions(line_code, model_evaluations, model_names, prediction_date,
historical_packages=None):
    """绘制 10 分钟颗粒度的预测对比图，显示连续两天数据，第一天有真实值对比"""
    plt.rcParams['font.sans-serif'] = ['SimHei']  # 用来正常显示中文标签
    plt.rcParams['axes.unicode_minus'] = False  # 用来正常显示负号

    # 创建图形
    fig = plt.figure(figsize=(20, 12))
    ax = fig.add_subplot(111)

    # 为每个模型选择不同的颜色和线型
    colors = ['red', 'blue', 'green', 'purple', 'orange', 'brown', 'pink', 'gray', 'cyan', 'magenta']
    linestyles = ['-', '--', '-.', ':', (0, (3, 1, 1, 1)), (0, (5, 1)), (0, (3, 5, 1, 5))]

    # 创建真实值数据
    actual_data_df = None

    # 查看历史数据有哪些列
    if historical_packages is not None:
        # 检查历史数据的列
        hist_columns = historical_packages.columns.tolist()
        print(f'历史数据列: {hist_columns}')

        # 获取历史时间区间
        prev_date = prediction_date - timedelta(days=1)  # 前一天

        # 筛选线路和日期 - 增加更多历史数据
        line_data = historical_packages[historical_packages['线路编码'] == line_code]
        # 包含前后多天数据以确保连续性
        date_range = [(prev_date - timedelta(days=1)).strftime('%Y-%m-%d'),
                        prev_date.strftime('%Y-%m-%d'),
                        prediction_date.strftime('%Y-%m-%d')]
        actual_data = line_data[pd.to_datetime(line_data['日期']).dt.strftime('%Y-%m-%d').isin(date_range)]

    if not actual_data.empty:

```

```

print(f'找到历史数据: {len(actual_data)}行')

# 打印第一行历史数据样本
print(f'历史数据样本:\n{actual_data.iloc[0].to_dict()}')

# 创建一个包含所需列的 DataFrame
actual_data_df = pd.DataFrame({
    '日期': pd.to_datetime(actual_data['日期'])
})

# 根据历史数据列动态添加
if '时间段开始' in hist_columns:
    actual_data_df['时间段开始'] = actual_data['时间段开始']
elif '分钟起始' in hist_columns:
    actual_data_df['时间段开始'] = actual_data['分钟起始']
elif '时间段' in hist_columns:
    actual_data_df['时间段开始'] = actual_data['时间段']
else:
    # 如果没有时间列，创建模拟的时间点
    hour_minutes = []
    for hour in range(0, 24):
        for minute in range(0, 60, 10): # 每 10 分钟
            hour_minutes.append(f'{hour:02d}:{minute:02d}')

    # 如果历史数据和时间点数量不匹配，适当裁剪或重复
    time_points = hour_minutes[:len(actual_data)] if len(hour_minutes) >=
len(actual_data) else hour_minutes * (len(actual_data) // len(hour_minutes) + 1)
    time_points = time_points[:len(actual_data)]

    actual_data_df['时间段开始'] = time_points

# 添加实际货量列
if '实际货量' in hist_columns:
    actual_data_df['实际货量'] = actual_data['实际货量']
elif '包裹量' in hist_columns:
    actual_data_df['实际货量'] = actual_data['包裹量']

# 找出所有模型的 10 分钟颗粒度预测数据
all_detailed_predictions = {}
for i, model_name in enumerate(model_names):
    if 'detailed_predictions' in model_evaluations[model_name]:
        detailed_preds = model_evaluations[model_name]['detailed_predictions']

```

```

        if not detailed_preds.empty:
            # 打印第一行预测数据样本
            print(f" 模型 {model_name} 预测数据样本:\n{detailed_preds.iloc[0].to_dict()}")

            # 检查是否有必要的列
            if '日期' in detailed_preds.columns and '时间段开始' in detailed_preds.columns and '预测货量' in detailed_preds.columns:
                all_detailed_predictions[model_name] = detailed_preds
                print(f"模型 {model_name} 的预测数据: {len(detailed_preds)}行")
            ")

        else:
            # 检查可能的替代列名
            alt_columns = detailed_preds.columns.tolist()
            date_col = None
            time_col = None
            pred_col = None

            # 寻找可能的日期列
            for col in alt_columns:
                if '日期' in col or 'date' in col.lower():
                    date_col = col
                    break

            # 寻找可能的时间列
            for col in alt_columns:
                if '时间' in col or 'time' in col.lower() or '分钟' in col:
                    time_col = col
                    break

            # 寻找可能的预测列
            for col in alt_columns:
                if '预测' in col or 'pred' in col.lower() or '货量' in col:
                    pred_col = col
                    break

            if date_col and time_col and pred_col:
                # 创建一个新的 DataFrame, 使用标准列名
                df_standardized = pd.DataFrame({
                    '日期': detailed_preds[date_col],
                    '时间段开始': detailed_preds[time_col],
                    '预测货量': detailed_preds[pred_col]

```

```

        })
        all_detailed_predictions[model_name] = df_standardized
        print(f" 模型  {model_name}  的预测数据已标准化：
{len(df_standardized)}行")
    else:
        print(f" 模型  {model_name}  的预测数据缺少必要的列：
{alt_columns}")

    if not all_detailed_predictions:
        print(f"没有找到线路 {line_code} 的 10 分钟颗粒度预测数据")
        # 如果没有预测数据但有真实数据，可以只显示真实数据
        if actual_data_df is None or actual_data_df.empty:
            print("也没有找到真实数据，无法绘制图表")
            return None

    # 设置更宽的时间窗口以确保连续性
    start_time = prediction_date - timedelta(days=2) # 前两天
    end_time = prediction_date + timedelta(days=2)   # 后两天
    time_window = (start_time, end_time)
    print(f"时间窗口: {start_time} 到 {end_time}")

    # 创建连续的时间点列表（每 10 分钟一个点）
    start_timestamp = pd.to_datetime(f"{start_time} 00:00:00")
    end_timestamp = pd.to_datetime(f"{end_time} 23:59:00")
    all_timestamps = []
    current = start_timestamp
    while current <= end_timestamp:
        all_timestamps.append(current)
        current += pd.Timedelta(minutes=10)

    print(f"生成连续时间点: {len(all_timestamps)}个")

    # 如果有实际值数据，创建实际值时间系列
    actual_time_series = pd.Series(index=all_timestamps, dtype=float)
    actual_time_series[:] = np.nan # 初始化为 NaN

    if actual_data_df is not None and not actual_data_df.empty and '实际货量' in
actual_data_df.columns:
        for i, row in actual_data_df.iterrows():
            try:
                date = row['日期']
                if isinstance(date, str):

```

```

        date = pd.to_datetime(date)

        date_str = date.strftime('%Y-%m-%d')
        time_str = row['时间段开始']
        try:
            time_point = pd.to_datetime(f"{date_str} {time_str}")
        except:
            # 如果时间字符串格式有问题, 尝试修复
            if ':' not in time_str:
                # 如果是纯数字, 假设是分钟
                minutes = int(time_str)
                hours = minutes // 60
                mins = minutes % 60
                time_str = f"{hours:02d}:{mins:02d}"
                time_point = pd.to_datetime(f"{date_str} {time_str}")
            else:
                raise

        # 确保时间点在我们的时间窗口内
        if time_window[0] <= time_point.date() <= time_window[1]:
            # 找到最接近的时间点
            nearest_timestamp = min(all_timestamps, key=lambda x: abs(x -
time_point))

            actual_time_series[nearest_timestamp] = row['实际货量']
        except Exception as e:
            print(f'处理真实值时序数据时出错: {e}, 行数据: {row}')

# 为每个模型创建预测时间系列
model_time_series = {}
for model_name, df in all_detailed_predictions.items():
    # 初始化模型预测时间序列
    model_series = pd.Series(index=all_timestamps, dtype=float)
    model_series[:] = np.nan

    for j, row in df.iterrows():
        try:
            date = row['日期']
            if isinstance(date, str):
                date = pd.to_datetime(date)
            elif isinstance(date, pd.Timestamp):
                pass
            else:

```

```

        date = pd.to_datetime(date)

        date_str = date.strftime('%Y-%m-%d')
        time_str = row['时间段开始']

        # 处理可能的时间格式问题
        try:
            time_point = pd.to_datetime(f"{date_str} {time_str}")
        except:
            # 如果时间字符串格式有问题，尝试修复
            if ':' not in time_str:
                # 如果是纯数字，假设是分钟
                minutes = int(time_str)
                hours = minutes // 60
                mins = minutes % 60
                time_str = f"{hours:02d}:{mins:02d}"
                time_point = pd.to_datetime(f"{date_str} {time_str}")
            else:
                # 如果还是失败，跳过
                continue

        # 确保时间点在我们的时间窗口内
        if time_window[0] <= time_point.date() <= time_window[1]:
            # 找到最接近的时间点
            nearest_timestamp = min(all_timestamps, key=lambda x: abs(x -
time_point))

            model_series[nearest_timestamp] = row['预测货量']
        except Exception as e:
            print(f'处理模型 {model_name} 预测时序数据时出错: {e}')

        # 添加到模型时间序列字典
        model_time_series[model_name] = model_series

    # 找到预测日期的午夜时间点
    prediction_midnight = pd.to_datetime(f"{prediction_date} 00:00:00")

    # 绘制实际值
    actual_valid = actual_time_series.dropna()
    if not actual_valid.empty:
        print(f'绘制真实值数据点: {len(actual_valid)}个')
        ax.plot(actual_valid.index, actual_valid.values, 'k-', linewidth=4, label='实际值')
        # 添加散点增强可视性

```

```

        ax.scatter(actual_valid.index, actual_valid.values, color='black', s=80, zorder=5)
    else:
        print("没有真实值数据点可以绘制")

# 为每个模型绘制预测曲线
for i, model_name in enumerate(model_time_series.keys()):
    model_series = model_time_series[model_name]
    color = colors[i % len(colors)]
    linestyle = linestyles[i % len(linestyles)]

    # 获取有效的预测值
    model_valid = model_series.dropna()

    if not model_valid.empty:
        print(f"绘制模型 {model_name} 预测数据点: {len(model_valid)}个")

        # 将模型预测与实际值最后一个点连接起来（如果有实际值）
        if not actual_valid.empty:
            # 获取实际值的最后一个时间点和值
            last_actual_time = actual_valid.index[-1]
            last_actual_value = actual_valid.values[-1]

            # 获取模型预测的第一个时间点和值
            first_model_time = model_valid.index[0]
            first_model_value = model_valid.values[0]

            # 如果最后一个实际值和第一个预测值之间有间隔，进行线性插值
            # 确保时间顺序正确
            if last_actual_time < first_model_time:
                time_gap = (first_model_time - last_actual_time).total_seconds() /

60

                # 如果间隔大于特定阈值（例如 30 分钟）
                if time_gap > 30:
                    # 创建中间的连接点
                    connect_times = []
                    connect_values = []

                    # 线性插值
                    steps = int(time_gap / 10) # 每 10 分钟一个点
                    for step in range(1, steps):
                        interp_time = last_actual_time +

```

```

pd.Timedelta(minutes=10*step)
    # 线性插值计算中间值
    weight = step / steps
    interp_value = last_actual_value * (1 - weight) +
first_model_value * weight

    connect_times.append(interp_time)
    connect_values.append(interp_value)

    # 绘制连接线
    all_times = [last_actual_time] + connect_times +
[first_model_time]
    all_values = [last_actual_value] + connect_values +
[first_model_value]
    ax.plot(all_times, all_values, linestyle='--', color=color,
alpha=0.7, linewidth=2)

    # 绘制预测曲线
    ax.plot(model_valid.index, model_valid.values, linestyle=linestyle,
color=color,
        linewidth=3, label=f'{model_name}')
    else:
        print(f"模型 {model_name} 没有有效的预测数据点")

# 添加垂直线标记预测日期的开始
ax.axvline(x=prediction_midnight, color='black', linestyle='--', alpha=0.5)

# 添加文本标签
y_range = ax.get_ylim()
y_max = y_range[1]
y_min = y_range[0]
y_pos = y_min + 0.95 * (y_max - y_min)

ax.text(prediction_midnight - pd.Timedelta(hours=2), y_pos, '历史数据',
        fontsize=20, ha='right', va='top')
ax.text(prediction_midnight + pd.Timedelta(hours=2), y_pos, '预测数据',
        fontsize=20, ha='left', va='top')

# 设置坐标轴标签
ax.set_xlabel('时间', fontsize=20)
ax.set_ylabel('货量', fontsize=20)
ax.grid(True)

```

```

# 调整刻度字体大小
ax.tick_params(axis='both', which='major', labelsize=18)

# 调整 x 轴日期格式
ax.xaxis.set_major_formatter(mdates.DateFormatter('%m-%d %H:%M'))

# 根据时间点数量确定刻度间隔
ax.xaxis.set_major_locator(mdates.HourLocator(interval=4)) # 每 4 小时一个刻度

plt.xticks(rotation=45)

# 处理图例 - 放在图片内部右上角
ax.legend(loc='upper right', fontsize=20)

plt.tight_layout()
plt.savefig(f'线路{line_code}_10 分钟颗粒度对比.png', dpi=300, bbox_inches='tight')
plt.close()

print(f'已为线路 {line_code} 生成 10 分钟颗粒度对比图 (连续两天数据, 含真实值)')

return fig

def main():
    """主函数"""
    # 加载数据
    print("加载数据中...")
    routes_data, historical_volume, known_volume = import_dataset()

    if routes_data is None:
        print("数据加载失败, 程序退出。")
        return

    # 选择一条线路进行评估
    line_codes = historical_volume['线路编码'].unique()
    print(f'发现 {len(line_codes)} 条唯一线路')

    # 优先选择一条 0600 结尾的线路
    selected_line_code = None
    for line in line_codes:
        if line.endswith('0600'):
            selected_line_code = line

```

break

```
# 如果没有找到 0600 结尾的线路，选择第一条线路
if selected_line_code is None:
    selected_line_code = line_codes[0]

selected_line_codes = [selected_line_code]

print(f"选择了线路 {selected_line_code} 进行评估")

# 评估不同模型组合
print("\n 开始评估不同模型组合...")
start_time = time.time()
evaluations, model_avg_metrics = evaluate_models(selected_line_codes,
historical_volume, known_volume)
end_time = time.time()

print(f"\n 评估完成，总耗时: {end_time - start_time:.2f} 秒")
print(f"评估结果已保存到 '模型评估结果.xlsx'")
print("\n 模型评估结果汇总:")
print(model_avg_metrics)

# 找出性能最好的模型组合
best_model_r2 = model_avg_metrics.loc[model_avg_metrics['R2'].idxmax()]
best_model_mse = model_avg_metrics.loc[model_avg_metrics['MSE'].idxmin()]
best_model_rmse = model_avg_metrics.loc[model_avg_metrics['RMSE'].idxmin()]
best_model_mae = model_avg_metrics.loc[model_avg_metrics['MAE'].idxmin()]
fastest_model = model_avg_metrics.loc[model_avg_metrics['训练时间(秒)'].idxmin()]

print(f"\n 各指标最佳模型组合:")
print(f"R2 最高的模型组合: {best_model_r2['模型组合']}, R2 = {best_model_r2['R2']:.4f}")
print(f"MSE 最低的模型组合: {best_model_mse['模型组合']}, MSE = {best_model_mse['MSE']:.4f}")
print(f"RMSE 最低的模型组合: {best_model_rmse['模型组合']}, RMSE = {best_model_rmse['RMSE']:.4f}")
print(f"MAE 最低的模型组合: {best_model_mae['模型组合']}, MAE = {best_model_mae['MAE']:.4f}")
print(f"训练速度最快的模型组合: {fastest_model['模型组合']}, 平均训练时间 = {fastest_model['训练时间(秒)']:.2f} 秒")

# 创建评估指标对比图
```

```
plt.figure(figsize=(16, 12))

# 设置 matplotlib 支持中文
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

# 各模型 MSE 对比
ax1 = plt.subplot(221)
bars = ax1.bar(model_avg_metrics['模型组合'], model_avg_metrics['MSE'])
ax1.set_title('各模型组合的平均 MSE')
ax1.tick_params(axis='x', rotation=45)
# 在柱状图上标注数值
for bar in bars:
    height = bar.get_height()
    ax1.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.2f}', ha='center', va='bottom')

# 各模型 RMSE 对比
ax2 = plt.subplot(222)
bars = ax2.bar(model_avg_metrics['模型组合'], model_avg_metrics['RMSE'])
ax2.set_title('各模型组合的平均 RMSE')
ax2.tick_params(axis='x', rotation=45)
# 在柱状图上标注数值
for bar in bars:
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.2f}', ha='center', va='bottom')

# 各模型 R2 对比
ax3 = plt.subplot(223)
bars = ax3.bar(model_avg_metrics['模型组合'], model_avg_metrics['R2'])
ax3.set_title('各模型组合的平均 R2')
ax3.tick_params(axis='x', rotation=45)
# 在柱状图上标注数值
for bar in bars:
    height = bar.get_height()
    ax3.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.4f}', ha='center', va='bottom')

# 各模型训练时间对比
ax4 = plt.subplot(224)
bars = ax4.bar(model_avg_metrics['模型组合'], model_avg_metrics['训练时间(秒)'])
```

```

ax4.set_title('各模型组合的平均训练时间(秒)')
ax4.tick_params(axis='x', rotation=45)
# 在柱状图上标注数值
for bar in bars:
    height = bar.get_height()
    ax4.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.2f}', ha='center', va='bottom')

plt.tight_layout()
plt.savefig('模型评估结果对比.png', dpi=300)
plt.close()

print(f'模型评估结果图表已保存到 模型评估结果对比.png')

# 添加 10 分钟颗粒度的预测对比图
prediction_date = datetime(2024, 12, 15).date()
line_eval = evaluations[selected_line_code]
model_names = [f"{combo[0]}-{combo[1]}" for combo in [
    ('GradientBoosting', 'GradientBoosting'),
    ('SARIMA', 'SARIMA'),
    ('XGBoost', 'XGBoost'),
    ('RandomForest', 'RandomForest'),
    ('XGBoost', 'LSTM')
]]

plot_10min_distributions(selected_line_code, line_eval, model_names, prediction_date,
historical_volume)

return evaluations, model_avg_metrics

def train_sarima_distribution_model(historical_data, line_code, prediction_date):
    """使用时间序列模型预测时间分布（改为使用指数平滑和滚动平均模型代替
    SARIMA）"""
    line_data, time_bin_data, valid_time_bins, spans_midnight =
prepare_distribution_data(historical_data, line_code)

    if time_bin_data is None or time_bin_data.empty:
        print(f'线路 {line_code} 没有有效的时间分布数据，将返回空结果')
        return None, None, {}

    start_time = time.time()

```

```

# 为每个时间点创建预测模型
time_bin_models = {}
time_bin_evaluations = {}

# 导入指数平滑模型
from statsmodels.tsa.holtwinters import ExponentialSmoothing

# 对每个时间点进行建模
for time_bin in valid_time_bins:
    bin_data = time_bin_data[time_bin_data['time_bin'] == time_bin].copy()

    if len(bin_data) < 7: # 至少需要 7 个数据点
        print(f"时间段 {time_bin} 的数据点数量不足 ({len(bin_data)} < 7)，使用
简单平均值")
        # 直接使用平均值，不训练模型
        time_bin_models[time_bin] = {
            'model_type': 'avg',
            'avg_ratio': bin_data['占比'].mean(),
            'latest_ratio': bin_data['占比'].iloc[-1] if not bin_data.empty else 0
        }
        continue

    try:
        # 准备时间序列数据
        ts_data = bin_data.set_index('处理日期')['占比']

        # 检查数据是否足够变化
        if ts_data.nunique() <= 1:
            print(f"时间段 {time_bin} 的数据没有变化（所有值都相同），使用
平均值")

            time_bin_models[time_bin] = {
                'model_type': 'avg',
                'avg_ratio': ts_data.mean(),
                'latest_ratio': ts_data.iloc[-1] if not ts_data.empty else 0
            }
            continue

        # 根据数据量选择模型
        if len(ts_data) < 14:
            # 数据少于两周，使用 7 天滚动平均
            rolling_mean = ts_data.rolling(window=min(7, len(ts_data)-1),
min_periods=1).mean()

```

```
# 计算评估指标
y_pred = rolling_mean # 滑动平均作为预测
y = ts_data

# 舍弃 NaN 值
mask = ~np.isnan(y_pred)
if sum(mask) > 1: # 确保有足够的非 NaN 值
    y_pred = y_pred[mask]
    y = y[mask]
    mse = mean_squared_error(y, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y, y_pred)
    mae = mean_absolute_error(y, y_pred)
else:
    mse = rmse = r2 = mae = np.nan

# 预测：使用最后 7 天的平均值
last_n_values = ts_data.iloc[-min(7, len(ts_data)):]
prediction = last_n_values.mean()

# 存储模型和结果
time_bin_models[time_bin] = {
    'model_type': 'rolling_mean',
    'window': min(7, len(ts_data)-1),
    'prediction': prediction,
    'avg_ratio': ts_data.mean(),
    'latest_ratio': ts_data.iloc[-1]
}

time_bin_evaluations[time_bin] = {
    'mse': mse,
    'rmse': rmse,
    'r2': r2,
    'mae': mae
}

print(f"时间段 {time_bin} 的滚动平均模型训练成功: MSE={mse}")
else:
    # 数据足够，使用指数平滑
    try:
        # 使用指数平滑模型 - 不使用季节性
```

```

model = ExponentialSmoothing(
    ts_data,
    trend='add', # 添加趋势
    seasonal=None, # 不使用季节性成分
    initialization_method='estimated'
)
fitted_model = model.fit()

# 计算评估指标
y_pred = fitted_model.fittedvalues
y = ts_data

mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y, y_pred) if len(y) > 1 else 0
mae = mean_absolute_error(y, y_pred)

# 存储模型和评估结果
time_bin_models[time_bin] = {
    'model_type': 'exp_smoothing',
    'model': fitted_model,
    'avg_ratio': ts_data.mean(),
    'latest_ratio': ts_data.iloc[-1]
}

time_bin_evaluations[time_bin] = {
    'mse': mse,
    'rmse': rmse,
    'r2': r2,
    'mae': mae
}

print(f"时间段 {time_bin} 的指数平滑模型训练成功：
MSE={mse:.6f}, R2={r2:.6f}")
except Exception as es_error:
    print(f"指数平滑模型失败: {es_error}, 使用滚动平均")
    # 回退到滚动平均
    rolling_mean = ts_data.rolling(window=7, min_periods=1).mean()

# 计算评估指标（同上）
y_pred = rolling_mean.dropna()
y = ts_data.iloc[len(ts_data)-len(y_pred):]

```

```

        if len(y) > 1:
            mse = mean_squared_error(y, y_pred)
            rmse = np.sqrt(mse)
            r2 = r2_score(y, y_pred)
            mae = mean_absolute_error(y, y_pred)
        else:
            mse = rmse = r2 = mae = np.nan

        # 预测：使用最后 7 天的平均值
        prediction = ts_data.iloc[-7:].mean()

        # 存储模型和结果
        time_bin_models[time_bin] = {
            'model_type': 'rolling_mean',
            'window': 7,
            'prediction': prediction,
            'avg_ratio': ts_data.mean(),
            'latest_ratio': ts_data.iloc[-1]
        }

        time_bin_evaluations[time_bin] = {
            'mse': mse,
            'rmse': rmse,
            'r2': r2,
            'mae': mae
        }
    except Exception as e:
        print(f"时间段 {time_bin} 的模型训练失败: {e}")
        print(f"数据统计：长度={len(bin_data)}, 最小值={bin_data['占比'].min()},
        最大值={bin_data['占比'].max()}, 唯一值数量={bin_data['占比'].nunique()}")
        continue

    end_time = time.time()
    training_time = end_time - start_time

    # 预测指定日期的时间分布
    pred_date = pd.to_datetime(prediction_date)
    day_of_week = pred_date.dayofweek
    is_weekend = 1 if day_of_week >= 5 else 0

    # 生成预测比例

```

```

predicted_ratios = {}

# 对于每个时间段，使用训练好的模型预测占比
for time_bin in range(144): # 24 小时 * 6 个 10 分钟时段
    # 对于 0600 结尾的线路，只考虑 21:00-23:50 和 00:00-05:50 的时间段
    if spans_midnight:
        hour = time_bin // 6
        if not (21 <= hour <= 23 or 0 <= hour < 6):
            continue
    # 对于 1400 结尾的线路，只考虑 11:00-13:50 的时间段
    else:
        hour = time_bin // 6
        if not (11 <= hour <= 13):
            continue

    if time_bin in time_bin_models:
        model_info = time_bin_models[time_bin]

        # 预测占比
        try:
            # 根据模型类型选择预测方法
            model_type = model_info.get('model_type', '')

            if model_type == 'exp_smoothing':
                # 使用指数平滑模型预测
                ratio = model_info['model'].forecast(1)[0]
            elif model_type == 'rolling_mean':
                # 使用之前计算的滚动平均预测
                ratio = model_info['prediction']
            elif model_type == 'avg':
                # 使用简单平均值
                ratio = model_info['avg_ratio']
            else:
                # 默认使用最近值
                ratio = model_info['latest_ratio']

            # 确保比例在合理范围内
            ratio = max(0.0, min(1.0, ratio))
        except Exception as pred_e:
            print(f'时间段 {time_bin} 的预测失败，使用备用平均值: {pred_e}')
            # 如果预测失败，使用备用平均值
            ratio = model_info['avg_ratio']

```

```

else:
    # 如果没有该时间点的模型，使用时间段的全局平均值
    similar_time_bins = [tb for tb in time_bin_models.keys()
                          if abs(tb - time_bin) <= 6] # 在同一小时左右的时
间点

    if similar_time_bins:
        ratio = np.mean([time_bin_models[tb]['avg_ratio'] for tb in
similar_time_bins])
    else:
        # 如果没有类似时间点，使用全局平均值
        if spans_midnight:
            # 跨午夜的线路有 9 小时有效时间（21:00-05:50）
            ratio = 1.0 / (9 * 6) # 9 小时，每小时 6 个 10 分钟时段
        else:
            # 非跨午夜的线路有 3 小时有效时间（11:00-13:50）
            ratio = 1.0 / (3 * 6) # 3 小时，每小时 6 个 10 分钟时段

    predicted_ratios[time_bin] = ratio

# 归一化比例，确保总和为 1
total_ratio = sum(predicted_ratios.values())
if total_ratio > 0:
    normalized_ratios = {tb: r/total_ratio for tb, r in predicted_ratios.items()}
else:
    print(f'警告：线路 {line_code} 的总预测比例为 0，使用均匀分布')
    # 创建均匀分布
    valid_time_bins_count = len(predicted_ratios)
    normalized_ratios = {tb: 1.0/valid_time_bins_count for tb in
predicted_ratios.keys()} if valid_time_bins_count > 0 else {}

# 总体评估指标（所有时间段的平均）
overall_evaluation = {
    'model_type': 'Time Series',
    'mse': np.mean([eval_dict['mse'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'rmse': np.mean([eval_dict['rmse'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'r2': np.mean([eval_dict['r2'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,
    'mae': np.mean([eval_dict['mae'] for eval_dict in time_bin_evaluations.values()]) if
time_bin_evaluations else np.nan,

```

```

        'training_time': training_time,
        'success_rate': len(time_bin_models) / len(valid_time_bins) if valid_time_bins else
0
    }

    print(f" 线路 {line_code} 的时间序列模型训练完成：成功率
={overall_evaluation['success_rate']:.2%}, 平均 MSE={overall_evaluation['mse']}")

    return time_bin_models, overall_evaluation, normalized_ratios

def train_lstm_distribution_model(historical_data, line_code, prediction_date):
    """使用 LSTM 神经网络模型预测时间分布"""
    line_data, time_bin_data, valid_time_bins, spans_midnight =
prepare_distribution_data(historical_data, line_code)

    if time_bin_data is None or time_bin_data.empty:
        print(f"线路 {line_code} 没有有效的时间分布数据，将返回空结果")
        return None, None, {}

    start_time = time.time()

    # 导入需要的库
    try:
        import tensorflow as tf
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import LSTM, Dense, Dropout
        from tensorflow.keras.callbacks import EarlyStopping
        from sklearn.preprocessing import MinMaxScaler
        print("TensorFlow 版本:", tf.__version__)
    except ImportError:
        print("TensorFlow 未安装，将使用滚动平均法替代 LSTM")
        # 如果 TensorFlow 未安装，使用 SARIMA 方法代替
        return train_sarima_distribution_model(historical_data, line_code, prediction_date)

    # 为每个时间点创建预测模型
    time_bin_models = {}
    time_bin_evaluations = {}

    # 设置 tensorflow 日志级别
    tf.get_logger().setLevel('ERROR')

    # 函数：创建序列数据

```

```

def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i:i + seq_length])
        y.append(data[i + seq_length])
    return np.array(X), np.array(y)

# 对每个时间点进行建模
for time_bin in valid_time_bins:
    bin_data = time_bin_data[time_bin_data['time_bin'] == time_bin].copy()

    # LSTM 需要较多数据
    if len(bin_data) < 20:
        print(f"时间段 {time_bin} 的数据点数量不足 ({len(bin_data)} < 20), 使用
简单平均值")
        # 直接使用平均值, 不训练模型
        time_bin_models[time_bin] = {
            'model_type': 'avg',
            'avg_ratio': bin_data['占比'].mean(),
            'latest_ratio': bin_data['占比'].iloc[-1] if not bin_data.empty else 0
        }

    # 添加评估指标, 即使使用平均值也要有评估
    time_bin_evaluations[time_bin] = {
        'mse': np.nan,
        'rmse': np.nan,
        'r2': np.nan,
        'mae': np.nan
    }
    continue

try:
    # 准备时间序列数据
    ts_data = bin_data.set_index('处理日期')['占比'].values.reshape(-1, 1)

    # 检查数据是否足够变化
    if np.std(ts_data) < 1e-6: # 几乎没有变化
        print(f"时间段 {time_bin} 的数据几乎没有变化, 使用平均值")
        time_bin_models[time_bin] = {
            'model_type': 'avg',
            'avg_ratio': np.mean(ts_data),
            'latest_ratio': ts_data[-1][0] if len(ts_data) > 0 else 0

```

```

    }

    # 添加评估指标
    time_bin_evaluations[time_bin] = {
        'mse': np.nan,
        'rmse': np.nan,
        'r2': np.nan,
        'mae': np.nan
    }
    continue

# 标准化数据
scaler = MinMaxScaler(feature_range=(0, 1))
ts_data_scaled = scaler.fit_transform(ts_data)

# 创建序列数据 (使用过去 7 天预测下一天)
seq_length = min(7, len(ts_data_scaled) // 2) # 确保序列长度不超过数据
长度的一半
X, y = create_sequences(ts_data_scaled, seq_length)

# 如果数据点太少, 使用滚动平均
if len(X) < 10:
    print(f'时间段 {time_bin} 序列数据量不足 ({len(X)} < 10), 使用滚动
平均")

    # 计算 7 天滚动平均
    rolling_mean = pd.Series(ts_data.flatten()).rolling(window=min(7,
len(ts_data)-1), min_periods=1).mean().values

# 存储滚动平均模型
time_bin_models[time_bin] = {
    'model_type': 'rolling_mean',
    'window': min(7, len(ts_data)-1),
    'prediction': np.mean(ts_data[-min(7, len(ts_data)):] ), # 最后 7 天
的平均值

    'avg_ratio': np.mean(ts_data),
    'latest_ratio': ts_data[-1][0] if len(ts_data) > 0 else 0,
    'scaler': scaler # 保存 scaler 用于反标准化
}

# 计算评估指标
y_true = ts_data.flatten()[seq_length:]
y_pred = rolling_mean[seq_length:]

```

```

        if len(y_true) > 1:
            mse = mean_squared_error(y_true, y_pred)
            rmse = np.sqrt(mse)
            r2 = r2_score(y_true, y_pred)
            mae = mean_absolute_error(y_true, y_pred)
        else:
            mse = rmse = r2 = mae = np.nan

        time_bin_evaluations[time_bin] = {
            'mse': mse,
            'rmse': rmse,
            'r2': r2,
            'mae': mae
        }
        continue

# 划分训练集和测试集
split = int(len(X) * 0.8)
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# 调整数据形状为 LSTM 输入格式: [samples, time steps, features]
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

# 构建 LSTM 模型
model = Sequential()
model.add(LSTM(24, input_shape=(seq_length, 1), return_sequences=True))
model.add(Dropout(0.1))
model.add(LSTM(12))
model.add(Dropout(0.1))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# 早停回调
early_stop = EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True)

# 训练模型
history = model.fit(
    X_train, y_train,

```

```

        epochs=100,
        batch_size=min(8, len(X_train)),
        validation_data=(X_test, y_test),
        callbacks=[early_stop],
        verbose=0
    )

    # 评估模型
    y_pred_scaled = model.predict(X_test)
    y_pred = scaler.inverse_transform(y_pred_scaled)
    y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))

    # 计算评估指标
    mse = mean_squared_error(y_test_actual, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test_actual, y_pred)
    mae = mean_absolute_error(y_test_actual, y_pred)

    # 生成预测
    # 使用最后 seq_length 个数据点预测下一个值
    last_sequence = ts_data_scaled[-seq_length:].reshape(1, seq_length, 1)
    next_pred_scaled = model.predict(last_sequence)
    next_pred = scaler.inverse_transform(next_pred_scaled)[0, 0]

    # 确保预测值在合理范围内
    next_pred = max(0.0, min(1.0, next_pred))

    # 存储模型和评估结果
    time_bin_models[time_bin] = {
        'model_type': 'lstm',
        'model': model,
        'scaler': scaler,
        'seq_length': seq_length,
        'last_sequence': ts_data_scaled[-seq_length:],
        'prediction': next_pred,
        'avg_ratio': np.mean(ts_data),
        'latest_ratio': ts_data[-1][0]
    }

    time_bin_evaluations[time_bin] = {
        'mse': mse,
        'rmse': rmse,

```

```

        'r2': r2,
        'mae': mae
    }

    print(f'时间段 {time_bin} 的 LSTM 模型训练成功: MSE={mse:.6f},
R2={r2:.6f}')
    except Exception as e:
        print(f'时间段 {time_bin} 的 LSTM 模型训练失败: {e}')
        print(f'数据统计: 长度={len(bin_data)}, 最小值={bin_data['占比'].min()},
最大值={bin_data['占比'].max()}, 唯一值数量={bin_data['占比'].nunique()}')

    # 使用简单平均作为备用
    time_bin_models[time_bin] = {
        'model_type': 'avg',
        'avg_ratio': bin_data['占比'].mean() if not bin_data.empty else 0,
        'latest_ratio': bin_data['占比'].iloc[-1] if not bin_data.empty else 0
    }

    # 添加评估指标
    time_bin_evaluations[time_bin] = {
        'mse': np.nan,
        'rmse': np.nan,
        'r2': np.nan,
        'mae': np.nan
    }
    continue

end_time = time.time()
training_time = end_time - start_time

# 预测指定日期的时间分布
pred_date = pd.to_datetime(prediction_date)
day_of_week = pred_date.dayofweek
is_weekend = 1 if day_of_week >= 5 else 0

# 生成预测比例
predicted_ratios = {}

# 对于每个时间段，使用训练好的模型预测占比
for time_bin in range(144): # 24 小时 * 6 个 10 分钟时段
    # 对于 0600 结尾的线路，只考虑 21:00-23:50 和 00:00-05:50 的时间段
    if spans_midnight:

```

```

        hour = time_bin // 6
        if not (21 <= hour <= 23 or 0 <= hour < 6):
            continue
# 对于 1400 结尾的线路，只考虑 11:00-13:50 的时间段
else:
    hour = time_bin // 6
    if not (11 <= hour <= 13):
        continue

if time_bin in time_bin_models:
    model_info = time_bin_models[time_bin]

    # 预测占比
    try:
        # 根据模型类型选择预测方法
        model_type = model_info.get('model_type', '')

        if model_type == 'lstm':
            # 使用预先计算的预测值
            ratio = model_info['prediction']
        elif model_type == 'rolling_mean':
            # 使用之前计算的滚动平均预测
            ratio = model_info['prediction']
        elif model_type == 'avg':
            # 使用简单平均值
            ratio = model_info['avg_ratio']
        else:
            # 默认使用最近值
            ratio = model_info['latest_ratio']

        # 确保比例在合理范围内
        ratio = max(0.0, min(1.0, ratio))
    except Exception as pred_e:
        print(f"时间段 {time_bin} 的预测失败，使用备用平均值: {pred_e}")
        # 如果预测失败，使用备用平均值
        ratio = model_info['avg_ratio']
else:
    # 如果没有该时间点的模型，使用时间段的全局平均值
    similar_time_bins = [tb for tb in time_bin_models.keys()
                        if abs(tb - time_bin) <= 6] # 在同一小时左右的时

```

间点

```

        if similar_time_bins:
            ratio = np.mean([time_bin_models[tb]['avg_ratio'] for tb in
similar_time_bins])
        else:
            # 如果没有类似时间点，使用全局平均值
            if spans_midnight:
                # 跨午夜的线路有 9 小时有效时间（21:00-05:50）
                ratio = 1.0 / (9 * 6) # 9 小时，每小时 6 个 10 分钟时段
            else:
                # 非跨午夜的线路有 3 小时有效时间（11:00-13:50）
                ratio = 1.0 / (3 * 6) # 3 小时，每小时 6 个 10 分钟时段

    predicted_ratios[time_bin] = ratio

# 归一化比例，确保总和为 1
total_ratio = sum(predicted_ratios.values())
if total_ratio > 0:
    normalized_ratios = {tb: r/total_ratio for tb, r in predicted_ratios.items()}
else:
    print(f"警告：线路 {line_code} 的总预测比例为 0，使用均匀分布")
    # 创建均匀分布
    valid_time_bins_count = len(predicted_ratios)
    normalized_ratios = {tb: 1.0/valid_time_bins_count for tb in
predicted_ratios.keys()} if valid_time_bins_count > 0 else {}

# 总体评估指标（所有时间段的平均）- 过滤掉 NaN 值
valid_mse = [eval_dict['mse'] for eval_dict in time_bin_evaluations.values() if not
np.isnan(eval_dict['mse'])]
valid_rmse = [eval_dict['rmse'] for eval_dict in time_bin_evaluations.values() if not
np.isnan(eval_dict['rmse'])]
valid_r2 = [eval_dict['r2'] for eval_dict in time_bin_evaluations.values() if not
np.isnan(eval_dict['r2'])]
valid_mae = [eval_dict['mae'] for eval_dict in time_bin_evaluations.values() if not
np.isnan(eval_dict['mae'])]

overall_evaluation = {
    'model_type': 'LSTM/Neural',
    'mse': np.mean(valid_mse) if valid_mse else np.nan,
    'rmse': np.mean(valid_rmse) if valid_rmse else np.nan,
    'r2': np.mean(valid_r2) if valid_r2 else np.nan,
    'mae': np.mean(valid_mae) if valid_mae else np.nan,
    'training_time': training_time,

```

```
        'success_rate': len([m for m in time_bin_models.values() if m.get('model_type') ==  
'lstm']) / len(valid_time_bins) if valid_time_bins else 0  
    }
```

```
    print(f" 线路    {line_code} 的 LSTM 模型训练完成：成功率  
={overall_evaluation['success_rate']:.2%}, 平均 MSE={overall_evaluation['mse']}")
```

```
    return time_bin_models, overall_evaluation, normalized_ratios
```

```
# 替代 auto_arima 的简单函数
```

```
def simple_auto_arima(timeseries, seasonal=True, m=7):
```

```
    """一个简单的替代 auto_arima 的函数，尝试几种常见的 ARIMA 参数组合"""
```

```
    best_model = None
```

```
    best_aic = float('inf')
```

```
    best_order = None
```

```
    best_seasonal_order = None
```

```
# 对于非季节性模型，尝试的参数组合
```

```
orders = [(1,0,0), (0,1,1), (1,1,1), (2,1,2)]
```

```
# 季节性参数组合
```

```
seasonal_orders = [(0,0,0,0)]
```

```
if seasonal:
```

```
    seasonal_orders = [(1,0,0,m), (0,1,1,m)]
```

```
for order in orders:
```

```
    for seasonal_order in seasonal_orders:
```

```
        try:
```

```
            if seasonal:
```

```
                model = SARIMAX(timeseries, order=order,  
seasonal_order=seasonal_order)
```

```
            else:
```

```
                model = ARIMA(timeseries, order=order)
```

```
        model_fit = model.fit(dispatch=False)
```

```
        aic = model_fit.aic
```

```
        if aic < best_aic:
```

```
            best_aic = aic
```

```
            best_model = model_fit
```

```
            best_order = order
```

```
            best_seasonal_order = seasonal_order
```

```

        except:
            continue

    if best_model is None:
        # 如果所有模型都失败，使用简单的 AR(1)模型
        try:
            model = ARIMA(timeseries, order=(1,0,0))
            best_model = model.fit(dispatch=False)
            best_order = (1,0,0)
            best_seasonal_order = (0,0,0,0)
        except:
            pass

    return best_model, best_order, best_seasonal_order

if __name__ == "__main__":
    main()

```

GBRT 分层预测代码

```

import pandas as pd
import numpy as np
from datetime import datetime, timedelta, time as dt_time
import itertools
import time
import os

print("初始化调度算法处理...")
start_time = time.time()

# 数据源定义
source_routes = r"C:\Users\16878\Desktop\D 题\附件\附件 1.xlsx" # 路线信息数据
source_chains = r"C:\Users\16878\Desktop\D 题\附件\附件 4.xlsx" # 可串点站点信息
source_fleet = r"C:\Users\16878\Desktop\D 题\附件\附件 5.xlsx" # 车队容量信息
source_pred1 = r"C:\Users\16878\Desktop\D 题\结果表 1(1).xlsx" # 总需求预测结果
source_pred2 = r"C:\Users\16878\Desktop\D 题\结果表 2(1).xlsx" # 颗粒度预测结果
output_result = r"C:\Users\16878\Desktop\D 题\结果表\结果表 3.xlsx" # 输出文件路径

# 确保输出目录存在
output_dir = os.path.dirname(output_result)
if not os.path.exists(output_dir):
    os.makedirs(output_dir)
print(f'创建输出目录: {output_dir}')

```

1. 导入数据集

```
print("导入数据集...")
route_info = pd.read_excel(source_routes)
chain_points = pd.read_excel(source_chains)
fleet_capacity = pd.read_excel(source_fleet)
pred_totals = pd.read_excel(source_pred1)
pred_details = pd.read_excel(source_pred2)
```

提取站点编号的辅助函数

```
def extract_site_id(site_name):
    if isinstance(site_name, str) and site_name.startswith('站点'):
        try:
            return int(site_name[2:])
        except ValueError:
            return None
    return None
```

2. 数据预处理

```
print("执行数据预处理...")
```

显示串点数据信息

```
print("\n 串点数据集(前 5 行):")
print(chain_points.head(5))
```

检查预测结果信息

```
print("\n 预测结果信息:")
print(f'预测结果列名: {pred_details.columns.tolist()}')
```

时间处理辅助函数

```
def ensure_datetime_column(df, date_col='日期', time_col='分钟起始',
target_col='datetime'):
    """确保 DataFrame 包含 datetime 列，将日期和时间列合并为完整的时间戳"""
    if target_col not in df.columns:
        if date_col in df.columns and time_col in df.columns:
            try:
                # 确保列类型正确
                if not pd.api.types.is_datetime64_dtype(df[date_col]):
                    df[date_col] = pd.to_datetime(df[date_col])

                if not pd.api.types.is_datetime64_dtype(df[time_col]):
                    df[time_col] = pd.to_datetime(df[time_col])
```

```

        # 合并日期和时间
        date_str = df[date_col].dt.strftime('%Y-%m-%d')
        time_str = df[time_col].dt.strftime('%H:%M:%S')
        df[target_col] = pd.to_datetime(date_str + ' ' + time_str)
        return True
    except Exception as e:
        print(f"创建时间戳列异常: {str(e)}")
        print(f"日期列类型: {df[date_col].dtype}")
        print(f"时间列类型: {df[time_col].dtype}")
        return False
    else:
        print(f"错误: 缺少必要的日期列 '{date_col}' 或时间列 '{time_col}'")
        return False
    return True

# 站点兼容性检查函数
def is_compatible_destination(current_loc, destination, chaining_map, compatible_pairs):
    """检查两个站点是否兼容（可串点或相同）"""
    # 如果站点相同，直接返回兼容
    if current_loc == destination:
        return True

    # 如果任一为 None，不兼容
    if current_loc is None or destination is None:
        return False

    # 检查整数 ID 的兼容性
    if isinstance(current_loc, int) and isinstance(destination, int):
        return (current_loc, destination) in compatible_pairs

    # 检查字符串 ID 的兼容性(如"站点 1")
    elif isinstance(current_loc, str) and isinstance(destination, str):
        return (current_loc in chaining_map and destination in chaining_map[current_loc])

    # 不同类型的 ID 不兼容
    return False

# 创建时间戳列
ensure_datetime_column(pred_details)

# 显示时间范围

```

```

if 'datetime' in pred_details.columns:
    print(f" 数据 时 间 范 围  :   {pred_details['datetime'].min()}    至
{pred_details['datetime'].max()}")
    print(f"包含日期: {pred_details['日期'].unique()}")

print(f" 时 间 粒 度 示 例 :   {pred_details['分钟起始'].head(3) if '分钟起始' in
pred_details.columns else '分钟起始列不存在'}")
print(f"发运节点值: {sorted(route_info['发运节点'].unique())}")

# 构建串点映射字典
chain_map = {}
compatible_pairs = set() # 存储兼容站点对

for _, row in chain_points.iterrows():
    # 获取站点编号
    site1, site2 = row['站点编号 1'], row['站点编号 2']

    # 添加到映射字典
    if site1 not in chain_map:
        chain_map[site1] = set()
    if site2 not in chain_map:
        chain_map[site2] = set()

    chain_map[site1].add(site2)
    chain_map[site2].add(site1)

    # 添加到兼容对集合
    compatible_pairs.add((site1, site2))
    compatible_pairs.add((site2, site1))

    # 同时添加数字编号的兼容对
    int_site1 = extract_site_id(site1)
    int_site2 = extract_site_id(site2)
    if int_site1 is not None and int_site2 is not None:
        compatible_pairs.add((int_site1, int_site2))
        compatible_pairs.add((int_site2, int_site1))

print(f"串点站点总对数: {len(chain_points)}")
print(f"可串点站点数: {len(chain_map)}")
print(f"串点站点示例: {list(chain_map.items())[3]}")
print(f" 数 字 编 号 兼 容 对 示 例 :   {list(filter(lambda x: isinstance(x[0], int),
list(compatible_pairs)[:10]))}")

```

```

# 创建车队配置字典
fleet_dict = dict(zip(fleet_capacity['车队编码'], fleet_capacity['自有车数量']))

# 转换发运节点为时间格式
try:
    # 尝试转换为时间格式
    route_info['发运节点'] = pd.to_datetime(route_info['发运节点']).dt.time
except:
    # 如果已经是 time 格式，则忽略错误
    pass

# 转换预测数据时间
pred_details['分钟起始'] = pd.to_datetime(pred_details['分钟起始'])
pred_details['datetime'] = pd.to_datetime(pred_details['日期'].astype(str) + ' ' + pred_details['分钟起始']).dt.strftime('%H:%M:%S'))

# 系统参数设置
MAX_LOAD = 1000 # 车辆最大装载量
LOADING_TIME = 45/60 # 装载时间（小时）
UNLOADING_TIME = 45/60 # 卸载时间（小时）
DAILY_FIXED_COST = 100 # 自有车日固定成本

# 3. 生成基本运输需求
print("生成基本运输需求...")

def generate_transport_requirements(pred_details):
    transport_requirements = []

    # 确保有 datetime 列
    if not ensure_datetime_column(pred_details):
        return pd.DataFrame()

    # 打印日期和分钟起始的信息，确认数据范围
    unique_dates = pred_details['日期'].unique()
    min_datetime = pred_details['datetime'].min()
    max_datetime = pred_details['datetime'].max()
    print(f'结果表 2 包含的日期: {unique_dates}')
    print(f'结果表 2 时间范围: {min_datetime} 到 {max_datetime}')

    # 按线路和日期分组
    for (route_code, date), group in pred_details.groupby(['线路编码', '日期']):

```

```

# 获取路线信息并计算截止时间
route_data = route_info[route_info['线路编码'] == route_code].iloc[0]
deadline_time = route_data['发运节点']
deadline_datetime = datetime.combine(date, dt_time(deadline_time.hour,
deadline_time.minute))

# 获取目的场地的站点编号
site_name = route_data['目的场地']
site_id = extract_site_id(site_name)
destination = site_id if site_id is not None else site_name

# 按时间顺序排序并计算累积包裹量
group = group.sort_values('datetime')
accumulated_volume = 0

for _, row in group.iterrows():
    accumulated_volume += row['预测包裹量']

# 创建运输需求，如果累积量超过最大装载量则分拆
while accumulated_volume > 0:
    current_load = min(accumulated_volume, MAX_LOAD)
    accumulated_volume -= current_load

    transport_requirements.append({
        'route_code': route_code,
        'date': date,
        'fleet_code': route_data['车队编码'],
        'destination': destination,
        'deadline': deadline_datetime,
        'volume': current_load,
        'latest_time': row['datetime']
    })

# 转换为 DataFrame 并确保日期格式正确
if transport_requirements:
    transport_df = pd.DataFrame(transport_requirements)
    if not pd.api.types.is_datetime64_dtype(transport_df['date']):
        transport_df['date'] = pd.to_datetime(transport_df['date'])
    return transport_df
else:
    print("警告：未生成任何运输需求")
    return pd.DataFrame()

```

4. 调度算法

```
def optimize_dispatch_with_chaining(transport_reqs, chaining_map, fleet_capacity):
```

```
    """
```

```
    执行包含串点优化的调度算法
```

```
    参数:
```

```
    transport_reqs: 运输需求 DataFrame
```

```
    chaining_map: 串点站点映射字典
```

```
    fleet_capacity: 车队自有车容量字典
```

```
    返回:
```

```
    调度结果 DataFrame
```

```
    """
```

```
    print("开始串点优化调度...")
```

```
    results = []
```

```
    # 按截止时间排序并按日期和车队分组处理
```

```
    transport_reqs = transport_reqs.sort_values('deadline')
```

```
    for (date, fleet_code), group in transport_reqs.groupby(['date', 'fleet_code']):
```

```
        print(f"\n 处理日期: {date.date()}, 车队: {fleet_code}")
```

```
        # 获取车队可用车辆并初始化
```

```
        available_vehicles = fleet_capacity.get(fleet_code, 0)
```

```
        vehicles = [
```

```
            {
```

```
                'id': f"{fleet_code}_{i+1}",
```

```
                'type': '自有',
```

```
                'loads': [],
```

```
                'ready_time': None,
```

```
                'current_loc': None
```

```
            }
```

```
            for i in range(available_vehicles)
```

```
        ]
```

```
        # 按截止时间和最晚时间排序需求
```

```
        group = group.sort_values(['deadline', 'latest_time'])
```

```
        external_vehicle_count = 0
```

```
        # 为每个需求分配车辆
```

```
        for idx, req in group.iterrows():
```

```

destination = req['destination']
volume = req['volume']
deadline = req['deadline']
route_code = req['route_code']
assigned = False

# 1. 尝试分配给已有车辆 - 优先考虑可以串点的情况
for vehicle in vehicles:
    # 检查车辆是否可用且时间兼容
    if (vehicle['ready_time'] is None or
        vehicle['ready_time'] <= deadline -
timedelta(hours=LOADING_TIME)):

        # 检查串点兼容性
        if vehicle['current_loc'] is not None and not
is_compatible_destination(
            vehicle['current_loc'], destination, chaining_map,
compatible_pairs):
            continue

        # 计算装载和卸载时间
        ready_time = vehicle['ready_time'] if vehicle['ready_time'] is not
None else deadline - timedelta(hours=LOADING_TIME)
        loading_end_time = ready_time +
timedelta(hours=LOADING_TIME)
        unloading_end_time = loading_end_time +
timedelta(hours=UNLOADING_TIME)

        # 分配给此车辆并更新状态
        vehicle['loads'].append({
            'route_code': route_code,
            'destination': destination,
            'volume': volume,
            'loading_start': ready_time,
            'loading_end': loading_end_time,
            'deadline': deadline,
            'unloading_end': unloading_end_time
        })

        vehicle['ready_time'] = unloading_end_time
        vehicle['current_loc'] = destination
        assigned = True

```

```

        break

# 2. 如果未分配且还有可用车辆，分配新车辆
if not assigned:
    unused_vehicles = [v for v in vehicles if not v['loads']]
    if unused_vehicles:
        vehicle = unused_vehicles[0]
        # 计算时间
        loading_start = deadline - timedelta(hours=LOADING_TIME)
        loading_end = deadline
        unloading_end = loading_end +
timedelta(hours=UNLOADING_TIME)

        # 分配给此车辆
        vehicle['loads'].append({
            'route_code': route_code,
            'destination': destination,
            'volume': volume,
            'loading_start': loading_start,
            'loading_end': loading_end,
            'deadline': deadline,
            'unloading_end': unloading_end
        })

        vehicle['ready_time'] = unloading_end
        vehicle['current_loc'] = destination
        assigned = True
# 3. 如果仍未分配，使用外部车辆
else:
    external_vehicle_count += 1
    external_id = f"外部_{date.date()}_{external_vehicle_count}"

    # 计算时间
    loading_start = deadline - timedelta(hours=LOADING_TIME)
    loading_end = deadline
    unloading_end = loading_end +
timedelta(hours=UNLOADING_TIME)

    # 添加外部车辆运输任务
    results.append({
        '日期': date,
        '车队编码': fleet_code,

```

```

        '车辆 ID': external_id,
        '车辆类型': '外部',
        '线路编码': route_code,
        '目的地': destination,
        '包裹量': volume,
        '装车开始时间': loading_start,
        '装车结束时间': loading_end,
        '发车时间': deadline,
        '卸车结束时间': unloading_end
    })

# 处理完该分组后，将自有车辆的结果添加到结果集
for vehicle in vehicles:
    if vehicle['loads']: # 只处理有分配任务的车辆
        for load in vehicle['loads']:
            results.append({
                '日期': date,
                '车队编码': fleet_code,
                '车辆 ID': vehicle['id'],
                '车辆类型': '自有',
                '线路编码': load['route_code'],
                '目的地': load['destination'],
                '包裹量': load['volume'],
                '装车开始时间': load['loading_start'],
                '装车结束时间': load['loading_end'],
                '发车时间': load['deadline'],
                '卸车结束时间': load['unloading_end']
            })

# 转换为 DataFrame
return pd.DataFrame(results) if results else pd.DataFrame()

# 生成基本运输需求
transport_requirements = generate_transport_requirements(pred_details)
print(f'生成了 {len(transport_requirements)} 个基本运输需求')

# 按日期和车队分组，打印需求数量
print("\n 各日期和车队的需求数量:")
req_counts = transport_requirements.groupby(['date', 'fleet_code']).size()
print(req_counts)

# 执行调度算法

```

```

print("\n 执行调度算法...")
final_schedule = optimize_dispatch_with_chaining(transport_requirements, chain_map,
fleet_dict)

# 计算指标
print("计算调度指标...")

def calculate_performance_metrics(schedule):
    """
    计算调度方案的性能指标，更高效的实现

    参数:
    schedule: 调度结果 DataFrame

    返回:
    metrics_dict: 包含各项指标的字典
    """
    if schedule.empty:
        print("警告: 调度结果为空，无法计算指标")
        return {}

    metrics = {}

    # 使用 groupby 和 value_counts 合并统计车辆信息
    vehicle_stats = schedule.groupby(['车辆类型', '车辆 ID']).size().reset_index()
    vehicle_counts = vehicle_stats['车辆类型'].value_counts()

    # 计算基础指标
    self_owned_vehicles = vehicle_counts.get('自有', 0)
    external_vehicles = vehicle_counts.get('外部', 0)
    total_vehicles = self_owned_vehicles + external_vehicles

    metrics.update({
        '自有车辆数': self_owned_vehicles,
        '外部车辆数': external_vehicles,
        '总车辆数': total_vehicles,
        '外部车辆占比': round(external_vehicles / total_vehicles * 100, 2) if
total_vehicles > 0 else 0,
        '运输总量': schedule['包裹量'].sum(),
    })

    # 计算平均装载率

```

```
metrics['平均装载率'] = round(metrics['运输总量'] / (total_vehicles * MAX_LOAD) *
100, 2) if total_vehicles > 0 else 0
```

```
# 使用多级分组和聚合函数计算统计指标
combined_stats = schedule.pivot_table(
    index=['车队编码', '日期'],
    values=['包裹量'],
    aggfunc={'包裹量': 'sum'},
    margins=False
).reset_index()

# 计算车辆数和外部车辆数
vehicle_counts_by_group = schedule.groupby(['车队编码', '日期', '车辆类型'])['车辆
ID'].nunique().unstack(fill_value=0)
if '自有' not in vehicle_counts_by_group.columns:
    vehicle_counts_by_group['自有'] = 0
if '外部' not in vehicle_counts_by_group.columns:
    vehicle_counts_by_group['外部'] = 0

vehicle_counts_by_group['车辆数'] = vehicle_counts_by_group['自有'] +
vehicle_counts_by_group['外部']
vehicle_counts_by_group['外部车辆占比'] = round(
    vehicle_counts_by_group['外部'] / vehicle_counts_by_group['车辆数'] * 100, 2
).fillna(0)

# 合并统计数据
combined_stats = combined_stats.merge(
    vehicle_counts_by_group.reset_index(),
    on=['车队编码', '日期']
)

# 计算每个分组的装载率
combined_stats['装载率'] = round(
    combined_stats['包裹量'] / (combined_stats['车辆数'] * MAX_LOAD) * 100, 2
)

# 按车队和日期汇总统计
fleet_stats = combined_stats.groupby('车队编码').agg({
    '车辆数': 'sum',
    '自有': 'sum',
    '外部': 'sum',
    '包裹量': 'sum',
```

```

        '装载率': 'mean',
        '外部车辆占比': 'mean'
    })

date_stats = combined_stats.groupby('日期').agg({
    '车辆数': 'sum',
    '自有': 'sum',
    '外部': 'sum',
    '包裹量': 'sum',
    '装载率': 'mean',
    '外部车辆占比': 'mean'
})

# 串点相关指标
chaining_stats = {}
if '串点标记' in schedule.columns:
    chained_vehicles = schedule[schedule['串点标记']].groupby('车辆 ID').size()
    chaining_stats['串点车次数'] = len(chained_vehicles)
    chaining_stats['串点车次占比'] = round(len(chained_vehicles) / total_vehicles *
100, 2) if total_vehicles > 0 else 0

    if '串点组合' in schedule.columns:
        chained_combinations = schedule[schedule['串点标记']].drop_duplicates(['车
辆 ID', '串点组合'])
        chaining_stats['串点组合分布'] = chained_combinations['串点组合
'].value_counts().to_dict()

# 存储统计结果
metrics['按车队统计'] = fleet_stats.to_dict()
metrics['按日期统计'] = date_stats.to_dict()
metrics['串点统计'] = chaining_stats

# 打印主要指标
print(f"\n 调度结果总结:")
print(f" 总 车 辆 数 : {total_vehicles} ( 自 有 : {self_owned_vehicles}, 外 部 :
{external_vehicles})")
print(f"外部车辆占比: {metrics['外部车辆占比']}%")
print(f"运输总量: {metrics['运输总量']}")
print(f"平均装载率: {metrics['平均装载率']}%")

if '串点标记' in schedule.columns:
    print(f"串点车次数: {chaining_stats['串点车次数']}")

```

```

        print(f'串点车次占比: {chaining_stats['串点车次占比']}%')

    return metrics

def format_schedule_output(schedule):
    """
    格式化调度结果用于输出，更高效的实现

    参数:
    schedule: 调度结果 DataFrame

    返回:
    formatted_schedule: 格式化后的 DataFrame
    """
    if schedule.empty:
        return pd.DataFrame()

    # 复制 DataFrame 以避免修改原始数据
    output = schedule.copy()

    # 添加串点标记(如果没有)
    if '串点标记' not in output.columns:
        # 计算每个车辆的目的地数量并标记串点情况
        dest_counts = output.groupby(['日期', '车辆 ID'])['目的地'].agg(['nunique',
list]).reset_index()
        dest_counts.columns = ['日期', '车辆 ID', '目的地数量', '目的地列表']

        # 创建串点标记和串点组合
        dest_counts['串点标记'] = dest_counts['目的地数量'] > 1
        dest_counts['串点组合'] = dest_counts.apply(
            lambda x: '-'.join(map(str, sorted(x['目的地列表']))) if x['串点标记'] else
None,
            axis=1
        )

        # 只保留需要的列并合并回主 DataFrame
        dest_counts = dest_counts[['日期', '车辆 ID', '串点标记', '串点组合']]
        output = output.merge(dest_counts, on=['日期', '车辆 ID'], how='left')

    # 确保时间列格式正确
    time_columns = ['装车开始时间', '装车结束时间', '发车时间', '卸车结束时间']
    for col in time_columns:

```

```

        if col in output.columns and not pd.api.types.is_datetime64_dtype(output[col]):
            output[col] = pd.to_datetime(output[col])

    # 按日期、车队、车辆 ID 和发车时间排序
    return output.sort_values(['日期', '车队编码', '车辆 ID', '发车时间']).reset_index(drop=True)

# 计算调度指标
metrics = calculate_performance_metrics(final_schedule)

# 格式化输出结果
formatted_schedule = format_schedule_output(final_schedule)

# 保存结果
print(f"\n 保存调度结果到: {output_result}")
formatted_schedule.to_csv(output_result, index=False, encoding='utf-8-sig')

print(f" 调度算法执行完成。共调度 {len(formatted_schedule)} 个运输任务，使用 {metrics['总车辆数']} 辆车。")
print(f" 结果已保存到: {output_result}")

```

贪心-MG 算法代码

```

import pandas as pd
import numpy as np
from datetime import datetime, timedelta, time as dt_time
import itertools
import time
import os

print("初始化调度算法处理...")
start_time = time.time()

# 数据源定义
source_routes = r"C:\Users\16878\Desktop\D 题\附件\附件 1.xlsx" # 路线信息数据
source_chains = r"C:\Users\16878\Desktop\D 题\附件\附件 4.xlsx" # 可串点站点信息
source_fleet = r"C:\Users\16878\Desktop\D 题\附件\附件 5.xlsx" # 车队容量信息
source_pred1 = r"C:\Users\16878\Desktop\D 题\结果表 1(1).xlsx" # 总需求预测结果
source_pred2 = r"C:\Users\16878\Desktop\D 题\结果表 2(1).xlsx" # 颗粒度预测结果
output_result = r"C:\Users\16878\Desktop\D 题\结果表\结果表 3.xlsx" # 输出文件路径

# 确保输出目录存在
output_dir = os.path.dirname(output_result)

```

```

if not os.path.exists(output_dir):
    os.makedirs(output_dir)
    print(f'创建输出目录: {output_dir}')

# 1. 导入数据集
print("导入数据集...")
route_info = pd.read_excel(source_routes)
chain_points = pd.read_excel(source_chains)
fleet_capacity = pd.read_excel(source_fleet)
pred_totals = pd.read_excel(source_pred1)
pred_details = pd.read_excel(source_pred2)

# 提取站点编号的辅助函数
def extract_site_id(site_name):
    if isinstance(site_name, str) and site_name.startswith('站点'):
        try:
            return int(site_name[2:])
        except ValueError:
            return None
    return None

# 2. 数据预处理
print("执行数据预处理...")

# 显示串点数据信息
print("\n 串点数据集(前 5 行):")
print(chain_points.head(5))

# 检查预测结果信息
print("\n 预测结果信息:")
print(f'预测结果列名: {pred_details.columns.tolist()}')

# 时间处理辅助函数
def ensure_datetime_column(df, date_col='日期', time_col='分钟起始',
target_col='datetime'):
    """确保 DataFrame 包含 datetime 列，将日期和时间列合并为完整的时间戳"""
    if target_col not in df.columns:
        if date_col in df.columns and time_col in df.columns:
            try:
                # 确保列类型正确
                if not pd.api.types.is_datetime64_dtype(df[date_col]):
                    df[date_col] = pd.to_datetime(df[date_col])

```

```

        if not pd.api.types.is_datetime64_dtype(df[time_col]):
            df[time_col] = pd.to_datetime(df[time_col])

        # 合并日期和时间
        date_str = df[date_col].dt.strftime('%Y-%m-%d')
        time_str = df[time_col].dt.strftime('%H:%M:%S')
        df[target_col] = pd.to_datetime(date_str + ' ' + time_str)
        return True
    except Exception as e:
        print(f"创建时间戳列异常: {str(e)}")
        print(f"日期列类型: {df[date_col].dtype}")
        print(f"时间列类型: {df[time_col].dtype}")
        return False
    else:
        print(f"错误: 缺少必要的日期列 '{date_col}' 或时间列 '{time_col}'")
        return False
return True

# 站点兼容性检查函数
def is_compatible_destination(current_loc, destination, chaining_map, compatible_pairs):
    """检查两个站点是否兼容（可串点或相同）"""
    # 如果站点相同，直接返回兼容
    if current_loc == destination:
        return True

    # 如果任一为 None，不兼容
    if current_loc is None or destination is None:
        return False

    # 检查整数 ID 的兼容性
    if isinstance(current_loc, int) and isinstance(destination, int):
        return (current_loc, destination) in compatible_pairs

    # 检查字符串 ID 的兼容性(如"站点 1")
    elif isinstance(current_loc, str) and isinstance(destination, str):
        return (current_loc in chaining_map and destination in chaining_map[current_loc])

    # 不同类型的 ID 不兼容
    return False

# 创建时间戳列

```

```

ensure_datetime_column(pred_details)

# 显示时间范围
if 'datetime' in pred_details.columns:
    print(f" 数 据 时 间 范 围 : {pred_details['datetime'].min()} 至 {pred_details['datetime'].max()}")
    print(f"包含日期: {pred_details['日期'].unique()}")

print(f" 时 间 粒 度 示 例 : {pred_details['分钟起始'].head(3)} if '分钟起始' in pred_details.columns else '分钟起始列不存在'")
print(f"发运节点值: {sorted(route_info['发运节点'].unique())}")

# 构建串点映射字典
chain_map = {}
compatible_pairs = set() # 存储兼容站点对

for _, row in chain_points.iterrows():
    # 获取站点编号
    site1, site2 = row['站点编号 1'], row['站点编号 2']

    # 添加到映射字典
    if site1 not in chain_map:
        chain_map[site1] = set()
    if site2 not in chain_map:
        chain_map[site2] = set()

    chain_map[site1].add(site2)
    chain_map[site2].add(site1)

    # 添加到兼容对集合
    compatible_pairs.add((site1, site2))
    compatible_pairs.add((site2, site1))

    # 同时添加数字编号的兼容对
    int_site1 = extract_site_id(site1)
    int_site2 = extract_site_id(site2)
    if int_site1 is not None and int_site2 is not None:
        compatible_pairs.add((int_site1, int_site2))
        compatible_pairs.add((int_site2, int_site1))

print(f"串点站点总对数: {len(chain_points)}")
print(f"可串点站点数: {len(chain_map)}")

```

```

print(f'串点站点示例: {list(chain_map.items())[:3]}')
print(f' 数字 编号 兼容 对 示例 : {list(filter(lambda x: isinstance(x[0], int),
list(compatible_pairs)[:10]))}')

# 创建车队配置字典
fleet_dict = dict(zip(fleet_capacity['车队编码'], fleet_capacity['自有车数量']))

# 转换发运节点为时间格式
try:
    # 尝试转换为时间格式
    route_info['发运节点'] = pd.to_datetime(route_info['发运节点']).dt.time
except:
    # 如果已经是 time 格式, 则忽略错误
    pass

# 转换预测数据时间
pred_details['分钟起始'] = pd.to_datetime(pred_details['分钟起始'])
pred_details['datetime'] = pd.to_datetime(pred_details['日期'].astype(str) + ' ' + pred_details['
分钟起始'].dt.strftime('%H:%M:%S'))

# 系统参数设置
MAX_LOAD = 1000 # 车辆最大装载量
LOADING_TIME = 45/60 # 装载时间 (小时)
UNLOADING_TIME = 45/60 # 卸载时间 (小时)
DAILY_FIXED_COST = 100 # 自有车日固定成本

# 3. 生成基本运输需求
print("生成基本运输需求...")

def generate_transport_requirements(pred_details):
    transport_requirements = []

    # 确保有 datetime 列
    if not ensure_datetime_column(pred_details):
        return pd.DataFrame()

    # 打印日期和分钟起始的信息, 确认数据范围
    unique_dates = pred_details['日期'].unique()
    min_datetime = pred_details['datetime'].min()
    max_datetime = pred_details['datetime'].max()
    print(f'结果表 2 包含的日期: {unique_dates}')
    print(f'结果表 2 时间范围: {min_datetime} 到 {max_datetime}')

```

```
# 按线路和日期分组
for (route_code, date), group in pred_details.groupby(['线路编码', '日期']):
    # 获取路线信息并计算截止时间
    route_data = route_info[route_info['线路编码'] == route_code].iloc[0]
    deadline_time = route_data['发运节点']
    deadline_datetime = datetime.combine(date, dt_time(deadline_time.hour,
deadline_time.minute))

    # 获取目的场地的站点编号
    site_name = route_data['目的场地']
    site_id = extract_site_id(site_name)
    destination = site_id if site_id is not None else site_name

    # 按时间顺序排序并计算累积包裹量
    group = group.sort_values('datetime')
    accumulated_volume = 0

    for _, row in group.iterrows():
        accumulated_volume += row['预测包裹量']

        # 创建运输需求，如果累积量超过最大装载量则分拆
        while accumulated_volume > 0:
            current_load = min(accumulated_volume, MAX_LOAD)
            accumulated_volume -= current_load

            transport_requirements.append({
                'route_code': route_code,
                'date': date,
                'fleet_code': route_data['车队编码'],
                'destination': destination,
                'deadline': deadline_datetime,
                'volume': current_load,
                'latest_time': row['datetime']
            })

    # 转换为 DataFrame 并确保日期格式正确
    if transport_requirements:
        transport_df = pd.DataFrame(transport_requirements)
        if not pd.api.types.is_datetime64_dtype(transport_df['date']):
            transport_df['date'] = pd.to_datetime(transport_df['date'])
        return transport_df
```

```
else:
    print("警告: 未生成任何运输需求")
    return pd.DataFrame()
```

4. 调度算法

```
def optimize_dispatch_with_chaining(transport_reqs, chaining_map, fleet_capacity):
```

```
    """
```

```
    执行包含串点优化的调度算法
```

```
    参数:
```

```
    transport_reqs: 运输需求 DataFrame
```

```
    chaining_map: 串点站点映射字典
```

```
    fleet_capacity: 车队自有车容量字典
```

```
    返回:
```

```
    调度结果 DataFrame
```

```
    """
```

```
    print("开始串点优化调度...")
```

```
    results = []
```

```
    # 按截止时间排序并按日期和车队分组处理
```

```
    transport_reqs = transport_reqs.sort_values('deadline')
```

```
    for (date, fleet_code), group in transport_reqs.groupby(['date', 'fleet_code']):
```

```
        print(f"\n 处理日期: {date.date()}, 车队: {fleet_code}")
```

```
        # 获取车队可用车辆并初始化
```

```
        available_vehicles = fleet_capacity.get(fleet_code, 0)
```

```
        vehicles = [
```

```
            {
```

```
                'id': f"{fleet_code}_{i+1}",
```

```
                'type': '自有',
```

```
                'loads': [],
```

```
                'ready_time': None,
```

```
                'current_loc': None
```

```
            }
```

```
            for i in range(available_vehicles)
```

```
        ]
```

```
        # 按截止时间和最晚时间排序需求
```

```
        group = group.sort_values(['deadline', 'latest_time'])
```

```
        external_vehicle_count = 0
```

```

# 为每个需求分配车辆
for idx, req in group.iterrows():
    destination = req['destination']
    volume = req['volume']
    deadline = req['deadline']
    route_code = req['route_code']
    assigned = False

    # 1. 尝试分配给已有车辆 - 优先考虑可以串点的情况
    for vehicle in vehicles:
        # 检查车辆是否可用且时间兼容
        if (vehicle['ready_time'] is None or
            vehicle['ready_time'] + timedelta(hours=LOADING_TIME) <= deadline -
            timedelta(hours=LOADING_TIME)):

            # 检查串点兼容性
            if vehicle['current_loc'] is not None and not
            is_compatible_destination(
                vehicle['current_loc'], destination, chaining_map,
                compatible_pairs):
                continue

            # 计算装载和卸载时间
            ready_time = vehicle['ready_time'] if vehicle['ready_time'] is not
            None else deadline - timedelta(hours=LOADING_TIME)
            loading_end_time = ready_time +
            timedelta(hours=LOADING_TIME)
            unloading_end_time = loading_end_time +
            timedelta(hours=UNLOADING_TIME)

            # 分配给此车辆并更新状态
            vehicle['loads'].append({
                'route_code': route_code,
                'destination': destination,
                'volume': volume,
                'loading_start': ready_time,
                'loading_end': loading_end_time,
                'deadline': deadline,
                'unloading_end': unloading_end_time
            })

```

```

        vehicle['ready_time'] = unloading_end_time
        vehicle['current_loc'] = destination
        assigned = True
        break

# 2. 如果未分配且还有可用车辆，分配新车辆
if not assigned:
    unused_vehicles = [v for v in vehicles if not v['loads']]
    if unused_vehicles:
        vehicle = unused_vehicles[0]
        # 计算时间
        loading_start = deadline - timedelta(hours=LOADING_TIME)
        loading_end = deadline
        unloading_end = loading_end +
timedelta(hours=UNLOADING_TIME)

        # 分配给此车辆
        vehicle['loads'].append({
            'route_code': route_code,
            'destination': destination,
            'volume': volume,
            'loading_start': loading_start,
            'loading_end': loading_end,
            'deadline': deadline,
            'unloading_end': unloading_end
        })

        vehicle['ready_time'] = unloading_end
        vehicle['current_loc'] = destination
        assigned = True
# 3. 如果仍未分配，使用外部车辆
else:
    external_vehicle_count += 1
    external_id = f"外部_{date.date()}_{external_vehicle_count}"

    # 计算时间
    loading_start = deadline - timedelta(hours=LOADING_TIME)
    loading_end = deadline
    unloading_end = loading_end +
timedelta(hours=UNLOADING_TIME)

    # 添加外部车辆运输任务

```

```

        results.append({
            '日期': date,
            '车队编码': fleet_code,
            '车辆 ID': external_id,
            '车辆类型': '外部',
            '线路编码': route_code,
            '目的地': destination,
            '包裹量': volume,
            '装车开始时间': loading_start,
            '装车结束时间': loading_end,
            '发车时间': deadline,
            '卸车结束时间': unloading_end
        })

# 处理完该分组后，将自有车辆的结果添加到结果集
for vehicle in vehicles:
    if vehicle['loads']: # 只处理有分配任务的车辆
        for load in vehicle['loads']:
            results.append({
                '日期': date,
                '车队编码': fleet_code,
                '车辆 ID': vehicle['id'],
                '车辆类型': '自有',
                '线路编码': load['route_code'],
                '目的地': load['destination'],
                '包裹量': load['volume'],
                '装车开始时间': load['loading_start'],
                '装车结束时间': load['loading_end'],
                '发车时间': load['deadline'],
                '卸车结束时间': load['unloading_end']
            })

# 转换为 DataFrame
return pd.DataFrame(results) if results else pd.DataFrame()

# 生成基本运输需求
transport_requirements = generate_transport_requirements(pred_details)
print(f'生成了 {len(transport_requirements)} 个基本运输需求')

# 按日期和车队分组，打印需求数量
print("\n 各日期和车队的需求数量:")
req_counts = transport_requirements.groupby(['date', 'fleet_code']).size()

```

```

print(req_counts)

# 执行调度算法
print("\n 执行调度算法...")
final_schedule = optimize_dispatch_with_chaining(transport_requirements, chain_map,
fleet_dict)

# 计算指标
print("计算调度指标...")

def calculate_performance_metrics(schedule):
    """
    计算调度方案的性能指标，更高效的实现

    参数:
    schedule: 调度结果 DataFrame

    返回:
    metrics_dict: 包含各项指标的字典
    """
    if schedule.empty:
        print("警告: 调度结果为空，无法计算指标")
        return {}

    metrics = {}

    # 使用 groupby 和 value_counts 合并统计车辆信息
    vehicle_stats = schedule.groupby(['车辆类型', '车辆 ID']).size().reset_index()
    vehicle_counts = vehicle_stats['车辆类型'].value_counts()

    # 计算基础指标
    self_owned_vehicles = vehicle_counts.get('自有', 0)
    external_vehicles = vehicle_counts.get('外部', 0)
    total_vehicles = self_owned_vehicles + external_vehicles

    metrics.update({
        '自有车辆数': self_owned_vehicles,
        '外部车辆数': external_vehicles,
        '总车辆数': total_vehicles,
        '外部车辆占比': round(external_vehicles / total_vehicles * 100, 2) if
total_vehicles > 0 else 0,
        '运输总量': schedule['包裹量'].sum(),

```

```

    })

    # 计算平均装载率
    metrics['平均装载率'] = round(metrics['运输总量'] / (total_vehicles * MAX_LOAD) *
100, 2) if total_vehicles > 0 else 0

    # 使用多级分组和聚合函数计算统计指标
    combined_stats = schedule.pivot_table(
        index=['车队编码', '日期'],
        values=['包裹量'],
        aggfunc={'包裹量': 'sum'},
        margins=False
    ).reset_index()

    # 计算车辆数和外部车辆数
    vehicle_counts_by_group = schedule.groupby(['车队编码', '日期', '车辆类型'])['车辆
ID'].nunique().unstack(fill_value=0)
    if '自有' not in vehicle_counts_by_group.columns:
        vehicle_counts_by_group['自有'] = 0
    if '外部' not in vehicle_counts_by_group.columns:
        vehicle_counts_by_group['外部'] = 0

    vehicle_counts_by_group['车辆数'] = vehicle_counts_by_group['自有'] +
vehicle_counts_by_group['外部']
    vehicle_counts_by_group['外部车辆占比'] = round(
        vehicle_counts_by_group['外部'] / vehicle_counts_by_group['车辆数'] * 100, 2
    ).fillna(0)

    # 合并统计数据
    combined_stats = combined_stats.merge(
        vehicle_counts_by_group.reset_index(),
        on=['车队编码', '日期']
    )

    # 计算每个分组的装载率
    combined_stats['装载率'] = round(
        combined_stats['包裹量'] / (combined_stats['车辆数'] * MAX_LOAD) * 100, 2
    )

    # 按车队和日期汇总统计
    fleet_stats = combined_stats.groupby('车队编码').agg({
        '车辆数': 'sum',

```

```

        '自有': 'sum',
        '外部': 'sum',
        '包裹量': 'sum',
        '装载率': 'mean',
        '外部车辆占比': 'mean'
    })

    date_stats = combined_stats.groupby('日期').agg({
        '车辆数': 'sum',
        '自有': 'sum',
        '外部': 'sum',
        '包裹量': 'sum',
        '装载率': 'mean',
        '外部车辆占比': 'mean'
    })

    # 串点相关指标
    chaining_stats = {}
    if '串点标记' in schedule.columns:
        chained_vehicles = schedule[schedule['串点标记']].groupby('车辆 ID').size()
        chaining_stats['串点车次数'] = len(chained_vehicles)
        chaining_stats['串点车次占比'] = round(len(chained_vehicles) / total_vehicles *
100, 2) if total_vehicles > 0 else 0

        if '串点组合' in schedule.columns:
            chained_combinations = schedule[schedule['串点标记']].drop_duplicates(['车
辆 ID', '串点组合'])
            chaining_stats['串点组合分布'] = chained_combinations['串点组合
'].value_counts().to_dict()

    # 存储统计结果
    metrics['按车队统计'] = fleet_stats.to_dict()
    metrics['按日期统计'] = date_stats.to_dict()
    metrics['串点统计'] = chaining_stats

    # 打印主要指标
    print(f"\n 调度结果总结:")
    print(f" 总 车 辆 数 : {total_vehicles} ( 自 有 : {self_owned_vehicles}, 外 部 :
{external_vehicles})")
    print(f"外部车辆占比: {metrics['外部车辆占比']}%")
    print(f"运输总量: {metrics['运输总量']}")
    print(f"平均装载率: {metrics['平均装载率']}%")

```

```

if '串点标记' in schedule.columns:
    print(f'串点车次数: {chaining_stats['串点车次数']}')
    print(f'串点车次占比: {chaining_stats['串点车次占比']}%')

return metrics

def format_schedule_output(schedule):
    """
    格式化调度结果用于输出，更高效的实现

    参数:
    schedule: 调度结果 DataFrame

    返回:
    formatted_schedule: 格式化后的 DataFrame
    """
    if schedule.empty:
        return pd.DataFrame()

    # 复制 DataFrame 以避免修改原始数据
    output = schedule.copy()

    # 添加串点标记(如果没有)
    if '串点标记' not in output.columns:
        # 计算每个车辆的目的地数量并标记串点情况
        dest_counts = output.groupby(['日期', '车辆 ID'])['目的地'].agg(['nunique',
list]).reset_index()
        dest_counts.columns = ['日期', '车辆 ID', '目的地数量', '目的地列表']

        # 创建串点标记和串点组合
        dest_counts['串点标记'] = dest_counts['目的地数量'] > 1
        dest_counts['串点组合'] = dest_counts.apply(
            lambda x: '-'.join(map(str, sorted(x['目的地列表'])))) if x['串点标记'] else
None,
            axis=1
        )

        # 只保留需要的列并合并回主 DataFrame
        dest_counts = dest_counts[['日期', '车辆 ID', '串点标记', '串点组合']]
        output = output.merge(dest_counts, on=['日期', '车辆 ID'], how='left')

```

```

# 确保时间列格式正确
time_columns = ['装车开始时间', '装车结束时间', '发车时间', '卸车结束时间']
for col in time_columns:
    if col in output.columns and not pd.api.types.is_datetime64_dtype(output[col]):
        output[col] = pd.to_datetime(output[col])

# 按日期、车队、车辆 ID 和发车时间排序
return output.sort_values(['日期', '车队编码', '车辆 ID', '发车时间']).reset_index(drop=True)

# 计算调度指标
metrics = calculate_performance_metrics(final_schedule)

# 格式化输出结果
formatted_schedule = format_schedule_output(final_schedule)

# 保存结果
print(f"\n 保存调度结果到: {output_result}")
formatted_schedule.to_csv(output_result, index=False, encoding='utf-8-sig')

print(f"调度算法执行完成。共调度 {len(formatted_schedule)} 个运输任务，使用 {metrics['总车辆数']} 辆车。")
print(f"结果已保存到: {output_result}")

```

标准容器下的贪心-MG 算法代码

```

import pandas as pd
import numpy as np
from datetime import datetime, timedelta, time as dt_time
import itertools
import time
import os

print("物流调度程序启动中...")
execution_start = time.time()

# 定义输入输出文件的位置
line_data_path = r"C:\Users\16878\Desktop\D 题\附件\附件 1.xlsx" # 线路信息
chaining_points_path = r"C:\Users\16878\Desktop\D 题\附件\附件 4.xlsx" # 可串点站点集合
vehicle_count_path = r"C:\Users\16878\Desktop\D 题\附件\附件 5.xlsx" # 车队自有车数量
forecast_total_path = r"C:\Users\16878\Desktop\D 题\结果表 1(1).xlsx" # 预测的总包裹

```

量

forecast_detail_path = r"C:\Users\16878\Desktop\D 题\结果表 2(1).xlsx" # 预测的 10 分钟
颗粒度包裹量

solution_path = r"C:\Users\16878\Desktop\D 题\结果表\结果表 4.xlsx" # 问题三输出结果

确保输出目录存在

output_folder = os.path.dirname(solution_path)

if not os.path.exists(output_folder):

 os.makedirs(output_folder)

 print(f'已创建输出目录: {output_folder}')

1. 导入数据文件

print("正在导入数据文件...")

line_data = pd.read_excel(line_data_path)

chaining_points = pd.read_excel(chaining_points_path)

vehicle_count = pd.read_excel(vehicle_count_path)

forecast_total = pd.read_excel(forecast_total_path)

forecast_detail = pd.read_excel(forecast_detail_path)

从站点名称提取编号的辅助函数

def parse_site_id(site_name):

 """从站点名称(如'站点 23')中提取编号 23"""

 if isinstance(site_name, str) and site_name.startswith('站点'):

 try:

 return int(site_name[2:])

 except ValueError:

 return None

 return None

2. 数据预处理阶段

print("正在预处理数据...")

检查串点站点数据

print("\n 串点站点集合数据示例:")

print(chaining_points.head(5))

查看预测数据格式

print("\n 预测数据信息:")

print(f'预测数据列名: {forecast_detail.columns.tolist()}')

处理日期时间格式

if 'datetime' not in forecast_detail.columns:

```

print("创建完整日期时间列...")
# 检查是否有必要的日期和时间列
if '日期' in forecast_detail.columns and '分钟起始' in forecast_detail.columns:
    try:
        # 转换日期列格式
        if not pd.api.types.is_datetime64_dtype(forecast_detail['日期']):
            forecast_detail['日期'] = pd.to_datetime(forecast_detail['日期'])

        # 转换时间列格式
        if not pd.api.types.is_datetime64_dtype(forecast_detail['分钟起始']):
            forecast_detail['分钟起始'] = pd.to_datetime(forecast_detail['分钟起始'])

        # 合并日期和时间为完整的 datetime
        date_str = forecast_detail['日期'].dt.strftime('%Y-%m-%d')
        time_str = forecast_detail['分钟起始'].dt.strftime('%H:%M:%S')
        forecast_detail['datetime'] = pd.to_datetime(date_str + ' ' + time_str)

        print("日期时间列创建完成")
    except Exception as e:
        print(f"创建日期时间列出错: {str(e)}")
        # 打印调试信息
        print(f"日期列类型: {forecast_detail['日期'].dtype}")
        print(f"时间列类型: {forecast_detail['分钟起始'].dtype}")
        print(f"日期样例: {forecast_detail['日期'].head(5)}")
        print(f"时间样例: {forecast_detail['分钟起始'].head(5)}")
    else:
        print("错误: 无法创建日期时间列, 缺少必要的日期或时间字段")

# 打印时间范围信息
if 'datetime' in forecast_detail.columns:
    print(f"数据时间范围: {forecast_detail['datetime'].min()} 至 {forecast_detail['datetime'].max()}")
    print(f"包含的日期: {forecast_detail['日期'].unique()}")

print(f"时间颗粒度样例: {forecast_detail['分钟起始'].head(3)} if '分钟起始' in forecast_detail.columns else '时间列不存在'")
print(f"发车时间唯一值: {sorted(line_data['发运节点'].unique())}")

# 构建串点映射关系
chain_map = {}
compatible_sites = set() # 存储可以串点的站点对

```

```
for _, row in chaining_points.iterrows():
    # 站点编号格式为"站点 XX"
    site1, site2 = row['站点编号 1'], row['站点编号 2']

    # 添加到映射字典
    if site1 not in chain_map:
        chain_map[site1] = set()
    if site2 not in chain_map:
        chain_map[site2] = set()

    chain_map[site1].add(site2)
    chain_map[site2].add(site1)

    # 添加到兼容站点集合
    compatible_sites.add((site1, site2))
    compatible_sites.add((site2, site1))

    # 也添加数字编号的站点对
    id1 = parse_site_id(site1)
    id2 = parse_site_id(site2)
    if id1 is not None and id2 is not None:
        compatible_sites.add((id1, id2))
        compatible_sites.add((id2, id1))

print(f'共有 {len(chaining_points)} 对可串点站点')
print(f'共有 {len(chain_map)} 个站点有串点关系')
print(f'串点站点示例: {list(chain_map.items())[:3]}')
print(f'数字编号兼容站点示例: {list(filter(lambda x: isinstance(x[0], int),
list(compatible_sites)[:10]))}')

# 构建车队规模映射
fleet_size_map = dict(zip(vehicle_count['车队编码'], vehicle_count['自有车数量']))

# 处理发运节点时间格式
try:
    # 转换为时间格式
    line_data['发运节点'] = pd.to_datetime(line_data['发运节点']).dt.time
except:
    # 已经是时间格式则忽略
    pass

# 转换颗粒度预测数据的时间格式
```

```

forecast_detail['分钟起始'] = pd.to_datetime(forecast_detail['分钟起始'])
forecast_detail['datetime'] = pd.to_datetime(forecast_detail['日期']).astype(str) + ' ' +
forecast_detail['分钟起始'].dt.strftime('%H:%M:%S'))

# 定义车辆相关常量
# 普通车辆参数
STD_VEHICLE_CAPACITY = 1000 # 普通车辆最大装载量
STD_LOADING_HOURS = 45/60 # 普通车辆装车时间（小时）
STD_UNLOADING_HOURS = 45/60 # 普通车辆卸车时间（小时）

# 使用标准容器的参数
CONTAINER_CAPACITY = 800 # 标准容器最大装载量
CONTAINER_LOADING_HOURS = 10/60 # 使用容器的装车时间（小时）
CONTAINER_UNLOADING_HOURS = 10/60 # 使用容器的卸车时间（小时）

# 成本参数
VEHICLE_DAILY_COST = 100 # 自有车日固定成本

# 兼容性保持（确保旧代码能继续使用）
MAX_LOAD = STD_VEHICLE_CAPACITY
LOADING_TIME = STD_LOADING_HOURS
UNLOADING_TIME = STD_UNLOADING_HOURS

# 3. 生成运输需求
print("正在生成运输需求...")

def generate_shipment_demands(forecast_data):
    """根据预测数据生成运输需求"""
    shipment_demands = []

    # 确保预测数据有完整的日期时间列
    if 'datetime' not in forecast_data.columns:
        print("创建完整日期时间列...")
        # 检查是否有必要的日期和时间列
        if '日期' in forecast_data.columns and '分钟起始' in forecast_data.columns:
            try:
                # 转换日期列格式
                if not pd.api.types.is_datetime64_dtype(forecast_data['日期']):
                    forecast_data['日期'] = pd.to_datetime(forecast_data['日期'])

                # 转换时间列格式
                if not pd.api.types.is_datetime64_dtype(forecast_data['分钟起始']):

```

```

forecast_data['分钟起始'] = pd.to_datetime(forecast_data['分钟起始
'])

    # 合并日期和时间
    date_str = forecast_data['日期'].dt.strftime('%Y-%m-%d')
    time_str = forecast_data['分钟起始'].dt.strftime('%H:%M:%S')
    forecast_data['datetime'] = pd.to_datetime(date_str + ' ' + time_str)

    print("日期时间列创建完成")
except Exception as e:
    print(f"创建日期时间列出错: {str(e)}")
    # 打印调试信息
    print(f"日期列类型: {forecast_data['日期'].dtype}")
    print(f"时间列类型: {forecast_data['分钟起始'].dtype}")
    print(f"日期样例: {forecast_data['日期'].head(5)}")
    print(f"时间样例: {forecast_data['分钟起始'].head(5)}")
else:
    print("错误: 无法创建日期时间列, 缺少必要的日期或时间字段")
    return pd.DataFrame() # 返回空 DataFrame

# 打印数据范围信息
unique_dates = forecast_data['日期'].unique()
min_datetime = forecast_data['datetime'].min()
max_datetime = forecast_data['datetime'].max()
print(f"预测数据包含的日期: {unique_dates}")
print(f"预测数据时间范围: {min_datetime} 到 {max_datetime}")

# 确定发运节点时间
dispatch_deadline_morning = pd.to_datetime('06:00:00').time() # 早上 6 点发运节点
dispatch_deadline_afternoon = pd.to_datetime('14:00:00').time() # 下午 14 点发运节点
点
print(f"发运节点时间: 早上 {dispatch_deadline_morning}, 下午 {dispatch_deadline_afternoon}")

# 按线路和日期分组
for (route_code, date), group in forecast_data.groupby(['线路编码', '日期']):
    route_info = line_data[line_data['线路编码'] == route_code].iloc[0]
    deadline_time = route_info['发运节点']
    deadline_hour = deadline_time.hour

# 打印该线路的发运节点和货物量
print(f"\n 处理线路 {route_code}, 日期 {date}, 发运节点 {deadline_time}")

```

```

total_packages = group['包裹量'].sum()
print(f'该线路总包裹量: {total_packages}')

# 获取目的场地的站点编号
site_name = route_info['目的场地']
site_number = parse_site_id(site_name)

# 按时间顺序排序
group = group.sort_values('datetime')

# 确定发运节点的日期时间
deadline_str = str(deadline_time)
if len(deadline_str) <= 5: # 如果是时间格式 HH:MM
    deadline_str = deadline_str + ":00"

# 确定日期: 如果发运节点是早上 6 点, 那么对应的日期应该是下一天
deadline_date = date
if deadline_hour < 12: # 早上 6 点
    deadline_date = pd.to_datetime(date) + pd.Timedelta(days=1)
deadline_datetime = pd.to_datetime(f"{deadline_date} {deadline_str}")

print(f'发运截止时间: {deadline_datetime}')

# 用于累积包裹量的变量
accumulated_packages = 0
current_batch = [] # 当前批次的记录

# 处理每个 10 分钟颗粒度的包裹量
for idx, row in group.iterrows():
    current_time = row['datetime']
    current_packages = row['包裹量']

    # 如果当前时间已经超过发运节点, 跳过
    if current_time > deadline_datetime:
        print(f"跳 过    {current_time} , 已 超 过 发 运 截 止 时 间
{deadline_datetime}")
        continue

    # 当前批次添加包裹
    accumulated_packages += current_packages
    current_batch.append({
        'time': current_time,

```

```

        'packages': current_packages
    })

    # 当累积包裹量达到满载量时，生成一个运输需求
    if accumulated_packages >= MAX_LOAD:
        # 发运时间为当前时间
        dispatch_time = current_time

        # 创建运输需求
        shipment_demands.append({
            '线路编码': route_code,
            '日期': date,
            '起始场地': route_info['起始场地'],
            '目的场地': route_info['目的场地'],
            '目的站点编号': site_number,
            '目的站点编号原始': site_name,
            '发运节点': deadline_time,
            '车队编码': route_info['车队编码'],
            '在途时长': route_info['在途时长'],
            '自有变动成本': route_info['自有变动成本'],
            '外部承运商成本': route_info['外部承运商成本'],
            '包裹量': MAX_LOAD,
            '预计发运时间': dispatch_time  # 直接使用达到满载量的时间
        })

    print(f"生成运输需求：发运时间={dispatch_time}，包裹量={MAX_LOAD}")

    # 重置累积器，保留剩余的包裹
    accumulated_packages = accumulated_packages - MAX_LOAD

    # 如果有剩余包裹，开始新的批次
    if accumulated_packages > 0:
        current_batch = [{
            'time': current_time,
            'packages': accumulated_packages
        }]
    else:
        current_batch = []

    # 处理剩余的包裹（不足满载量）
    if accumulated_packages > 0:

```

```

# 对于不足满载量的包裹，在发运节点前 10 分钟发出
dispatch_time = deadline_datetime - pd.Timedelta(minutes=10)

if current_batch:
    # 如果时间允许，使用当前批次的最后一个包裹时间作为发运时间
    last_package_time = current_batch[-1]['time']

    # 检查最后包裹时间是否比发运节点早至少 10 分钟
    if (deadline_datetime - last_package_time).total_seconds() >= 600:
        dispatch_time = last_package_time

# 创建运输需求
shipment_demands.append({
    '线路编码': route_code,
    '日期': date,
    '起始场地': route_info['起始场地'],
    '目的场地': route_info['目的场地'],
    '目的站点编号': site_number,
    '目的站点编号原始': site_name,
    '发运节点': deadline_time,
    '车队编码': route_info['车队编码'],
    '在途时长': route_info['在途时长'],
    '自有变动成本': route_info['自有变动成本'],
    '外部承运商成本': route_info['外部承运商成本'],
    '包裹量': accumulated_packages,
    '预计发运时间': dispatch_time # 设置为最后包裹时间或发运节点前
10 分钟
})

print(f"生成剩余运输需求：发运时间={dispatch_time}，包裹量={accumulated_packages}")

# 转换为 DataFrame 并返回
demands_df = pd.DataFrame(shipment_demands)

# 打印运输需求的时间范围
if not demands_df.empty:
    min_dispatch = demands_df['预计发运时间'].min()
    max_dispatch = demands_df['预计发运时间'].max()
    print(f"\n生成的运输需求发运时间范围：{min_dispatch} 到 {max_dispatch}")

# 验证时间分布

```

```

        for hour in range(24):
            hour_count = len(demands_df[demands_df['预计发运时间'].dt.hour == hour])
            if hour_count > 0:
                print(f"{hour}点发运的需求数量: {hour_count}")

    return demands_df

# 生成基本运输需求
shipment_demands = generate_shipment_demands(forecast_detail)
print(f"生成了 {len(shipment_demands)} 个基本运输需求")

# 4. 生成串点组合
print("生成串点组合...")

# 检查两个站点是否可串点
def can_string_sites(site_num1, site_num2, time_code1=None, time_code2=None):
    """
    检查两个站点是否可以串点
    site_num1, site_num2: 站点编号（整数或字符串格式的"站点 XX"）
    time_code1, time_code2: 发运节点代码，如"0600"或"1400"，若提供则会检查时间匹
    配
    """
    if site_num1 is None or site_num2 is None:
        return False

    # 检查发运节点是否匹配
    if time_code1 is not None and time_code2 is not None:
        # 提取发运节点时间代码
        if time_code1 != time_code2:
            return False # 发运节点不同，不能串点

    # 首先检查整数编号是否在兼容对中
    if isinstance(site_num1, int) and isinstance(site_num2, int):
        result = (site_num1, site_num2) in compatible_sites

    # 如果没找到，尝试转换为字符串格式再检查
    if not result:
        str_site1 = f"站点{site_num1}"
        str_site2 = f"站点{site_num2}"
        result = (str_site1, str_site2) in compatible_sites

    if result:

```

```

        print(f"站点 {site_num1} 和 {site_num2} 可以串点")
        return True

    return False

# 主调度算法
def schedule_with_string_points(transport_demands, string_points_dict, fleet_dict):
    """
    使用串点策略的调度算法

    Args:
        transport_demands: 运输需求 DataFrame
        string_points_dict: 可串点站点字典
        fleet_dict: 车队自有车数量字典

    Returns:
        final_schedule: 最终调度结果列表
    """
    print("开始执行调度算法...")
    final_schedule = []

    # 打印部分需求示例，检查数据格式
    print("\n 运输需求示例:")
    print(transport_demands[['线路编码', '目的场地', '目的站点编号', '预计发运时间', '自有变动成本', '外部承运商成本']].head(5))

    # 添加发运节点时间代码到运输需求中
    transport_demands_with_time_code = transport_demands.copy()
    transport_demands_with_time_code['发运节点代码'] =
transport_demands_with_time_code['线路编码'].apply(
        lambda x: x.split(' ')[-1] if isinstance(x, str) and ' ' in x else None
    )

    # 按起始场地、车队和发运节点分组
    grouped_demands = transport_demands_with_time_code.groupby(['起始场地', '车队编码', '发运节点'])
    total_groups = len(grouped_demands)
    current_group = 0

    string_count = 0 # 记录成功串点的数量
    total_cost_saving = 0 # 记录总成本节约

```

```

for (start_site, fleet, deadline), group in grouped_demands:
    current_group += 1
    print(f"\n 处理分组  {current_group}/{total_groups}:  {start_site},  {fleet},
{deadline}")

    demands_list = group.to_dict('records')
    print(f"该分组包含 {len(demands_list)} 个需求")

    # 打印该分组的站点编号
    site_nums = [d['目的站点编号'] for d in demands_list]
    print(f"该分组的站点编号: {site_nums}")

    # 检查站点兼容性
    for i in range(len(site_nums)):
        for j in range(i+1, len(site_nums)):
            if site_nums[i] is not None and site_nums[j] is not None:
                time_code1 = demands_list[i]['发运节点代码']
                time_code2 = demands_list[j]['发运节点代码']
                is_compatible = can_string_sites(site_nums[i], site_nums[j],
time_code1, time_code2)
                if is_compatible:
                    print(f"站点 {site_nums[i]} 和 {site_nums[j]} 可串点 (节点
代码: {time_code1}, {time_code2})")

    # 如果分组中需求数量小于 2，直接调度
    if len(demands_list) < 2:
        for demand in demands_list:
            # 使用预先计算的发运时间
            final_schedule.append({
                '线路编码': demand['线路编码'],
                '日期': demand['日期'],
                '预计发运时间': demand['预计发运时间'], # 使用预先计算的发
运时间

                '串点': False,
                '串点组合': None,
                '车队编码': demand['车队编码'],
                '自有变动成本': demand['自有变动成本'],
                '外部承运商成本': demand['外部承运商成本'],
                '包裹量': demand['包裹量'],
                '在途时长': demand.get('在途时长', 2), # 默认 2 小时
                '发运节点代码': demand['发运节点代码'] # 保存发运节点代码
            })

```

```

        continue

    # 生成所有可能的二联串点组合和三联串点组合
    potential_combinations = []

    # 生成二联串点组合
    for i in range(len(demands_list)):
        demand1 = demands_list[i]
        site1 = demand1['目的站点编号']
        time_code1 = demand1['发运节点代码']

        for j in range(i+1, len(demands_list)):
            demand2 = demands_list[j]
            site2 = demand2['目的站点编号']
            time_code2 = demand2['发运节点代码']

            # 检查是否可串点 - 包括站点串点关系和发运节点一致性
            if can_string_sites(site1, site2, time_code1, time_code2):
                # 检查包裹量是否超限
                total_packages = demand1['包裹量'] + demand2['包裹量']
                if total_packages <= MAX_LOAD:
                    # 计算串点成本和不串点成本
                    # 串点成本 = max(需求 1 自有成本, 需求 2 自有成本)
                    # 不串点成本 = 需求 1 自有成本 + 需求 2 自有成本
                    string_cost = max(demand1['自有变动成本'], demand2['自有
变动成本'])

                    no_string_cost = demand1['自有变动成本'] + demand2['自有
变动成本']

                    cost_saving = no_string_cost - string_cost

                    # 可以串点 - 保存组合信息
                    dispatch_time1 = demand1['预计发运时间']
                    dispatch_time2 = demand2['预计发运时间']
                    later_time = max(dispatch_time1, dispatch_time2)

                    potential_combinations.append({
                        '类型': '二联串点',
                        '需求列表': [demand1, demand2],
                        '发运时间': later_time,
                        '总包裹量': total_packages,
                        '成本节约': cost_saving,
                        '在途时长': max(demand1.get('在途时长', 2),

```

```

demand2.get('在途时长', 2)),
                                '发运节点代码': time_code1  # 保存公共的发运节点代
码

                                })

# 生成三联串点组合
for i in range(len(demands_list)):
    demand1 = demands_list[i]
    site1 = demand1['目的站点编号']
    time_code1 = demand1['发运节点代码']

    for j in range(i+1, len(demands_list)):
        demand2 = demands_list[j]
        site2 = demand2['目的站点编号']
        time_code2 = demand2['发运节点代码']

        # 检查第一对站点是否可串点 - 包括发运节点匹配
        if not can_string_sites(site1, site2, time_code1, time_code2):
            continue

        for k in range(j+1, len(demands_list)):
            demand3 = demands_list[k]
            site3 = demand3['目的站点编号']
            time_code3 = demand3['发运节点代码']

            # 检查所有站点对是否都可串点，包括发运节点匹配
            if (can_string_sites(site1, site3, time_code1, time_code3) and
                can_string_sites(site2, site3, time_code2, time_code3)):

                # 检查包裹量是否超限
                total_packages = demand1['包裹量'] + demand2['包裹量'] +
demand3['包裹量']

                if total_packages <= MAX_LOAD:
                    # 计算串点成本和不串点成本
                    string_cost = max(demand1['自有变动成本'], demand2['
自有变动成本'], demand3['自有变动成本'])
                    no_string_cost = demand1['自有变动成本'] + demand2['
自有变动成本'] + demand3['自有变动成本']
                    cost_saving = no_string_cost - string_cost

                    # 可以三联串点 - 保存组合信息
                    dispatch_time1 = demand1['预计发运时间']

```

```

dispatch_time2 = demand2['预计发运时间']
dispatch_time3 = demand3['预计发运时间']
latest_time    =    max(dispatch_time1,    dispatch_time2,
dispatch_time3)

        potential_combinations.append({
            '类型': '三联串点',
            '需求列表': [demand1, demand2, demand3],
            '发运时间': latest_time,
            '总包裹量': total_packages,
            '成本节约': cost_saving,
            '在途时长': max(demand1.get('在途时长', 2),
demand2.get('在途时长', 2), demand3.get('在途时长', 2)),
            '发运节点代码': time_code1    # 保存公共的发运节
点代码
        })

# 根据成本节约对潜在组合进行排序（降序）
potential_combinations.sort(key=lambda x: x['成本节约'], reverse=True)

# 跟踪已分配的需求
allocated_demands = set()

# 逐个处理成本节约最大的组合
for combo in potential_combinations:
    # 检查该组合中的需求是否已被分配
    if any(id(demand) in allocated_demands for demand in combo['需求列表']):
        continue

    # 可以串点 - 标记这些需求已被分配
    for demand in combo['需求列表']:
        allocated_demands.add(id(demand))

    # 创建串点组合名称
    string_combo = "-".join([demand['目的场地'] for demand in combo['需求列表
']]

    # 累计成本节约
    total_cost_saving += combo['成本节约']

    print(f'成功{combo['类型']}: {string_combo}, 发运时间: {combo['发运时间
']], 成本节约: {combo['成本节约']], 节点代码: {combo['发运节点代码']})

```

```

string_count += len(combo['需求列表']) # 记录串点需求数量

# 为这些需求添加调度记录
for demand in combo['需求列表']:
    final_schedule.append({
        '线路编码': demand['线路编码'],
        '日期': demand['日期'],
        '预计发运时间': combo['发运时间'], # 使用组合的发运时间
        '串点': True,
        '串点组合': string_combo,
        '车队编码': demand['车队编码'],
        '自有变动成本': demand['自有变动成本'],
        '外部承运商成本': demand['外部承运商成本'],
        '包裹量': demand['包裹量'],
        '在途时长': combo['在途时长'], # 使用组合的最大在途时长
        '发运节点代码': demand['发运节点代码'] # 保存发运节点代码
    })

# 处理剩余未串点的需求
for demand in demands_list:
    if id(demand) not in allocated_demands:
        final_schedule.append({
            '线路编码': demand['线路编码'],
            '日期': demand['日期'],
            '预计发运时间': demand['预计发运时间'], # 使用预先计算的发
运时间
            '串点': False,
            '串点组合': None,
            '车队编码': demand['车队编码'],
            '自有变动成本': demand['自有变动成本'],
            '外部承运商成本': demand['外部承运商成本'],
            '包裹量': demand['包裹量'],
            '在途时长': demand.get('在途时长', 2), # 默认 2 小时
            '发运节点代码': demand['发运节点代码'] # 保存发运节点代码
        })

print(f"\n 总共有 {string_count} 个需求被串点")
print(f'总成本节约: {total_cost_saving}')
return final_schedule

# 执行调度算法
print("执行调度算法...")

```

```

final_schedule      =      schedule_with_string_points(shipment_demands,      chain_map,
fleet_size_map)

# 计算指标
string_count = sum(1 for item in final_schedule if item['串点'])
total_count = len(final_schedule)
string_rate = string_count / total_count if total_count > 0 else 0
print(f'串点率: {string_rate*100:.2f}% ({string_count}/{total_count})')

# 输出结果
print("输出结果...")

# 为每个车队的每辆车编号并创建车辆使用时间跟踪
vehicles = {}
for fleet, count in fleet_size_map.items():
    vehicles[fleet] = []
    for i in range(count):
        # 创建车辆对象，包含编号和下一次可用时间
        vehicles[fleet].append({
            "id": f"{fleet}_车辆{i+1}",
            "next_available_time": datetime.min # 使用 datetime.min 作为初始可用时
间
        })

print(f'各车队自有车数量: {fleet_size_map}')
print(f'初始化车辆数量: {sum(len(v) for v in vehicles.values())}')

# 查找可用车辆（按日期）
def find_available_vehicle_by_date(date, fleet_code, required_departure_time, route_time,
vehicles_by_date):
    """
    根据日期查找可用车辆
    date: 日期
    fleet_code: 车队编码
    required_departure_time: 需要的发车时间
    route_time: 路线时间（小时）
    vehicles_by_date: 按日期和车队组织的车辆字典
    返回: 可用车辆，如果没有可用车辆则返回 None
    """
    # 检查日期和车队是否存在
    if date not in vehicles_by_date or fleet_code not in vehicles_by_date[date]:
        return None

```

```

# 查找该车队中第一辆可用的车辆
for vehicle in vehicles_by_date[date][fleet_code]:
    if vehicle['next_available_time'] <= required_departure_time:
        # 返回找到的车辆，但不更新其下次可用时间（将在决策函数中更新）
        return vehicle

# 如果没有可用车辆，返回 None
return None

# 决策函数：确定是否使用标准容器以及使用哪辆车
def decide_assignment_and_container(packages, dispatch_time, route_time, date, fleet_code,
vehicles_by_date):
    """
    决定是否使用标准容器以及使用哪辆车

    Args:
        packages: 包裹量
        dispatch_time: 发运时间
        route_time: 路线时间（小时）
        date: 日期
        fleet_code: 车队编码
        vehicles_by_date: 按日期和车队组织的车辆字典

    Returns:
        (vehicle_id, use_container): 分配的车辆 ID 和是否使用标准容器
    """
    # 检查是否可以使用标准容器
    can_use_container = (packages <= CONTAINER_CAPACITY)

    # 尝试方案 1（不用容器）
    v1 = None
    turnaround_normal = LOADING_TIME + route_time * 2 + UNLOADING_TIME
    if date in vehicles_by_date and fleet_code in vehicles_by_date[date]:
        for vehicle in vehicles_by_date[date][fleet_code]:
            if vehicle['next_available_time'] <= dispatch_time:
                # 找到可用的普通方案车辆
                v1 = vehicle
                break

    # 尝试方案 2（用容器）
    v2 = None

```

```

    if can_use_container:
        turnaround_container = CONTAINER_LOADING_HOURS + route_time * 2 +
CONTAINER_UNLOADING_HOURS
        if date in vehicles_by_date and fleet_code in vehicles_by_date[date]:
            for vehicle in vehicles_by_date[date][fleet_code]:
                if vehicle['next_available_time'] <= dispatch_time:
                    # 找到可用的容器方案车辆
                    v2 = vehicle
                    break

# 决策逻辑
if v1 is not None and (not can_use_container or v2 is None):
    # 只有普通方案可行，或者无法使用容器
    vehicle_id = v1['id']
    use_container = False
    # 更新车辆下次可用时间
    end_time = dispatch_time + timedelta(hours=turnaround_normal)
    v1['next_available_time'] = end_time
elif v1 is None and can_use_container and v2 is not None:
    # 只有容器方案可行
    vehicle_id = v2['id']
    use_container = True
    # 更新车辆下次可用时间
    end_time = dispatch_time + timedelta(hours=turnaround_container)
    v2['next_available_time'] = end_time
elif v1 is not None and can_use_container and v2 is not None:
    # 两种方案都可行，优先使用容器提高周转率
    vehicle_id = v2['id']
    use_container = True
    # 更新车辆下次可用时间
    end_time = dispatch_time + timedelta(hours=turnaround_container)
    v2['next_available_time'] = end_time
else:
    # 自有车辆都不可用，使用外部车
    vehicle_id = "外部"
    use_container = False # 外部车默认不使用容器

return vehicle_id, use_container

# 计算任务完成时间（结束时间）- 考虑容器
def calculate_end_time(dispatch_datetime, route_time, use_container=False):
    """

```

```

    计算任务结束时间
    dispatch_datetime: 发车时间(datetime 对象)
    route_time: 在途时间 (小时)
    use_container: 是否使用标准容器
    返回: 完成时间(datetime 对象)
    """

    # 选择适当的装卸时间
    loading_time = CONTAINER_LOADING_HOURS if use_container else
LOADING_TIME
    unloading_time = CONTAINER_UNLOADING_HOURS if use_container else
UNLOADING_TIME

    # 计算总耗时 (小时): 装车时间 + 在途时间 (往返) + 卸车时间
    total_hours = loading_time + route_time * 2 + unloading_time

    # 计算结束时间 (自动处理跨天情况)
    end_time = dispatch_datetime + timedelta(hours=total_hours)

    return end_time

def calculate_route_time(item):
    """
    计算路线时间
    item: 排程项目
    返回: 路线时间 (小时)
    """

    # 如果项目中有运行时间, 则使用该值
    if '运行时间' in item:
        return item['运行时间']

    # 如果项目中有在途时长, 则使用该值
    if '在途时长' in item:
        return item['在途时长']

    # 否则使用默认值 2 小时
    return 2

# 分配车辆的函数 - 整合容器使用决策
def distribute_vehicles_and_containers(schedule_data, available_vehicles, fleet_capacity):
    """
    为各运输任务分配车辆 - 结合优先级调度算法, 优先处理重要时段

```

Args:

schedule_data: 调度日程表
available_vehicles: 可用车辆信息
fleet_capacity: 各车队容量信息

Returns:

添加了车辆信息和容器使用决策的调度表

"""

创建调度数据副本保留原始信息

enhanced_schedule = []

for entry in schedule_data:

 enhanced_entry = entry.copy() # 复制所有字段包括串点信息
 enhanced_schedule.append(enhanced_entry)

车辆初始化 - 按日期和车队组织

daily_fleet_vehicles = {}

提取所有排程中的日期

schedule_dates = {entry['日期'] for entry in enhanced_schedule}

为每个日期初始化车辆

for date in schedule_dates:

 daily_fleet_vehicles[date] = {}

 for fleet in available_vehicles:

 # 为每个车队创建新的车辆列表，以避免跨日期共享车辆状态

 daily_fleet_vehicles[date][fleet] = []

 for i in range(fleet_capacity[fleet]):

 daily_fleet_vehicles[date][fleet].append({

 "id": f"{fleet}-{i+1}",

 "fleet": fleet,

 "next_available_time": datetime.combine(date, dt_time(0, 0)), #

从当天 0 点开始可用

 "trips": 0 # 跟踪车辆的行程数

 })

按日期分组

date_groups = {}

for entry in enhanced_schedule:

 entry_date = entry['日期']

 if entry_date not in date_groups:

 date_groups[entry_date] = []

 date_groups[entry_date].append(entry)

```

# 计算每个需求的自有车与外部车成本差异及优先级
for date, date_items in date_groups.items():
    for entry in date_items:
        # 计算使用自有车的成本
        own_cost = entry['自有变动成本']
        # 计算使用外部车的成本
        outsource_cost = entry['外部承运商成本']
        # 成本差异（外部 - 自有）
        entry['成本差异'] = outsource_cost - own_cost

        # 按时间段统计包裹量 - 用于确定高峰期
        hour_value = entry['预计发运时间'].hour if hasattr(entry['预计发运时间'],
'hour') else pd.to_datetime(entry['预计发运时间']).hour
        entry['小时值'] = hour_value

# 统计每个日期每个小时的包裹总量，用于识别高峰期
hourly_package_volume = {}
for date, date_items in date_groups.items():
    if date not in hourly_package_volume:
        hourly_package_volume[date] = {}

    for entry in date_items:
        hour_value = entry['小时值']
        if hour_value not in hourly_package_volume[date]:
            hourly_package_volume[date][hour_value] = 0
        hourly_package_volume[date][hour_value] += entry['包裹量']

# 确定每个日期的高峰期时段（包裹量排序前 50%）
peak_hours = {}
for date, hour_volumes in hourly_package_volume.items():
    # 按包裹量排序时间段
    sorted_hours = sorted(hour_volumes.items(), key=lambda x: x[1], reverse=True)
    # 取前 50%作为高峰期
    peak_count = max(1, len(sorted_hours) // 2)
    peak_hours[date] = [time_slot[0] for time_slot in sorted_hours[:peak_count]]
    print(f"日期 {date} 的高峰期时段: {peak_hours[date]}")

# 总成本统计
total_fixed_cost = 0
total_variable_cost = 0
total_outsource_cost = 0

```

```

# 记录车辆使用情况
vehicle_usage_log = {}

# 处理每一天的排程
for date, date_items in date_groups.items():
    # 计算该日期的自有车固定成本
    fleet_vehicle_count = sum(fleet_capacity.values())
    daily_fixed_cost = fleet_vehicle_count * VEHICLE_DAILY_COST
    total_fixed_cost += daily_fixed_cost

# 标记高峰期条目
for entry in date_items:
    entry['is_peak_hour'] = entry['小时值'] in peak_hours.get(date, [])

# 按车队分组
fleet_items = {}
for entry in date_items:
    entry_fleet = entry['车队编码']
    if entry_fleet not in fleet_items:
        fleet_items[entry_fleet] = []
    fleet_items[entry_fleet].append(entry)

# 处理每个车队的排程
for fleet, fleet_entries in fleet_items.items():
    # 修改排序方式：先按高峰期标记，再按包裹量，再按成本差异，再按发
    # 运时间
    fleet_entries.sort(key=lambda x: (
        x['is_peak_hour'],
        x['包裹量'],
        x['成本差异'],
        x['预计发运时间']
    ), reverse=True)

# 找出同组串点记录
chaining_groups = {}
for entry in fleet_entries:
    if entry.get('串点', False) and entry.get('串点组合'):
        group_name = entry['串点组合']
        if group_name not in chaining_groups:
            chaining_groups[group_name] = []
        chaining_groups[group_name].append(entry)

```

```
# 先处理串点记录
processed_entries = set()
for group_name, group_entries in chaining_groups.items():
    if len(group_entries) > 0:
        # 计算路线时间
        group_route_time = max(calculate_route_time(entry) for entry in
group_entries)

        # 使用相同的发运时间
        dispatch_time = max(entry['预计发运时间'] for entry in
group_entries)

        # 计算总成本差异
        total_cost_diff = sum(entry.get('成本差异', 0) for entry in
group_entries)

        # 计算组内总包裹量
        total_group_packages = sum(entry['包裹量'] for entry in
group_entries)

        # 是否高峰期串点
        is_peak_chaining = any(entry['is_peak_hour'] for entry in
group_entries)

        # 获取车队编码
        fleet_code = group_entries[0]['车队编码']

        # 分配车辆和选择容器策略
        vehicle_id, use_container = decide_assignment_and_container(
            total_group_packages, dispatch_time, group_route_time, date,
fleet_code, daily_fleet_vehicles
        )

        # 为所有串点项目分配相同的车辆和容器决策
        if vehicle_id != "外部":
            # 使用自有车
            assignment_status = '已分配'

            # 记录车辆使用情况
            if vehicle_id not in vehicle_usage_log:
                vehicle_usage_log[vehicle_id] = []
```

```

# 添加使用记录
vehicle_usage_log[vehicle_id].append({
    '日期': date,
    '发运时间': dispatch_time,
    '线路': group_name,
    '包裹量': total_group_packages,
    '在途时长': group_route_time,
    '是高峰期': is_peak_chaining,
    '使用容器': use_container
})

# 更新车辆行程数
fleet_code = vehicle_id.split('-')[0]
vehicle_index = int(vehicle_id.split('-')[1]) - 1
daily_fleet_vehicles[date][fleet_code][vehicle_index]['trips'] +=

1

# 累计自有变动成本
total_variable_cost += max(entry.get('自有变动成本', 0) for
entry in group_entries)
else:
# 使用外部车
assignment_status = '未分配'

# 累计外部成本
total_outsource_cost += max(entry.get('外部承运商成本', 0)
for entry in group_entries)

# 为所有串点项目分配相同的车辆信息和容器决策
for entry in group_entries:
    entry['车辆 ID'] = vehicle_id
    entry['车辆分配状态'] = assignment_status
    entry['使用标准容器'] = use_container
    processed_entries.add(id(entry))

# 处理剩余未串点或未处理的项目
i = 0
while i < len(fleet_entries):
    entry = fleet_entries[i]

# 跳过已处理的项目

```

```

        if id(entry) in processed_entries:
            i += 1
            continue

        # 为未处理项目分配车辆与容器
        entry_fleet = entry['车队编码']
        route_time = calculate_route_time(entry)
        dispatch_time = entry['预计发运时间']

        # 决策车辆分配与容器使用
        vehicle_id, use_container = decide_assignment_and_container(
            entry['包裹量'], dispatch_time, route_time, date, entry_fleet,
daily_fleet_vehicles
        )

        if vehicle_id != "外部":
            # 使用自有车
            entry['车辆 ID'] = vehicle_id
            entry['车辆分配状态'] = '已分配'
            entry['使用标准容器'] = use_container

            # 记录车辆使用情况
            if vehicle_id not in vehicle_usage_log:
                vehicle_usage_log[vehicle_id] = []

            # 添加使用记录
            vehicle_usage_log[vehicle_id].append({
                '日期': date,
                '发运时间': dispatch_time,
                '线路': entry['线路编码'],
                '包裹量': entry['包裹量'],
                '在途时长': route_time,
                '是高峰期': entry['is_peak_hour'],
                '使用容器': use_container
            })

            # 更新车辆行程数
            fleet_code = vehicle_id.split('-')[0]
            vehicle_index = int(vehicle_id.split('-')[1]) - 1
            daily_fleet_vehicles[date][fleet_code][vehicle_index]['trips'] += 1

            # 累计自有变动成本

```

```

        total_variable_cost += entry.get('自有变动成本', 0)
    else:
        # 高峰期尝试调整低峰期任务
        if entry['is_peak_hour'] and entry['包裹量'] > MAX_LOAD * 0.7:
            # 尝试查找可调整的低峰期车辆
            adjustment_success = False

            # 查找已分配的低峰期任务
            for v_id, trips in vehicle_usage_log.items():
                if not v_id.startswith(entry_fleet) or v_id == "外部":
                    continue

                for trip_idx, trip in enumerate(trips):
                    if trip.get('是高峰期', False): # 跳过高峰期任务
                        continue

                    # 检查车辆可用性
                    vehicle_index = int(v_id.split('-')[1]) - 1
                    target_vehicle =
daily_fleet_vehicles[date][entry_fleet][vehicle_index]

                    if target_vehicle['next_available_time'] <=
dispatch_time:

                        # 此车可直接用于当前任务
                        continue

                    # 检查低峰期任务时间
                    low_peak_time = trip['发运时间']
                    if low_peak_time < dispatch_time:
                        continue # 已经发生的任务无法调整

                    # 判断容器可用性
                    can_use_container = (entry['包裹量'] <=
CONTAINER_CAPACITY)

                    # 调整分配
                    print(f"高峰期优化: 低峰期任务 {trip['线路']} 改
为外部车, 自有车分配给高峰期任务 {entry['线路编码']}")

                    # 更新当前任务车辆信息
                    entry['车辆 ID'] = v_id
                    entry['车辆分配状态'] = '已分配'

```

```

        entry['使用标准容器'] = can_use_container

        # 记录高峰期任务
        vehicle_usage_log[v_id].append({
            '日期': date,
            '发运时间': dispatch_time,
            '线路': entry['线路编码'],
            '包裹量': entry['包裹量'],
            '在途时长': route_time,
            '是高峰期': True,
            '使用容器': can_use_container
        })

        # 更新成本
        total_variable_cost += entry.get('自有变动成本', 0)

        # 更新车辆可用时间
        loading_time = CONTAINER_LOADING_HOURS
    if can_use_container else LOADING_TIME
        unloading_time =
CONTAINER_UNLOADING_HOURS if can_use_container else UNLOADING_TIME
        target_vehicle['next_available_time'] = dispatch_time
+ timedelta(hours=loading_time + route_time * 2 + unloading_time)

        adjustment_success = True
        break

    if adjustment_success:
        break

    # 使用外部车（如果高峰期优化失败或非高峰期）
    if 'adjustment_success' not in locals() or not adjustment_success:
        entry['车辆 ID'] = "外部"
        entry['车辆分配状态'] = '未分配'
        entry['使用标准容器'] = False # 外部车默认不使用容器

    # 累计外部成本
    total_outsource_cost += entry.get('外部承运商成本', 0)

    i += 1

# 计算车辆周转率

```

```

fleet_turnover_stats = {}
for fleet, fleet_vehicles in available_vehicles.items():
    fleet_trips = sum(1 for v_id, trips in vehicle_usage_log.items()
                      if v_id.startswith(fleet) and v_id != "外部"
                      for _ in trips)
    turnover_rate = fleet_trips / fleet_capacity[fleet] if fleet_capacity[fleet] > 0 else 0
    fleet_turnover_stats[fleet] = turnover_rate

# 计算总成本
total_cost = total_fixed_cost + total_variable_cost + total_outsource_cost

print("\n=== 成本统计 ===")
print(f"自有车固定成本: {total_fixed_cost}")
print(f"自有车变动成本: {total_variable_cost}")
print(f"外部承运商成本: {total_outsource_cost}")
print(f"总成本: {total_cost}")

print("\n=== 车辆周转率 ===")
for fleet, rate in fleet_turnover_stats.items():
    print(f"车队 {fleet} 周转率: {rate:.2f}")

# 平均周转率
avg_turnover = sum(fleet_turnover_stats.values()) / len(fleet_turnover_stats) if
fleet_turnover_stats else 0
print(f"平均周转率: {avg_turnover:.2f}")

# 统计容器使用情况
container_usage_count = sum(1 for entry in enhanced_schedule if entry.get('使用标准容
器', False))
total_tasks = len(enhanced_schedule)
container_usage_rate = container_usage_count / total_tasks * 100 if total_tasks > 0 else
0
print(f"\n=== 容器使用统计 ===")
print(f"容器使用次数: {container_usage_count}")
print(f"容器使用率: {container_usage_rate:.2f}%")

# 统计高峰期资源使用
peak_self_vehicle_usage = sum(1 for _, trips in vehicle_usage_log.items()
                               for trip in trips if trip.get('是高峰期', False) and _ != "外
部")
non_peak_self_vehicle_usage = sum(1 for _, trips in vehicle_usage_log.items()
                                   for trip in trips if not trip.get('是高峰期', False) and

```

```

_ != "外部")
    peak_external_vehicle_usage = sum(1 for entry in enhanced_schedule
                                     if entry.get('is_peak_hour', False) and entry.get('车辆
ID') == "外部")

    print("\n=== 高峰期资源使用统计 ===")
    print(f'高峰期自有车辆使用次数: {peak_self_vehicle_usage}')
    print(f'非高峰期自有车辆使用次数: {non_peak_self_vehicle_usage}')
    print(f'高峰期外部车辆使用次数: {peak_external_vehicle_usage}')

    if peak_self_vehicle_usage + peak_external_vehicle_usage > 0:
        peak_utilization_rate = peak_self_vehicle_usage / (peak_self_vehicle_usage +
peak_external_vehicle_usage) * 100
        print(f'高峰期自有车辆使用率: {peak_utilization_rate:.2f}%')

    return enhanced_schedule

# 调用函数分配车辆并更新最终调度结果
merged_schedules = distribute_vehicles_and_containers(final_schedule, vehicles,
fleet_size_map)

# 创建最终结果 DataFrame
result_df = pd.DataFrame(merged_schedules)

# 确保结果中包含串点信息
if '串点' not in result_df.columns and '串点组合' not in result_df.columns:
    # 如果没有串点信息, 可能是 assign_vehicles 没有保留, 重新从 final_schedule 获取
    串点信息 = {(item['线路编码'], item['日期'], item['预计发运时间']):
                {'串点': item.get('串点', False), '串点组合': item.get('串点组合', None)}}
                for item in final_schedule}

    # 添加串点信息到结果
    result_df['串点'] = result_df.apply(
        lambda row: 串点信息.get((row['线路编码'], row['日期'], row['预计发运时间']),
{}).get('串点', False),
        axis=1
    )
    result_df['串点组合'] = result_df.apply(
        lambda row: 串点信息.get((row['线路编码'], row['日期'], row['预计发运时间']),
{}).get('串点组合', None),
        axis=1
    )

```

```
print(f"\n 原始调度记录数: {len(result_df)}")

# 确保日期列是日期类型并格式化
if not pd.api.types.is_datetime64_dtype(result_df['日期']):
    result_df['日期'] = pd.to_datetime(result_df['日期'])
result_df['日期_格式化'] = result_df['日期'].dt.strftime('%Y-%m-%d')

# 确保时间列是时间类型并格式化
if not pd.api.types.is_datetime64_dtype(result_df['预计发运时间']):
    result_df['预计发运时间'] = pd.to_datetime(result_df['预计发运时间'])
result_df['发运时间_格式化'] = result_df['预计发运时间'].dt.strftime('%Y-%m-%d %H:%M:%S')

# 创建最终结果列表, 修正 05:50:00 时间的日期问题
final_results = []

# 处理每个串点组合
有效串点组合 = set(result_df['串点组合'].dropna())
print(f'识别到 {len(有效串点组合)} 个有效串点组合')

# 首先修正日期问题
修正后记录 = []
for index, row in result_df.iterrows():
    new_row = row.copy()

    # 提取时间部分
    发运时间 = row['发运时间_格式化']
    时间部分 = 发运时间.split(' ')[1] if ' ' in 发运时间 else 发运时间

    # 判断是否需要修正日期
    早上时间 = int(时间部分.split(':')[0]) < 12 if 时间部分 and ':' in 时间部分 else False
    是 0600 线路 = "0600" in str(row['线路编码'])
    日期 = row['日期_格式化']

    if 早上时间 and 是 0600 线路 and "2024-12-15" in 日期:
        # 将 12-15 修改为 12-16
        修正日期 = "2024-12-16"
        new_row['日期_格式化'] = 修正日期
        print(f'修正日期: {日期} -> {修正日期}, 时间: {时间部分}, 线路: {row['线路编码']}')

```

```

# 添加简化的时间字段
new_row['时间部分'] = 时间部分
修正后记录.append(new_row)

# 将修正后的记录转换回 DataFrame
修正后 df = pd.DataFrame(修正后记录)

# 按串点组合重新分组，确保即使日期被修正也能正确合并
串点组_分组 = {}
for 组合名 in 有效串点组合:
    # 获取该组合的所有记录
    组合记录 = 修正后 df[修正后 df['串点组合'] == 组合名]

    # 为每个组合创建唯一键：修正后日期+时间+车辆 ID
    for (日期, 时间, 车辆 ID), 组内记录 in 组合记录.groupby(['日期_格式化', '时间部分', '车辆 ID']):
        组键 = f"{组合名}_{日期}_{时间}_{车辆 ID}"
        if 组键 not in 串点组_分组:
            串点组_分组[组键] = []

        # 添加所有匹配记录
        for _, record in 组内记录.iterrows():
            串点组_分组[组键].append(record)

# 处理每个分组，合并线路编码
for 组键, 组内记录_列表 in 串点组_分组.items():
    # 合并线路编码
    线路列表 = [记录['线路编码'] for 记录 in 组内记录_列表]
    拼接线路 = " + ".join(线路列表)

    # 获取其他信息（从第一条记录）
    记录 0 = 组内记录_列表[0]

    # 添加到最终结果
    final_results.append({
        '线路编码': 拼接线路,
        '日期': 记录 0['日期_格式化'],
        '预计发运时间': 记录 0['时间部分'],
        '车辆 ID': 记录 0['车辆 ID'],
        '串点': True,
        '串点组合': 记录 0['串点组合'],
        '使用标准容器': 记录 0.get('使用标准容器', False) # 添加容器使用字段
    })

```

```

    })

# 添加非串点记录
for _, row in 修正后 df[修正后 df['串点组合'].isna()].iterrows():
    final_results.append({
        '线路编码': row['线路编码'],
        '日期': row['日期_格式化'],
        '预计发运时间': row['时间部分'],
        '车辆 ID': row['车辆 ID'],
        '串点': False,
        '串点组合': None,
        '使用标准容器': row.get('使用标准容器', False) # 添加容器使用字段
    })

# 创建最终 DataFrame
final_df = pd.DataFrame(final_results)
print(f'合并后记录数: {len(final_df)}')
print(f'串点记录数: {len(final_df[final_df['串点']])}')

# 确保使用标准容器列中的 NA 值被替换为 False
final_df['使用标准容器'] = final_df['使用标准容器'].fillna(False)
# 确保车辆 ID 列中的空值被替换为"外部"
final_df['车辆 ID'] = final_df['车辆 ID'].fillna("外部")
print(f'使用标准容器记录数: {len(final_df[final_df['使用标准容器'] == True])}')

# 分析串点组合情况
print("\n=== 串点组合分析 ===")
串点组合计数 = {}
for 组合名 in 有效串点组合:
    组合内站点数 = len(组合名.split('-'))
    if 组合内站点数 not in 串点组合计数:
        串点组合计数[组合内站点数] = 0
    串点组合计数[组合内站点数] += 1

for 站点数, 数量 in sorted(串点组合计数.items()):
    print(f'{站点数}站点串联组合数量: {数量}')

# 如果存在三站点串联，输出示例
if 串点组合计数.get(3, 0) > 0:
    三站点组合 = [组合 for 组合 in 有效串点组合 if len(组合.split('-')) == 3]
    print(f'\n三站点串联组合示例: {三站点组合[:3]}')

```

```

# 输出这些三站点组合的明细
for 组合 in 三站点组合[:3]:
    组合记录 = 修正后 df[修正后 df['串点组合'] == 组合]
    print(f"\n 组合 '{组合}' 的记录:")
    print(组合记录[['线路编码', '日期_格式化', '时间部分', '车辆 ID', '使用标准容器
']].to_string())

# 更新结果 DataFrame
result_df = final_df[['线路编码', '日期', '预计发运时间', '车辆 ID', '使用标准容器']]
# 确保车辆 ID 列中的空值被替换为"外部"
result_df['车辆 ID'] = result_df['车辆 ID'].fillna("外部")

# 统计标准容器使用情况
容器使用记录数 = len(result_df[result_df['使用标准容器'] == True])
自有车使用记录数 = len(result_df[result_df['车辆 ID'] != "外部"])
容器使用占比 = 容器使用记录数 / len(result_df) * 100 if len(result_df) > 0 else 0
自有车容器使用占比 = 容器使用记录数 / 自有车使用记录数 * 100 if 自有车使用记录数 > 0 else 0

print(f"\n=== 标准容器使用统计 ===")
print(f'总记录数: {len(result_df)}')
print(f'使用标准容器记录数: {容器使用记录数}')
print(f'容器使用占比: {容器使用占比:.2f}%')
print(f'自有车使用标准容器占比: {自有车容器使用占比:.2f}%')

# 保存结果，处理文件可能被占用的情况
try:
    # 确保外部车辆 ID 在保存到 Excel 前显示为"外部"
    result_df_save = result_df.copy()
    # 将空值替换为"外部"
    result_df_save['车辆 ID'] = result_df_save['车辆 ID'].fillna("外部")

    result_df_save.to_excel(solution_path, index=False)
    print(f"\n 调度完成，结果已保存到 {solution_path}")
except PermissionError:
    # 如果文件被占用，尝试保存到备用文件
    backup_file = r"C:\Users\16878\Desktop\D 题\结果表 4_backup.xlsx"
    result_df_save.to_excel(backup_file, index=False)
    print(f"\n 原文件被占用，结果已保存到备用文件: {backup_file}")
except Exception as e:
    # 其他错误情况
    print(f"\n 保存文件时出错: {str(e)}")

```

```
# 尝试保存到当前目录
current_dir_file = "结果表 4.xlsx"
result_df.save_to_excel(current_dir_file, index=False)
print(f'结果已保存到当前目录: {current_dir_file}')

print(f'总耗时: {time.time() - execution_start:.2f} 秒')
print(f'生成了 {len(result_df)} 条调度记录')

# 查找并输出特定线路的调度结果
target_routes = ["场地 3 - 站点 83 - 0600", "场地 3 - 站点 83 - 1400"]
print("\n 指定线路的调度结果:")
for route in target_routes:
    route_results = result_df[result_df['线路编码'].str.contains(route, regex=False)]
    print(f'\n 线路 {route} 的调度结果:')
    if len(route_results) > 0:
        print(route_results)
    else:
        print("未找到该线路的调度记录")

# 显示部分结果示例
print("\n 部分调度结果示例:")
print(result_df.head(10))

# 计算最终的串点率
串点记录数 = len(final_df[final_df['串点'] == True])
总记录数 = len(final_df)
串点率 = 串点记录数 / 总记录数 * 100 if 总记录数 > 0 else 0

print("\n=== 最终串点统计 ===")
print(f'串点记录数: {串点记录数}')
print(f'总记录数: {总记录数}')
print(f'串点率: {串点率:.2f}%')

# 计算平均车辆利用率
# 首先创建一个新的列，用于统计，将空值替换为"外部"
车辆_ID_计算列 = result_df['车辆 ID'].fillna("外部")
# 计算自有车辆记录（排除外部车辆和空值）
自有车辆记录 = result_df[车辆_ID_计算列 != "外部"]
自有车辆记录数 = len(自有车辆记录)
自有车辆占比 = 自有车辆记录数 / 总记录数 * 100 if 总记录数 > 0 else 0

print(f'自有车辆记录数: {自有车辆记录数}')
```

```

print(f'自有车辆利用率: {自有车辆占比:.2f}%')

# 输出外部车辆和空值统计，用于调试
外部车辆记录数 = len(result_df[车辆 ID_计算列 == "外部"])
print(f'外部车辆记录数（包括空值）: {外部车辆记录数}')
真实外部车辆记录数 = len(result_df[result_df[车辆 ID_] == "外部"])
空值车辆记录数 = 外部车辆记录数 - 真实外部车辆记录数
print(f'其中空值记录数: {空值车辆记录数}')

# 估算总成本和车辆均包裹
# 统计具有包裹量信息的运输需求
需求总包裹量 = sum(shipment_demands['包裹量']) if '包裹量' in shipment_demands.columns else "未知"
# 使用一致的方法计算车辆数（使用车辆 ID_计算列）
总车辆数 = len(自有车辆记录[车辆 ID_].unique()) + len(result_df[车辆 ID_计算列 == "外部"])
车辆均包裹 = 需求总包裹量 / 总车辆数 if isinstance(需求总包裹量, (int, float)) and 总车辆数 > 0 else "未知"

print("\n=== 资源利用统计 ===")
print(f'总包裹量: {需求总包裹量}')
print(f'总车辆数: {总车辆数}')
print(f'车辆均包裹: {车辆均包裹}')

# 计算和显示最终总成本
print("\n=== 最终方案总成本统计 ===")

def compute_total_resource_usage(solution_data):
    """计算方案的总资源使用情况与成本"""
    # 初始化统计数据
    fixed_expenses = 0
    operational_expenses = 0
    third_party_expenses = 0

    # 累计自有车固定成本
    used_vehicle_ids = {record.get('车辆 ID', "") for record in solution_data
                        if record.get('车辆 ID', "") != "外部" and record.get('车辆 ID', "")
is not None}
    fleet_usage = {}

    for vehicle_id in used_vehicle_ids:
        if '-' in vehicle_id:

```

```

        fleet_code = vehicle_id.split('-')[0]
        if fleet_code not in fleet_usage:
            fleet_usage[fleet_code] = 0
        fleet_usage[fleet_code] += 1

# 自有车每天固定成本
for fleet, count in fleet_usage.items():
    fixed_expenses += count * VEHICLE_DAILY_COST

# 防止重复计算
processed_records = set()

# 累计变动成本
for record in solution_data:
    # 跳过已处理的串点项
    if id(record) in processed_records:
        continue

    if record.get('串点', False) and record.get('串点组合'):
        # 找出同一串点组的所有项
        chain_group = record['串点组合']
        group_records = [r for r in solution_data
                        if r.get('串点组合') == chain_group
                        and str(r.get('预计发运时间', '')) == str(record['预计发运时
间'])]

        # 标记为已处理
        for r in group_records:
            processed_records.add(id(r))

# 串点项目的成本为各项成本的最大值
if record.get('车辆 ID', '') != "外部" and record.get('车辆 ID', '') is not None:
    # 自有车成本
    if '自有变动成本' in record:
        operational_expenses += max((r.get('自有变动成本', 0) for r in
group_records if '自有变动成本' in r), default=0)
    else:
        # 外部车成本
        if '外部承运商成本' in record:
            third_party_expenses += max((r.get('外部承运商成本', 0) for r in
group_records if '外部承运商成本' in r), default=0)
        else:

```

```

        # 非串点项目
        if record.get('车辆 ID', "") != "外部" and record.get('车辆 ID', "") is not None:
            # 自有车成本
            if '自有变动成本' in record:
                operational_expenses += record.get('自有变动成本', 0)
            else:
                # 外部车成本
                if '外部承运商成本' in record:
                    third_party_expenses += record.get('外部承运商成本', 0)

    total_expenses = fixed_expenses + operational_expenses + third_party_expenses
    return total_expenses, fixed_expenses, operational_expenses, third_party_expenses

# 主函数
def main():
    """主程序入口"""
    try:
        # 计算最终方案总成本
        solution_total_cost, fixed_cost, variable_cost, external_cost =
compute_total_resource_usage(merged_schedules)

        print(f"最终方案总成本: {solution_total_cost}")

        # 统计自有车使用数量
        owned_vehicles_count = len(set(item.get('车辆 ID', "") for item in merged_schedules
            if item.get('车辆 ID', "") != "外部" and item.get('车辆
ID', "") is not None))

        # 统计外部车使用数量
        external_vehicles_count = len(set(str(item.get('线路编码', "")) + '_' + str(item.get('预
计发运时间', ""))
            for item in merged_schedules if item.get('车辆 ID',
") == "外部" or item.get('车辆 ID', "") is None))

        # 显示成本构成
        print(f"自有车固定成本: {fixed_cost} (使用 {owned_vehicles_count} 辆自有
车)")
        print(f"自有车变动成本: {variable_cost}")
        print(f"外部承运商成本: {external_cost} (使用 {external_vehicles_count} 辆外
部车)")
        print(f"总车辆数量: {owned_vehicles_count + external_vehicles_count} 辆")

```

```

# 计算成本比例
total_cost = fixed_cost + variable_cost + external_cost
if total_cost > 0:
    print(f"自有车固定成本占比: {fixed_cost/total_cost*100:.2f}%")
    print(f"自有车变动成本占比: {variable_cost/total_cost*100:.2f}%")
    print(f"外部承运商成本占比: {external_cost/total_cost*100:.2f}%")

# 计算平均每车成本
print(f" 平 均 每 车 成 本 : {total_cost/(owned_vehicles_count +
external_vehicles_count):.2f}")

except Exception as e:
    print(f"计算总成本时出错: {str(e)}")
    import traceback
    traceback.print_exc()

# 执行主函数
if __name__ == "__main__":
    main()

```

不同扰动方式代码

```

#!/usr/bin/env python
# -*- coding: utf-8 -*-

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
import time
from datetime import datetime, timedelta
import seaborn as sns
from tqdm import tqdm
import warnings
warnings.filterwarnings('ignore')

# 设置随机种子以确保结果的可重复性
np.random.seed(42)

# 配置中文字体显示
plt.rcParams['font.sans-serif'] = ['SimHei'] # 正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 正常显示负号

```

```
# 定义全局常量
MAX_CONTAINER_LOAD = 2000 # 标准容器的最大负载
VEHICLE_MAX_LOAD = 1000 # 普通车辆的最大负载
PREFER_CONTAINER = True # 是否优先使用标准容器

# 结果文件路径
result_file_path = "结果表/结果表 5.xlsx"
os.makedirs(os.path.dirname(result_file_path), exist_ok=True)

# 创建图表保存目录
charts_directory = os.path.join(os.path.dirname(result_file_path), "扰动分析图表")
if not os.path.exists(charts_directory):
    os.makedirs(charts_directory)
    print(f'图表保存目录已创建: {charts_directory}')

def fetch_data():
    """数据加载"""
    # 读取路线信息
    route_data_file = "附件 1.xlsx"
    route_data = pd.read_excel(route_data_file)
    print(f'路线信息加载完成: {len(route_data)}条记录")

    # 读取串点信息
    string_data_file = "附件 4.xlsx"
    string_data = pd.read_excel(string_data_file)
    print(f'串点信息加载完成: {len(string_data)}条记录")

    # 读取车队规模信息
    fleet_data_file = "附件 5.xlsx"
    try:
        fleet_info = pd.read_excel(fleet_data_file)
        print(f'附件 5 列名: {fleet_info.columns.tolist()}")
        # 使用默认车队数据，因为附件 5 似乎不包含车队信息
        fleet_sizes = {
            'A': 50, # A 车队的车辆数量
            'B': 50, # B 车队的车辆数量
            'C': 50, # C 车队的车辆数量
            'D': 50, # D 车队的车辆数量
            'E': 50 # E 车队的车辆数量
        }
    except Exception as e:
        print(f'车队规模信息加载失败，使用默认值: {e}')
```

```
        fleet_sizes = {
            'A': 50, # A 车队的车辆数量
            'B': 50, # B 车队的车辆数量
            'C': 50, # C 车队的车辆数量
            'D': 50, # D 车队的车辆数量
            'E': 50  # E 车队的车辆数量
        }
    print(f"使用的车队规模信息: {fleet_sizes}")

    # 读取总包裹预测结果
    total_prediction_file = "结果表/结果表 1.xlsx"
    total_pred_data = pd.read_excel(total_prediction_file)
    print(f"总包裹预测结果加载完成: {len(total_pred_data)}条记录")

    # 读取详细预测结果
    detailed_prediction_file = "结果表/结果表 2.xlsx"
    detailed_pred_data = pd.read_excel(detailed_prediction_file)
    print(f"详细预测结果加载完成: {len(detailed_pred_data)}条记录")

    # 检查之前的结果是否存在
    previous_results_file = "结果表/结果表 3.xlsx"
    try:
        previous_results = pd.read_excel(previous_results_file)
        print(f"问题 3 结果加载完成: {len(previous_results)}条记录")

        # 检查是否需要创建 datetime 列
        if 'datetime' not in previous_results.columns:
            # 尝试从预计发运时间创建 datetime 列
            if '预计发运时间' in previous_results.columns:
                try:
                    previous_results['datetime'] = pd.to_datetime(previous_results['预计
发运时间'])

                    print("datetime 列创建成功")
                except:
                    print("无法从预计发运时间创建 datetime 列")
            else:
                print("未找到问题 3 结果文件，将从头开始")
                previous_results = None
        else:
            return route_data, string_data, fleet_sizes, total_pred_data, detailed_pred_data,
previous_results
```

```

def data_preprocessing(route_data, string_data, fleet_sizes, total_pred_data,
detailed_pred_data, previous_results):
    """数据预处理"""
    # 打印一些列名，以便了解数据结构
    print("路线数据列名:", route_data.columns.tolist())
    print("车队规模数据示例:", fleet_sizes)

    # 检查路线编码格式，确定如何提取设施 ID
    sample_route_codes = route_data['线路编码'].iloc[:5].tolist()
    print("线路编码示例:", sample_route_codes)

    # 直接处理线路编码，从中提取场地和站点信息
    formatted_route_names = []

    for _, row in route_data.iterrows():
        line_code = row['线路编码']
        if isinstance(line_code, str):
            if '-' in line_code:
                # 直接使用原有的线路编码
                formatted_route = line_code
            else:
                # 尝试解析其他格式
                formatted_route = line_code
        else:
            # 默认格式化
            formatted_route = f"未知-未知-0000"

        formatted_route_names.append(formatted_route)

    route_data['格式化路线名称'] = formatted_route_names
    print("生成格式化路线名称样例:", route_data['格式化路线名称'].iloc[:5].tolist())

    # 如果有目标路线，尝试匹配
    if 'target_routes' in globals():
        target_formatted_routes = []
        for route in target_routes:
            # 尝试与格式化路线名称匹配
            matched = False
            for fmt_route in formatted_route_names:
                if route in fmt_route or fmt_route in route:
                    target_formatted_routes.append(fmt_route)
                    matched = True

```

```

        break

    if not matched:
        print(f"警告: 无法匹配目标路线 {route}")

    if target_formatted_routes:
        print(f"成功匹配 {len(target_formatted_routes)}/{len(target_routes)} 条目标
路线")
        globals()['target_routes'] = target_formatted_routes

    return route_data, string_data, fleet_sizes, total_pred_data, detailed_pred_data,
previous_results

# 新的扰动模型
def random_error_model(df, package_col, std_dev=0.1):
    """随机误差模型: 通过正态分布随机函数模拟传感器噪声和随机波动"""
    df_copy = df.copy()
    noise = np.random.normal(0, std_dev, size=len(df_copy))
    df_copy['扰动包裹量'] = df_copy[package_col] * (1 + noise)
    return df_copy

def systematic_error_model(df, package_col, bias_factor=0.2):
    """系统性误差模型: 通过固定百分比对所有数据进行偏移, 模拟预测模型整体偏差"""
    df_copy = df.copy()
    df_copy['扰动包裹量'] = df_copy[package_col] * (1 + bias_factor)
    return df_copy

def periodic_error_model(df, package_col, amp=0.2, cycle=144, phase_shift=0):
    """周期性误差模型: 使用正弦函数模拟一天内的周期性波动"""
    df_copy = df.copy()
    # 计算时间索引 (以 10 分钟为单位)
    min_time = df_copy['datetime'].min()
    time_idx = np.array([(dt - min_time).total_seconds() / 600 for dt in df_copy['datetime']])
    periodic_err = amp * np.sin(2 * np.pi * time_idx / cycle + phase_shift)
    df_copy['扰动包裹量'] = df_copy[package_col] * (1 + periodic_err)
    return df_copy

def time_shift_error_model(df, package_col, shift_steps=6):
    """时间偏移误差模型: 通过时间序列数据的偏移操作模拟高峰期时间的预测偏差"""
    df_copy = df.copy()

```

```

# 按线路和日期分组进行偏移
shifted_vals = {}

for (route, date), group in df_copy.groupby(['线路编码', df_copy['datetime'].dt.date]):
    # 按时间排序
    group = group.sort_values('datetime')
    # 偏移包裹量
    shifted = group[package_col].shift(shift_steps,
fill_value=group[package_col].mean())

    # 存储偏移后的值
    for idx, val in zip(group.index, shifted):
        shifted_vals[idx] = val

# 应用偏移后的值
df_copy['扰动包裹量'] = df_copy.index.map(lambda idx: shifted_vals.get(idx,
df_copy.loc[idx, package_col]))
return df_copy

def burst_error_model(df, package_col, burst_multiplier=3.0, burst_start_time=None,
burst_length=6):
    """突发性误差模型：在指定时间段内将包裹量乘以一个倍数，模拟突发事件"""
    df_copy = df.copy()

    # 如果没有指定突发开始时间，使用 8:00 和 17:00 作为默认时间
    if burst_start_time is None:
        # 获取数据中的日期
        dates = df_copy['datetime'].dt.date.unique()

        # 对每个日期的 8:00 和 17:00 应用突发扰动
        for date in dates:
            # 早高峰：8:00 - 9:00
            morning_start = pd.Timestamp(date) + pd.Timedelta(hours=8)
            morning_end = morning_start + pd.Timedelta(minutes=burst_length*10)

            # 晚高峰：17:00 - 18:00
            evening_start = pd.Timestamp(date) + pd.Timedelta(hours=17)
            evening_end = evening_start + pd.Timedelta(minutes=burst_length*10)

            # 应用突发扰动
            morning_mask = (df_copy['datetime'] >= morning_start) &
(df_copy['datetime'] < morning_end)

```

```

        evening_mask = (df_copy['datetime'] >= evening_start) & (df_copy['datetime']
< evening_end)

        df_copy.loc[morning_mask, '扰动包裹量'] = df_copy.loc[morning_mask,
package_col] * burst_multiplier
        df_copy.loc[evening_mask, '扰动包裹量'] = df_copy.loc[evening_mask,
package_col] * burst_multiplier
    else:
        # 使用指定的突发开始时间
        burst_end = burst_start_time + pd.Timedelta(minutes=burst_length*10)
        burst_mask = (df_copy['datetime'] >= burst_start_time) & (df_copy['datetime'] <
burst_end)
        df_copy.loc[burst_mask, '扰动包裹量'] = df_copy.loc[burst_mask, package_col] *
burst_multiplier

    # 对未设置的时间段应用原始包裹量
    mask = df_copy['扰动包裹量'].isna()
    df_copy.loc[mask, '扰动包裹量'] = df_copy.loc[mask, package_col]

    return df_copy

def monte_carlo_error_model(df, package_col, num_simulations=100, std_range=(0.05, 0.3),
systematic_range=(-0.2, 0.2)):
    """蒙特卡洛模拟随机扰动：通过多次随机模拟，生成更真实的扰动场景

    Args:
        df: 原始数据框
        package_col: 包裹量列名
        num_simulations: 模拟次数
        std_range: 随机误差标准差范围
        systematic_range: 系统性误差范围

    Returns:
        具有扰动包裹量的数据框
    """
    df_copy = df.copy()

    # 初始化结果数组
    all_simulations = np.zeros((len(df_copy), num_simulations))

    # 为每个路线生成随机扰动参数
    unique_routes = df_copy['线路编码'].unique()

```

```

route_params = {}

for route in unique_routes:
    # 为每条路线随机生成一组扰动参数，调低参数幅度
    route_params[route] = {
        'std': np.random.uniform(std_range[0], std_range[1] * 0.6), # 降低随机误差
        'bias': np.random.uniform(systematic_range[0] * 0.7, systematic_range[1] *
        0.7), # 降低系统误差幅度
        'has_burst': np.random.random() < 0.15, # 降低突发事件概率
        'burst_factor': np.random.uniform(1.3, 2.5) if np.random.random() < 0.15 else
        1.0 # 降低突发事件倍数
    }

# 进行多次模拟
print(f'执行蒙特卡洛模拟，共{num_simulations}次')
for sim in tqdm(range(num_simulations)):
    # 对每个时间点模拟包裹量
    for i, (idx, row) in enumerate(df_copy.iterrows()):
        route = row['线路编码']
        params = route_params[route]

        # 原始包裹量
        base_value = row[package_col]

        # 应用随机误差
        random_error = np.random.normal(0, params['std'])

        # 应用系统性误差
        systematic_error = params['bias']

        # 组合误差
        combined_error = 1 + random_error + systematic_error

        # 应用突发事件（如果有）
        if params['has_burst']:
            # 检查是否在早晚高峰
            hour = row['datetime'].hour
            if hour in [8, 9, 17, 18]:
                combined_error *= params['burst_factor']

# 确保扰动值非负

```

```

        disturbed_value = max(0, base_value * combined_error)

        # 存储模拟结果
        all_simulations[i, sim] = disturbed_value

# 计算平均扰动包裹量
df_copy['扰动包裹量'] = np.mean(all_simulations, axis=1)

# 计算 95%置信区间
df_copy['扰动下限'] = np.percentile(all_simulations, 2.5, axis=1)
df_copy['扰动上限'] = np.percentile(all_simulations, 97.5, axis=1)
df_copy['扰动方差'] = np.var(all_simulations, axis=1)

# 保存模拟路线参数
route_params_df = pd.DataFrame([
    {
        '线路编码': route,
        '随机误差标准差': params['std'],
        '系统性误差': params['bias'],
        '突发事件概率': 0.15 if params['has_burst'] else 0,
        '突发事件倍数': params['burst_factor'] if params['has_burst'] else 1.0
    }
    for route, params in route_params.items()
])

print(f"蒙特卡洛模拟完成，随机参数统计：")
print(f"随机误差标准差范围：{route_params_df['随机误差标准差'].min():.2f} - {route_params_df['随机误差标准差'].max():.2f}")
print(f"系统性误差范围：{route_params_df['系统性误差'].min():.2f} - {route_params_df['系统性误差'].max():.2f}")
print(f"突发事件路线数量：{sum(route_params_df['突发事件概率'] > 0)}/{len(route_params_df)}")

return df_copy, route_params_df

def run_disturbance_analysis(route_data, string_data, fleet_sizes, total_pred_data,
detailed_pred_data, previous_results, disturbance_params=None, test_mode=False):
    """
    执行扰动分析
    """
    print("\n 开始扰动分析...")

```

```

if detailed_pred_data is None or detailed_pred_data.empty:
    print("详细预测数据为空，无法进行扰动分析")
    return None

# 如果是测试模式，创建一个简单的测试数据集
if test_mode or detailed_pred_data is None or detailed_pred_data.empty:
    print("使用测试模式数据...")
    # 创建测试数据
    data = {
        '线路编码': ['场地 1-站点 1-0600', '场地 1-站点 1-0600', '场地 1-站点 1-1400',
'场地 2-站点 2-0600'],
        '日期': ['2023-01-01', '2023-01-01', '2023-01-01', '2023-01-01'],
        '时间段开始': [60, 120, 180, 240], # 1:00, 2:00, 3:00, 4:00
        '预测货量': [100, 200, 300, 400]
    }
    detailed_pred_data = pd.DataFrame(data)

# 添加 datetime 列
detailed_pred_data['日期'] = pd.to_datetime(detailed_pred_data['日期'])
minutes = detailed_pred_data['时间段开始'].astype(int)
hours = minutes // 60
mins = minutes % 60
time_str = hours.astype(str).str.zfill(2) + ':' + mins.astype(str).str.zfill(2)
detailed_pred_data['datetime'] = pd.to_datetime(detailed_pred_data['日期']
'].dt.strftime('%Y-%m-%d') + ' ' + time_str)

# 确保有 datetime 列
if 'datetime' not in detailed_pred_data.columns:
    try:
        # 确保日期列是日期格式
        if '日期' in detailed_pred_data.columns:
            if not pd.api.types.is_datetime64_dtype(detailed_pred_data['日期']):
                detailed_pred_data['日期'] = pd.to_datetime(detailed_pred_data['日期'])

# 尝试使用不同可能的时间列
time_col = None
for col_name in ['时间段开始', '时间段', '时间', '分钟起始']:
    if col_name in detailed_pred_data.columns:
        time_col = col_name
        break

```

```

        if time_col:
            print(f"使用 '{time_col}' 列作为时间数据")
            try:
                # 如果是整数形式(如 60, 120...)表示分钟数
                if
pd.api.types.is_numeric_dtype(detailed_pred_data[time_col]):
                    minutes = detailed_pred_data[time_col].astype(int)
                    hours = minutes // 60
                    mins = minutes % 60
                    time_str = hours.astype(str).str.zfill(2) + ':' +
mins.astype(str).str.zfill(2)
                    detailed_pred_data['临时时间'] = pd.to_datetime(time_str,
format='%H:%M').dt.time
            else:
                # 尝试直接转换为时间
                detailed_pred_data['临时时间'] =
pd.to_datetime(detailed_pred_data[time_col]).dt.time
            except Exception as e:
                print(f"转换时间数据时出错: {e}")
                print("使用序列索引作为时间")
                # 创建一个序列, 每天 24 小时, 每小时 6 个 10 分钟间隔
                detailed_pred_data['临时时间'] =
pd.Series(range(len(detailed_pred_data)) % 144
                detailed_pred_data['临时时间'] = detailed_pred_data['临时时
间'].apply(
                    lambda x: f"{x // 6:02d}:{(x % 6) * 10:02d}"
                )
                detailed_pred_data['临时时间'] =
pd.to_datetime(detailed_pred_data['临时时间'], format='%H:%M').dt.time

# 合并日期和时间
if '临时时间' in detailed_pred_data.columns:
    date_str = detailed_pred_data['日期'].dt.strftime('%Y-%m-%d')
    time_str = detailed_pred_data['临时时间'].astype(str)
    detailed_pred_data['datetime'] = pd.to_datetime(date_str + ' ' +
time_str)

    detailed_pred_data.drop('临时时间', axis=1, inplace=True)
    print("成功创建 datetime 列")
else:
    # 如果无法创建时间列, 使用日期加序列索引
    base_date = detailed_pred_data['日期'].min()
    detailed_pred_data['datetime'] = pd.date_range(

```

```

        start=base_date,
        periods=len(detailed_pred_data),
        freq='10min'
    )
    print("使用默认日期序列创建 datetime 列")
else:
    # 如果没有日期列，创建一个默认的时间序列
    detailed_pred_data['datetime'] = pd.date_range(
        start='2023-01-01',
        periods=len(detailed_pred_data),
        freq='10min'
    )
    print("使用完全默认序列创建 datetime 列")
except Exception as e:
    print(f"创建 datetime 列失败: {e}")
    # 创建简单的序列索引
    detailed_pred_data['datetime'] = pd.date_range(
        start='2023-01-01',
        periods=len(detailed_pred_data),
        freq='10min'
    )
    print("使用默认序列创建 datetime 列")

# 检查 detailed_pred_data 的列名
print(f"详细预测数据列名: {detailed_pred_data.columns.tolist()}")

# 确定包裹量列的名称
package_col = None
if '预测货量' in detailed_pred_data.columns:
    package_col = '预测货量'
else:
    # 尝试猜测包裹量列
    for col in detailed_pred_data.columns:
        if '货' in col or '包' in col or '量' in col:
            package_col = col
            print(f"猜测包裹量列为: {package_col}")
            break

if package_col is None:
    print("错误: 无法确定包裹量列")
    return None

```

```
print(f'使用列 '{package_col}' 作为包裹量数据")
```

```
# 如果没有提供扰动参数，使用默认扰动参数
```

```
if disturbance_params is None:
```

```
    disturbance_params = [  
        {  
            'type': 'random',  
            'description': '随机误差(±10%)',  
            'std': 0.1  
        },  
        {  
            'type': 'random',  
            'description': '随机误差(±30%)',  
            'std': 0.3  
        },  
        {  
            'type': 'systematic',  
            'description': '系统性高估(+20%)',  
            'bias': 0.2  
        },  
        {  
            'type': 'systematic',  
            'description': '系统性低估(-20%)',  
            'bias': -0.2  
        },  
        {  
            'type': 'periodic',  
            'description': '周期性波动(±20%)',  
            'amplitude': 0.2,  
            'period': 144,  
            'phase': 0  
        },  
        {  
            'type': 'time_shift',  
            'description': '高峰期提前(-60 分钟)',  
            'shift_units': -6  
        },  
        {  
            'type': 'time_shift',  
            'description': '高峰期延后(+60 分钟)',  
            'shift_units': 6  
        }  
    ],
```

```

        {
            'type': 'burst',
            'description': '突发性需求(3 倍,早晚高峰)',
            'burst_factor': 3.0,
            'burst_duration': 6
        },
        {
            'type': 'monte_carlo',
            'description': '蒙特卡洛综合扰动',
            'num_simulations': 100,
            'std_range': (0.05, 0.2), # 降低随机误差上限
            'systematic_range': (-0.15, 0.15) # 降低系统误差范围
        }
    ]

# 创建结果容器
results_per_disturbance = []

# 对于每种扰动类型，应用扰动并分析结果
for disturbance in disturbance_params:
    print(f"\n 应用扰动: {disturbance['description']}")

    # 复制原始数据
    disturbed_data = None

    # 对包裹量应用扰动
    if disturbance['type'] == 'random':
        # 随机误差
        std = disturbance.get('std', 0.1)
        disturbed_data = random_error_model(detailed_pred_data, package_col, std)
    elif disturbance['type'] == 'systematic':
        # 系统性误差
        bias = disturbance.get('bias', 0.2)
        disturbed_data = systematic_error_model(detailed_pred_data, package_col,
bias)
    elif disturbance['type'] == 'periodic':
        # 周期性误差
        amplitude = disturbance.get('amplitude', 0.2)
        period = disturbance.get('period', 144)
        phase = disturbance.get('phase', 0)
        disturbed_data = periodic_error_model(detailed_pred_data, package_col,
amplitude, period, phase)

```

```

elif disturbance['type'] == 'time_shift':
    # 时间偏移误差
    shift_units = disturbance.get('shift_units', 6)
    disturbed_data = time_shift_error_model(detailed_pred_data, package_col,
shift_units)
elif disturbance['type'] == 'burst':
    # 突发性误差
    burst_factor = disturbance.get('burst_factor', 3.0)
    burst_duration = disturbance.get('burst_duration', 6)
    burst_start = disturbance.get('burst_start', None)
    disturbed_data = burst_error_model(detailed_pred_data, package_col,
burst_factor, burst_start, burst_duration)
elif disturbance['type'] == 'monte_carlo':
    # 蒙特卡洛模拟
    num_simulations = disturbance.get('num_simulations', 100)
    std_range = disturbance.get('std_range', (0.05, 0.3))
    systematic_range = disturbance.get('systematic_range', (-0.2, 0.2))
    disturbed_data, route_params_df =
monte_carlo_error_model(detailed_pred_data, package_col,
num_simulations, std_range, systematic_range)
    # 保存路线参数到结果中
    disturbance['route_params_df'] = route_params_df
else:
    print(f"警告: 未知的扰动类型 {disturbance['type']}")
    disturbed_data = detailed_pred_data.copy()
    disturbed_data['扰动包裹量'] = disturbed_data[package_col]

# 确保扰动后的包裹量非负
disturbed_data['扰动包裹量'] = disturbed_data['扰动包裹量'].clip(lower=0)

# 原始数据和扰动后数据的统计比较
original_sum = disturbed_data[package_col].sum()
disturbed_sum = disturbed_data['扰动包裹量'].sum()
percent_change = (disturbed_sum - original_sum) / original_sum * 100 if
original_sum > 0 else 0

print(f"原始总包裹量: {original_sum:.2f}, 扰动后总包裹量:
{disturbed_sum:.2f}, 变化率: {percent_change:.2f}%")

# 运行模拟计算
# 1. 分析不同路线的变化情况

```

```

route_changes = disturbed_data.groupby('线路编码').agg({
    package_col: 'sum',
    '扰动包裹量': 'sum'
})
route_changes['变化量'] = route_changes['扰动包裹量'] -
route_changes[package_col]
route_changes['变化率'] = route_changes['变化量'] / route_changes[package_col] *
100

# 打印变化最大的前 5 条路线
print("\n 变化最大的前 5 条路线:")
top_changes = route_changes.sort_values('变化率', ascending=False).head(5)
print(top_changes)

# 将扰动后的数据替换原始包裹量，用于后续调度分析
disturbed_data_for_scheduling = disturbed_data.copy()
disturbed_data_for_scheduling['原始包裹量'] =
disturbed_data_for_scheduling[package_col] # 保存原始值
disturbed_data_for_scheduling[package_col] = disturbed_data_for_scheduling['扰动包裹量'] # 替换为扰动值

# 计算扰动对调度的影响
# 这里可以调用前面问题的调度算法，但为简化起见，我们采用简单的模拟
# 实际实现中，应该调用完整的调度函数来分析扰动下的调度差异

# 简单的车辆需求估算
original_vehicles_needed = np.ceil(original_sum / VEHICLE_MAX_LOAD)
disturbed_vehicles_needed = np.ceil(disturbed_sum / VEHICLE_MAX_LOAD)
vehicles_change = disturbed_vehicles_needed - original_vehicles_needed

print(f" 原始车辆需求： {original_vehicles_needed}, 扰动后车辆需求：
{disturbed_vehicles_needed}, 差异: {vehicles_change}")

# 添加到结果列表
results_per_disturbance.append({
    'disturbance_type': disturbance['description'],
    'disturbance_params': disturbance,
    'original_sum': original_sum,
    'disturbed_sum': disturbed_sum,
    'percent_change': percent_change,
    'original_vehicles': original_vehicles_needed,
    'disturbed_vehicles': disturbed_vehicles_needed,

```

```

        'vehicles_change': vehicles_change,
        'route_changes': route_changes.reset_index(),
        'disturbed_data': disturbed_data  # 保存扰动后的详细数据，用于可视化
    })

    return results_per_disturbance

def visualize_disturbance_results(results_per_disturbance, charts_dir, sensitivity_df=None):
    """
    可视化扰动分析结果
    """
    if not os.path.exists(charts_dir):
        os.makedirs(charts_dir)
        print(f'创建图表目录: {charts_dir}')

    if not results_per_disturbance:
        print("没有扰动分析结果用于可视化")
        return

    # 1. 绘制扰动前后包裹量对比柱状图
    disturbance_types = []
    original_sums = []
    disturbed_sums = []

    for result in results_per_disturbance:
        disturbance_types.append(result['disturbance_type'])
        original_sums.append(result['original_sum'])
        disturbed_sums.append(result['disturbed_sum'])

    plt.figure(figsize=(10, 6))
    bar_width = 0.35
    index = np.arange(len(disturbance_types))

    plt.bar(index, original_sums, bar_width, label='原始包裹量', color='skyblue')
    plt.bar(index + bar_width, disturbed_sums, bar_width, label='扰动后包裹量',
            color='salmon')

    for i, (orig, dist) in enumerate(zip(original_sums, disturbed_sums)):
        change_pct = (dist - orig) / orig * 100 if orig > 0 else 0
        plt.text(i, orig, f'{orig:.1f}', ha='center', va='bottom')
        plt.text(i + bar_width, dist, f'{dist:.1f}', ha='center', va='bottom')
        plt.text(i + bar_width/2, max(orig, dist) + max(orig, dist)*0.02,

```

```

        f'{change_pct:+.1f}%', ha='center', va='bottom',
        color='green' if change_pct >= 0 else 'red')

plt.xlabel('扰动类型')
plt.ylabel('总包裹量')
plt.title('不同扰动类型对总包裹量的影响')
plt.xticks(index + bar_width / 2, disturbance_types, rotation=45)
plt.legend()
plt.tight_layout()

# 保存图表
plt.savefig(os.path.join(charts_dir, '扰动分析_包裹量对比.png'), dpi=300,
bbox_inches='tight')
plt.close()

# 2. 绘制扰动敏感性雷达图 (使用传入的 sensitivity_df)
if sensitivity_df is not None and not sensitivity_df.empty:
    metrics = [col for col in sensitivity_df.columns if '敏感性' in col]

    if metrics:
        # 设置雷达图
        plt.figure(figsize=(10, 8))

        # 计算雷达图角度
        N = len(metrics)
        angles = np.linspace(0, 2*np.pi, N+1, endpoint=True)

        # 创建子图
        ax = plt.subplot(111, polar=True)

        # 为每种扰动类型绘制雷达线
        for disturbance_type in sensitivity_df['扰动类型'].unique():
            # 获取该扰动类型的敏感性数据
            sensitivities = []
            disturbance_row = sensitivity_df[sensitivity_df['扰动类型'] ==
disturbance_type].iloc[0]

            for metric in metrics:
                # 使用原始敏感性指标的绝对值
                sensitivities.append(abs(disturbance_row[metric]))

        # 闭合雷达图

```

```

sensitivities += sensitivities[:1]

# 绘制该扰动类型的雷达线
ax.plot(angles, sensitivities, linewidth=2, label=disturbance_type)
ax.fill(angles, sensitivities, alpha=0.1)

# 设置雷达图的坐标轴
ax.set_xticks(angles[:-1])
ax.set_xticklabels([metric.replace('敏感性', '') for metric in metrics],
fontsize=12)

# 添加图例，靠右上角更近一点
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.0))

# 保存雷达图
plt.savefig(os.path.join(charts_dir, '扰动分析敏感性雷达图.png'), dpi=300,
bbox_inches='tight')
plt.close()

# 3. 合并绘制不同扰动类型下的路线变化热力图
# 首先找出所有扰动类型下变化率最大的共同路线
common_routes = set()
route_data_by_disturbance = {}

# 第一步：收集所有扰动类型的路线变化数据
for result in results_per_disturbance:
    disturbance_type = result['disturbance_type']
    route_changes = result['route_changes']
    route_data_by_disturbance[disturbance_type] = route_changes

# 选择变化率最大的路线
top_routes = route_changes.sort_values('变化率', ascending=False).head(20)
if common_routes:
    common_routes = common_routes.union(set(top_routes['线路编码']))
else:
    common_routes = set(top_routes['线路编码'])

# 限制路线数量，避免图表过于拥挤
common_routes = list(common_routes)
if len(common_routes) > 15:
    # 找出在所有扰动类型中平均变化率最大的 15 条路线
    all_routes_avg_change = {}

```

```

for route in common_routes:
    changes = []
    for disturbance_type, route_changes in route_data_by_disturbance.items():
        if route in route_changes['线路编码'].values:
            changes.append(abs(route_changes.loc[route_changes['线路编码']
== route, '变化率'].iloc[0]))
        else:
            changes.append(0)
    all_routes_avg_change[route] = sum(changes) / len(changes)

# 选择平均变化率最大的 15 条路线
common_routes = sorted(all_routes_avg_change.keys(), key=lambda x:
all_routes_avg_change[x], reverse=True)[:15]

# 创建合并热力图
if common_routes:
    # 准备热力图数据
    heatmap_data = []

    for disturbance_type, route_changes in route_data_by_disturbance.items():
        row_data = {'扰动类型': disturbance_type}

        for route in common_routes:
            if route in route_changes['线路编码'].values:
                row_data[route] = route_changes.loc[route_changes['线路编码'] ==
route, '变化率'].iloc[0]
            else:
                row_data[route] = 0

        heatmap_data.append(row_data)

# 创建 DataFrame
heatmap_df = pd.DataFrame(heatmap_data)
heatmap_df = heatmap_df.set_index('扰动类型')

# 对线路进行重新排序，按照变化率绝对值的总和排序，避免蒙特卡洛扰动主
导排序
if '蒙特卡洛综合扰动' in heatmap_df.index:
    # 创建一个没有蒙特卡洛扰动的 DataFrame，用于计算排序值
    sort_df = heatmap_df.drop('蒙特卡洛综合扰动', errors='ignore')
    if not sort_df.empty:

```

```

# 按照变化率绝对值的总和排序
route_sum_changes = sort_df.abs().sum().sort_values(ascending=False)
sorted_routes = route_sum_changes.index.tolist()
# 重新排序热力图
heatmap_df = heatmap_df[sorted_routes]

# 绘制热力图
fig = plt.figure(figsize=(20, 8)) # 调整图形大小，使之更宽一些

# 创建主要热力图轴
ax = plt.axes([0.25, 0.1, 0.65, 0.8]) # 左, 下, 宽, 高

hm = sns.heatmap(
    heatmap_df,
    annot=True,
    fmt='.1f',
    cmap='coolwarm',
    center=0,
    linewidths=1.0, # 增加线宽
    linecolor='black', # 设置线颜色为黑色，增加对比度
    annot_kws={"size": 16, "weight": "bold", "color": "black"}, # 增大字体并
加粗
    cbar_kws={
        'label': '变化率(%)',
        'shrink': 0.8,
        'pad': 0.01, # 减少色条与图的距离
        'aspect': 30, # 调整色条宽高比
        'use_gridspec': False # 不使用 gridspec 布局
    },
    ax=ax
)

# 获取色条对象并设置标签字体大小
cbar = hm.collections[0].colorbar
cbar.set_label('变化率 (%)', fontsize=18, fontweight='bold', rotation=270,
labelpad=20)
cbar.ax.yaxis.set_label_position("right")

# 设置色条刻度字体大小与热力图内数字一致
cbar.ax.tick_params(labelsize=16)

# 调整标签样式

```

```

plt.ylabel("", fontsize=18, fontweight='bold') # 移除 y 轴标签文字
plt.xlabel('路线编码', fontsize=18, fontweight='bold')

# 旋转 x 轴标签，使其更易读
plt.xticks(rotation=45, ha='right', fontsize=16)

# 使 y 轴标签字体更大并调整位置
ax.set_yticklabels(heatmap_df.index, fontsize=16, rotation=0, va='center')

# 保存热力图
plt.savefig(os.path.join(charts_dir, '扰动分析_路线热力图_合并.png'), dpi=300,
bbox_inches='tight')
plt.close()

# 4. 仍然绘制单独的热力图，以便查看更多细节
for i, result in enumerate(results_per_disturbance):
    disturbance_type = result['disturbance_type']
    route_changes = result['route_changes']

    # 选择变化率最大的 20 条路线
    top_routes = route_changes.sort_values('变化率', ascending=False).head(20)

    plt.figure(figsize=(12, 8))
    sns.heatmap(
        top_routes[['变化率']].T,
        annot=True,
        fmt='.1f',
        cmap='coolwarm',
        center=0,
        linewidths=.5,
        yticklabels=['变化率(%)'],
        xticklabels=top_routes['线路编码'],
        cbar_kws={'label': '变化率(%)'}
    )

    plt.title(f'{disturbance_type}下各路线包裹量变化率')
    plt.xticks(rotation=90)
    plt.tight_layout()

    # 保存图表
    plt.savefig(os.path.join(charts_dir, f'扰动分析_路线热力图_{i+1}_{disturbance_type}.png'), dpi=300, bbox_inches='tight')

```

```

plt.close()

# 5. 为每个指标绘制所有扰动类型的时间序列对比图
if sensitivity_df is not None and not sensitivity_df.empty:
    # 定义评价指标及其对应的基线值和扰动后值列名
    metrics_mapping = {
        '总包裹量': ('原始包裹量', '扰动后包裹量'),
        '车辆需求': ('原始车辆数', '扰动后车辆数'),
        '运输成本': ('原始成本', '扰动后成本'),
        '自有车周转率': ('原始自有车周转率', '扰动后自有车周转率'),
        '自有车使用率': ('原始自有车使用率', '扰动后自有车使用率'),
        '串点率': ('原始串点率', '扰动后串点率')
    }

    # 为每个指标创建图表
    for metric_name, (orig_col, disturbed_col) in metrics_mapping.items():
        if orig_col in sensitivity_df.columns and disturbed_col in
sensitivity_df.columns:
            plt.figure(figsize=(12, 8))

            # 设置柱状图参数
            num_disturbances = len(sensitivity_df)
            bar_width = 0.35
            x = np.arange(num_disturbances)

            # 原始值和扰动后值
            orig_values = sensitivity_df[orig_col]
            disturbed_values = sensitivity_df[disturbed_col]

            # 绘制柱状图
            plt.bar(x - bar_width/2, orig_values, bar_width, label='原始值',
color='skyblue')
            plt.bar(x + bar_width/2, disturbed_values, bar_width, label='扰动后值',
color='salmon')

            # 添加数据标签
            for i, (orig, dist) in enumerate(zip(orig_values, disturbed_values)):
                change_pct = (dist - orig) / orig * 100 if orig > 0 else 0
                plt.text(i - bar_width/2, orig, f'{orig:.2f}', ha='center', va='bottom',
fontsize=9)
                plt.text(i + bar_width/2, dist, f'{dist:.2f}', ha='center', va='bottom',
fontsize=9)

```

```

plt.text(i, max(orig, dist) + max(orig, dist)*0.05,
         f'{change_pct:+.1f}%', ha='center', va='bottom',
         color='green' if change_pct >= 0 else 'red', fontsize=10,
fontweight='bold')

# 设置图表
plt.xlabel('扰动类型')
plt.ylabel(metric_name)
plt.title(f'不同扰动类型对{metric_name}的影响')
plt.xticks(x, sensitivity_df['扰动类型'], rotation=45, ha='right')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()

# 保存图表
plt.savefig(os.path.join(charts_dir, f'扰动分析_{metric_name}对比.png'),
            dpi=300, bbox_inches='tight')
plt.close()

# 6. 为所有路线创建六张性能指标图表（所有扰动类型在同一图中）
# 找出变化最大的路线
if results_per_disturbance:
    # 分析所有扰动类型下变化最大的路线
    max_change_routes = set()
    for result in results_per_disturbance:
        route_changes = result['route_changes']
        if not route_changes.empty:
            top_routes = route_changes.sort_values('变化率',
            ascending=False).head(3)['线路编码'].tolist()
            max_change_routes.update(top_routes)

# 限制路线数量为最多变化前 3 个
max_change_routes = list(max_change_routes)[:3]

# 对于每个高变化路线，为所有指标创建时间序列图
for route in max_change_routes:
    # 检查该路线在各扰动类型下是否有数据
    route_data_available = False
    for result in results_per_disturbance:
        disturbed_data = result['disturbed_data']
        if route in disturbed_data['线路编码'].values:
            route_data_available = True

```

```

        break

    if not route_data_available:
        continue

    # 为每个指标创建一个图表，包含所有扰动类型的曲线
    # 确定包裹量列名
    package_cols = []
    for result in results_per_disturbance:
        disturbed_data = result['disturbed_data']
        for col in disturbed_data.columns:
            if '货量' in col or '包裹量' in col or '预测' in col:
                package_cols.append(col)
                break

    if not package_cols:
        continue

    package_col = package_cols[0] # 使用第一个找到的包裹量列名

    # 绘制时间序列图
    plt.figure(figsize=(14, 8))

    color_cycle = plt.rcParams['axes.prop_cycle'].by_key()['color']

    # 先绘制原始数据
    original_plotted = False
    for i, result in enumerate(results_per_disturbance):
        disturbed_data = result['disturbed_data']
        route_data = disturbed_data[disturbed_data['线路编码'] == route]

        if not route_data.empty and not original_plotted:
            # 确保数据按时间排序
            route_data = route_data.sort_values('datetime')
            plt.plot(route_data['datetime'], route_data[package_col],
                     label='原始包裹量', color='black', linestyle='-',
linewidth=2)

            original_plotted = True
            break

    # 然后绘制各种扰动类型的数据
    for i, result in enumerate(results_per_disturbance):

```

```

        disturbance_type = result['disturbance_type']
        disturbed_data = result['disturbed_data']
        route_data = disturbed_data[disturbed_data['线路编码'] == route]

        if not route_data.empty:
            # 确保数据按时间排序
            route_data = route_data.sort_values('datetime')
            plt.plot(route_data['datetime'], route_data['扰动包裹量'],
                    label=disturbance_type, color=color_cycle[i %
len(color_cycle)],
                    marker='.', markersize=4)

        # 放大坐标轴标签
        plt.xlabel('时间', fontsize=16, fontweight='bold')
        plt.ylabel('包裹量', fontsize=16, fontweight='bold')

        # 放大坐标轴刻度标签
        plt.xticks(fontsize=16)
        plt.yticks(fontsize=14)

        # 放大图例
        plt.legend(loc='upper right', fontsize=12)
        plt.grid(True, alpha=0.3)

        # 格式化 x 轴日期并增大字体
        fig = plt.gcf()
        fig.autofmt_xdate()
        for label in plt.gca().get_xticklabels():
            label.set_fontsize(16)

        plt.tight_layout()

        # 保存图表
        plt.savefig(os.path.join(charts_dir, f'扰动分析_路线{route}_时间序列.png'),
            dpi=300, bbox_inches='tight')
        plt.close()

    # 可视化蒙特卡洛模拟的置信区间
    for i, result in enumerate(results_per_disturbance):
        if 'disturbance_params' in result and result['disturbance_params'].get('type') ==
'monte_carlo':
            disturbed_data = result['disturbed_data']

```

```

# 提取有置信区间的路线数据
if 'route_params_df' in result['disturbance_params']:
    route_params_df = result['disturbance_params']['route_params_df']

    # 选择突发事件概率最高和随机误差最大的前 3 条路线
    top_burst_routes = route_params_df.sort_values(by='突发事件倍数',
ascending=False).head(3)['线路编码'].tolist()
    top_random_routes = route_params_df.sort_values(by='随机误差标准差',
ascending=False).head(3)['线路编码'].tolist()

    # 合并列表并去重
    top_routes = list(set(top_burst_routes + top_random_routes))

    # 为每条高风险路线绘制置信区间图
    for route in top_routes:
        route_data = disturbed_data[disturbed_data['线路编码'] == route]

        if not route_data.empty:
            plt.figure(figsize=(14, 8))

            # 确保数据按时间排序
            route_data = route_data.sort_values('datetime')

            # 绘制原始值
            plt.plot(route_data['datetime'], route_data['package_col'],
label='原始包裹量', color='black', linestyle='-',
linewidth=2)

            # 绘制平均扰动值
            plt.plot(route_data['datetime'], route_data['扰动包裹量'],
label='平均扰动值', color='red', marker='.',
markersize=4)

            # 绘制 95%置信区间
            plt.fill_between(route_data['datetime'],
route_data['扰动下限'],
route_data['扰动上限'],
color='red', alpha=0.2, label='95%置信区间')

            # 放大坐标轴标签
            plt.xlabel('时间', fontsize=16, fontweight='bold')

```

```

plt.ylabel('包裹量', fontsize=16, fontweight='bold')

# 放大坐标轴刻度标签
plt.xticks(fontsize=16)
plt.yticks(fontsize=14)

# 放大图例
plt.legend(loc='upper right', fontsize=12)
plt.grid(True, alpha=0.3)

# 格式化 x 轴日期并增大字体
fig = plt.gcf()
fig.autofmt_xdate()
for label in plt.gca().get_xticklabels():
    label.set_fontsize(16)

plt.tight_layout()

# 保存图表
plt.savefig(os.path.join(charts_dir, f'蒙特卡洛模拟_{route}_置信区间.png'), dpi=300, bbox_inches='tight')
plt.close()

# 绘制路线参数热力图
plt.figure(figsize=(12, len(route_params_df) * 0.3 + 3))

# 准备热力图数据
heatmap_data = route_params_df.set_index('线路编码')[['随机误差标准差', '系统性误差', '突发事件概率', '突发事件倍数']]

# 标准化数据
normalized_data = (heatmap_data - heatmap_data.min()) / (heatmap_data.max() - heatmap_data.min())

# 绘制热力图
cbar_kws = {'label': '归一化参数值'}
ax = sns.heatmap(normalized_data.sort_values(by='突发事件倍数', ascending=False).head(15),
                  cmap='YlOrRd', annot=heatmap_data.sort_values(by='突发事件倍数', ascending=False).head(15).round(2),
                  fmt='.2f', cbar_kws=cbar_kws, linewidths=0.5)

```

```
plt.title('蒙特卡洛模拟 - 路线扰动参数热力图 (按风险降序)')
plt.tight_layout()

# 保存图表
plt.savefig(os.path.join(charts_dir, f'蒙特卡洛模拟_路线参数热力图.png'), dpi=300, bbox_inches='tight')
plt.close()

print(f'扰动分析图表已保存到目录: {charts_dir}')

def perform_sensitivity_analysis(results_per_disturbance):
    """
    执行敏感性分析，计算各种扰动类型对不同指标的敏感性系数
    """
    if not results_per_disturbance:
        print("没有扰动分析结果进行敏感性分析")
        return None

    print("\n 执行敏感性分析...")

    # 创建敏感性分析 DataFrame
    sensitivity_data = []

    # 针对多个核心指标
    metrics = ['总包裹量', '车辆需求', '运输成本', '自有车周转率', '自有车使用率', '串点率', '容器使用率']

    # 计算每种扰动类型的敏感性系数
    for result in results_per_disturbance:
        disturbance_type = result['disturbance_type']

        # 提取原始值和扰动后值
        original_sum = result['original_sum']
        disturbed_sum = result['disturbed_sum']
        original_vehicles = result['original_vehicles']
        disturbed_vehicles = result['disturbed_vehicles']

        # 简化成本计算 (实际应使用更准确的成本模型)
        original_cost = original_vehicles * 2000 # 假设每辆车固定成本 2000
        disturbed_cost = disturbed_vehicles * 2000

    # 模拟自有车数据
```

```

original_own_vehicles = original_vehicles * 0.8 # 假设 80%为自有车
disturbed_own_vehicles = disturbed_vehicles * 0.8
original_fleet_size = 100 # 假设车队总规模

# 自有车周转率 (平均每辆自有车运送的需求数)
original_turnover = original_own_vehicles / original_fleet_size if
original_fleet_size > 0 else 0
disturbed_turnover = disturbed_own_vehicles / original_fleet_size if
original_fleet_size > 0 else 0

# 自有车使用率 (使用的自有车占总自有车的比例)
original_usage_rate = original_own_vehicles / 50 if 50 > 0 else 0 # 假设每个车
队 50 辆自有车
disturbed_usage_rate = disturbed_own_vehicles / 50 if 50 > 0 else 0

# 串点率 (串点需求占总需求的比例) - 假设数据
original_string_rate = 0.25 # 假设原始串点率 25%
# 扰动可能会影响串点率, 假设按照总包裹量变化的一定比例变化
disturbed_string_rate = original_string_rate * (1 + (disturbed_sum - original_sum) /
original_sum * 0.5)
disturbed_string_rate = max(0, min(1, disturbed_string_rate)) # 确保在 0-1 之间

# 容器使用率 (使用标准容器的需求占总需求的比例) - 假设数据
original_container_rate = 0.3 # 假设原始容器使用率 30%
# 扰动可能会影响容器使用率, 假设按照总包裹量变化的一定比例变化
disturbed_container_rate = original_container_rate * (1 + (disturbed_sum -
original_sum) / original_sum * 0.7)
disturbed_container_rate = max(0, min(1, disturbed_container_rate)) # 确保在
0-1 之间

# 计算敏感性系数 (变化率)
pkg_sensitivity = (disturbed_sum - original_sum) / original_sum * 100 if
original_sum > 0 else 0
vehicle_sensitivity = (disturbed_vehicles - original_vehicles) / original_vehicles *
100 if original_vehicles > 0 else 0
cost_sensitivity = (disturbed_cost - original_cost) / original_cost * 100 if
original_cost > 0 else 0
turnover_sensitivity = (disturbed_turnover - original_turnover) / original_turnover *
100 if original_turnover > 0 else 0
usage_rate_sensitivity = (disturbed_usage_rate - original_usage_rate) /
original_usage_rate * 100 if original_usage_rate > 0 else 0
string_rate_sensitivity = (disturbed_string_rate - original_string_rate) /

```

```

original_string_rate * 100 if original_string_rate > 0 else 0
    container_rate_sensitivity = (disturbed_container_rate - original_container_rate) /
original_container_rate * 100 if original_container_rate > 0 else 0

# 添加到结果
sensitivity_data.append({
    '扰动类型': disturbance_type,
    '总包裹量敏感性': pkg_sensitivity,
    '车辆需求敏感性': vehicle_sensitivity,
    '运输成本敏感性': cost_sensitivity,
    '自有车周转率敏感性': turnover_sensitivity,
    '自有车使用率敏感性': usage_rate_sensitivity,
    '串点率敏感性': string_rate_sensitivity,
    '容器使用率敏感性': container_rate_sensitivity,
    '原始包裹量': original_sum,
    '扰动后包裹量': disturbed_sum,
    '原始车辆数': original_vehicles,
    '扰动后车辆数': disturbed_vehicles,
    '原始成本': original_cost,
    '扰动后成本': disturbed_cost,
    '原始自有车周转率': original_turnover,
    '扰动后自有车周转率': disturbed_turnover,
    '原始自有车使用率': original_usage_rate,
    '扰动后自有车使用率': disturbed_usage_rate,
    '原始串点率': original_string_rate,
    '扰动后串点率': disturbed_string_rate,
    '原始容器使用率': original_container_rate,
    '扰动后容器使用率': disturbed_container_rate
})

# 转换为 DataFrame
sensitivity_df = pd.DataFrame(sensitivity_data)

# 计算平均敏感性
sensitivity_cols = [col for col in sensitivity_df.columns if '敏感性' in col]
avg_sensitivities = sensitivity_df[sensitivity_cols].mean()

print("敏感性分析结果:")
print(sensitivity_df[['扰动类型'] + sensitivity_cols])
print("\n 平均敏感性:")
print(avg_sensitivities)

```

```

return sensitivity_df

def generate_summary_report(results_per_disturbance, sensitivity_df, charts_dir):
    """生成扰动分析综合报告"""
    if not results_per_disturbance or sensitivity_df is None:
        print("没有足够的数据生成报告")
        return

    report_file = os.path.join(charts_dir, "扰动分析综合报告.txt")

    with open(report_file, 'w', encoding='utf-8') as f:
        f.write("=" * 80 + "\n")
        f.write("                                扰动分析综合报告                                \n")
        f.write("=" * 80 + "\n\n")

        # 1. 分析概述
        f.write("分析概述:\n")
        f.write("-" * 40 + "\n")
        f.write(f"本报告分析了{len(results_per_disturbance)}种不同类型的扰动对物流\n系统的\n影响。")
        f.write(f"扰动类型包括：{'， ' + ', '.join([r['disturbance_type'] for r in\nresults_per_disturbance])}")
        f.write("分析指标包括总包裹量、车辆需求、运输成本、自有车周转率、自有\n车使用率、串点率和容器使用率。")

        # 2. 敏感性分析
        f.write("敏感性分析:\n")
        f.write("-" * 40 + "\n")

        # 获取各指标的敏感性排名
        sensitivity_metrics = {
            '总包裹量': '总包裹量敏感性',
            '车辆需求': '车辆需求敏感性',
            '运输成本': '运输成本敏感性',
            '自有车周转率': '自有车周转率敏感性',
            '自有车使用率': '自有车使用率敏感性',
            '串点率': '串点率敏感性',
            '容器使用率': '容器使用率敏感性'
        }

        for metric_name, metric_col in sensitivity_metrics.items():
            ranking = sensitivity_df.sort_values(metric_col, ascending=False)['扰动类型']

```

```
'].tolist()
```

```
    f.write(f"\n{metric_name}敏感性排名 (从高到低):\n")
    for i, dist_type in enumerate(ranking):
        sensitivity = sensitivity_df[sensitivity_df['扰动类型'] ==
dist_type][metric_col].iloc[0]
        f.write(f"    {i+1}. {dist_type}: {sensitivity:.2f}%\n")
```

```
# 3. 关键发现
```

```
f.write("\n 关键发现:\n")
```

```
f.write("-" * 40 + "\n")
```

```
# 找出各指标最敏感的扰动类型
```

```
most_sensitive_types = {}
```

```
for metric_name, metric_col in sensitivity_metrics.items():
```

```
    max_change = sensitivity_df[metric_col].abs().max()
```

```
    max_type = sensitivity_df.loc[sensitivity_df[metric_col].abs().idxmax(), '扰动
类型']
```

```
    most_sensitive_types[metric_name] = (max_type, max_change)
```

```
    f.write(f"{len(most_sensitive_types)}. {metric_name}对{max_type}的变化最
为敏感，变化率达到{max_change:.2f}%。 \n")
```

```
# 找出每种扰动下变化最大的路线
```

```
f.write("\n 各扰动类型下变化最显著的路线:\n")
```

```
for result in results_per_disturbance:
```

```
    disturbance_type = result['disturbance_type']
```

```
    route_changes = result['route_changes']
```

```
    if not route_changes.empty:
```

```
        max_change_idx = route_changes['变化率'].abs().idxmax()
```

```
        max_change_route = route_changes.loc[max_change_idx, '线路编码']
```

```
        max_change_rate = route_changes.loc[max_change_idx, '变化率']
```

```
        f.write(f"    • {disturbance_type}: 路线 {max_change_route}, 变化率
{max_change_rate:.2f}%\n")
```

```
# 4. 管理建议
```

```
f.write("\n 管理建议:\n")
```

```
f.write("-" * 40 + "\n")
```

```
f.write("\n基于敏感性分析结果，建议采取以下措施:\n")
```

```
# 针对各指标的特定建议
cost_max_type = most_sensitive_types['运输成本'][0]
turnover_max_type = most_sensitive_types['自有车周转率'][0]

# 根据敏感性排名提供针对性建议
suggestion_count = 1

# 成本相关建议
if '随机扰动' in cost_max_type.lower():
    f.write(f"{suggestion_count}. 加强预测模型的准确性，降低日常随机波动
对运输成本的影响。\\n")
    suggestion_count += 1
    f.write(f"{suggestion_count}. 增加车辆储备以应对随机成本波动，尤其在
高成本敏感性路线上。\\n")
    suggestion_count += 1

    if '周期' in cost_max_type.lower():
        f.write(f"{suggestion_count}. 针对周期性成本波动，调整排班模式，在预
期高峰期优化车辆配置。\\n")
        suggestion_count += 1
        f.write(f"{suggestion_count}. 建立周期性成本优化机制，根据历史数据预
测并提前调整资源分配。\\n")
        suggestion_count += 1

    if '突发' in cost_max_type.lower():
        f.write(f"{suggestion_count}. 设立突发情况应急预案，在特定时段预留足
够的车辆资源以控制成本。\\n")
        suggestion_count += 1

# 周转率相关建议
f.write(f"{suggestion_count}. 针对{turnover_max_type}制定特殊的车辆调度策
略，提高自有车周转率。\\n")
suggestion_count += 1

# 串点率相关建议
string_max_type = most_sensitive_types['串点率'][0]
f.write(f"{suggestion_count}. 在{string_max_type}情况下优化串点方案，确保串
点效率不受扰动影响。\\n")
suggestion_count += 1

# 容器使用率相关建议
```

```

        container_max_type = most_sensitive_types['容器使用率'][0]
        f.write(f"{suggestion_count}. 根据{container_max_type}下的包裹量变化调整容器使用策略，提高资源利用效率。\\n")
        suggestion_count += 1

    # 通用建议
    f.write(f"{suggestion_count}. 对高敏感性路线实施特别监控，优先保障其资源分配。\\n")
    suggestion_count += 1
    f.write(f"{suggestion_count}. 建立动态调度机制，能够根据实时需求变化快速调整运力并优化指标表现。\\n")

# 5. 结论
f.write("\\n 结论:\\n")
f.write("-" * 40 + "\\n")
f.write("通过对不同类型扰动的分析，我们发现物流系统的各项指标对扰动的敏感程度不同:\\n")

# 按敏感性大小排序各指标
avg_sensitivities = {}
for metric_name, metric_col in sensitivity_metrics.items():
    avg_sensitivities[metric_name] = sensitivity_df[metric_col].abs().mean()

sorted_metrics = sorted(avg_sensitivities.items(), key=lambda x: x[1], reverse=True)

f.write("指标敏感性排名 (从高到低):\\n")
for i, (metric, sensitivity) in enumerate(sorted_metrics):
    f.write(f" {i+1}. {metric}: 平均敏感性 {sensitivity:.2f}\\n")

most_sensitive_metric = sorted_metrics[0][0]
most_sensitive_type = most_sensitive_types[most_sensitive_metric][0]

f.write(f"\\n 总体来看， {most_sensitive_metric} 对扰动最为敏感，特别是对 {most_sensitive_type} 的反应最强烈。\\n")
f.write("建立对扰动的预测和应对机制，将有助于提高物流系统的稳定性和各项指标的优化。\\n")
f.write("建议根据本分析结果，针对高敏感性指标和扰动类型制定专项管理措施。\\n")

f.write("\\n" + "=" * 80 + "\\n")
f.write("报告生成时间: " + datetime.now().strftime("%Y-%m-%d %H:%M:%S")) +

```

```

"\n")
        f.write("=" * 80 + "\n")

    print(f'扰动分析综合报告已生成: {report_file}')
    return report_file

def save_results_to_excel(results_per_disturbance, sensitivity_df):
    """将分析结果保存到 Excel 文件"""
    # 创建结果目录
    results_dir = os.path.dirname(result_file_path)
    os.makedirs(results_dir, exist_ok=True)

    # 保存敏感性分析结果
    results_file = None

    try:
        if sensitivity_df is not None:
            sensitivity_file = os.path.join(results_dir, "敏感性分析结果.xlsx")
            try:
                sensitivity_df.to_excel(sensitivity_file, index=False)
                print(f'敏感性分析结果已保存: {sensitivity_file}')
            except PermissionError:
                print(f'权限错误: 无法保存文件 {sensitivity_file}, 该文件可能被其他
程序占用。")
                # 尝试使用不同的文件名
                alt_file = os.path.join(results_dir, f"敏感性分析结果
_{int(time.time())}.xlsx")
                try:
                    sensitivity_df.to_excel(alt_file, index=False)
                    print(f'敏感性分析结果已保存到替代文件: {alt_file}')
                except Exception as e:
                    print(f'保存替代文件也失败: {e}')

    # 保存总体结果
    if results_per_disturbance:
        # 提取关键结果数据
        summary_data = []
        for result in results_per_disturbance:
            summary_data.append({
                '扰动类型': result['disturbance_type'],
                '原始包裹总量': result['original_sum'],
                '扰动后包裹总量': result['disturbed_sum'],

```

```

        '变化率(%)': result['percent_change'],
        '原始车辆需求': result['original_vehicles'],
        '扰动后车辆需求': result['disturbed_vehicles'],
        '车辆变化': result['vehicles_change']
    })

    summary_df = pd.DataFrame(summary_data)

    # 将结果保存到 Excel
    results_file = result_file_path
    try:
        summary_df.to_excel(results_file, index=False)
        print(f"扰动分析总结结果已保存: {results_file}")
    except PermissionError:
        print(f"权限错误: 无法保存文件 {results_file}, 该文件可能被其他程
序占用。")
        # 尝试使用不同的文件名
        alt_file = os.path.join(results_dir, f"结果表 5_{int(time.time())}.xlsx")
        try:
            summary_df.to_excel(alt_file, index=False)
            print(f"扰动分析总结结果已保存到替代文件: {alt_file}")
            results_file = alt_file
        except Exception as e:
            print(f"保存替代文件也失败: {e}")
            results_file = None
    except Exception as e:
        print(f"保存结果时发生未预期的错误: {e}")

    return results_file

def main():
    """主函数，执行扰动分析流程"""
    start_time = time.time()
    print("开始执行问题 4 扰动分析...")

    try:
        # 1. 加载数据
        route_data, string_data, fleet_sizes, total_pred_data, detailed_pred_data,
previous_results = fetch_data()

        # 2. 数据预处理
        route_data, string_data, fleet_sizes, total_pred_data, detailed_pred_data,

```

```

previous_results = data_preprocessing(
    route_data, string_data, fleet_sizes, total_pred_data, detailed_pred_data,
    previous_results
)

# 3. 检查是否有有效路线
valid_routes = set()
if detailed_pred_data is not None and not detailed_pred_data.empty and '线路编码'
in detailed_pred_data.columns:
    valid_routes = set(detailed_pred_data['线路编码'].unique())

if not valid_routes:
    print("警告: 没有找到有效的路线数据, 将使用测试模式")
    test_mode = True
else:
    print(f'找到 {len(valid_routes)} 条有效路线')
    test_mode = False

# 4. 执行扰动分析
disturbance_params = [
    {
        'type': 'random',
        'description': '随机误差(±10%)',
        'std': 0.1
    },
    {
        'type': 'random',
        'description': '随机误差(±30%)',
        'std': 0.3
    },
    {
        'type': 'systematic',
        'description': '系统性高估(+20%)',
        'bias': 0.2
    },
    {
        'type': 'systematic',
        'description': '系统性低估(-20%)',
        'bias': -0.2
    },
    {
        'type': 'periodic',

```

```

        'description': '周期性波动(±20%)',
        'amplitude': 0.2,
        'period': 144,
        'phase': 0
    },
    {
        'type': 'time_shift',
        'description': '高峰期提前(-60 分钟)',
        'shift_units': -6
    },
    {
        'type': 'time_shift',
        'description': '高峰期延后(+60 分钟)',
        'shift_units': 6
    },
    {
        'type': 'burst',
        'description': '突发性需求(3 倍,早晚高峰)',
        'burst_factor': 3.0,
        'burst_duration': 6
    },
    {
        'type': 'monte_carlo',
        'description': '蒙特卡洛综合扰动',
        'num_simulations': 100,
        'std_range': (0.05, 0.2), # 降低随机误差上限
        'systematic_range': (-0.15, 0.15) # 降低系统误差范围
    }
]

```

```

disturbance_results = run_disturbance_analysis(
    route_data, string_data, fleet_sizes, total_pred_data, detailed_pred_data,
    previous_results,
    disturbance_params=disturbance_params, test_mode=test_mode
)

```

```

# 如果结果为空，创建测试结果
if not disturbance_results:
    print("扰动分析没有返回结果，创建测试结果...")
    # 创建一个基本的测试结果
    disturbance_results = [
        {

```

```

        'disturbance_type': '随机扰动(10%)',
        'original_sum': 10000,
        'disturbed_sum': 10500,
        'percent_change': 5.0,
        'original_vehicles': 10,
        'disturbed_vehicles': 11,
        'vehicles_change': 1,
        'route_changes': pd.DataFrame({
            '线路编码': ['路线 A', '路线 B', '路线 C'],
            '包裹量': [1000, 2000, 3000],
            '扰动包裹量': [1100, 2200, 3300],
            '变化量': [100, 200, 300],
            '变化率': [10.0, 10.0, 10.0]
        })
    }
]

# 5. 执行敏感性分析
sensitivity_df = perform_sensitivity_analysis(disturbance_results)

# 6. 可视化分析结果
visualize_disturbance_results(disturbance_results, charts_directory, sensitivity_df)

# 7. 生成综合报告
report_file = generate_summary_report(disturbance_results, sensitivity_df,
charts_directory)

# 8. 保存结果到 Excel
results_file = save_results_to_excel(disturbance_results, sensitivity_df)
# 计算总运行时间
end_time = time.time()
runtime = end_time - start_time
print(f"\n 问题 4 扰动分析完成，总用时: {runtime:.2f} 秒")
print(f"结果保存在: {results_file}")
print(f"分析报告: {report_file}")
print(f"图表目录: {charts_directory}")
except Exception as e:
    print(f"执行过程中发生错误: {e}")
    import traceback
    traceback.print_exc()
if __name__ == "__main__":
    main()

```
